



ESCUELA DE INGENIERÍA DE FUENLABRADA

GRADO EN INGENIERÍA EN
TECNOLOGÍAS DE LA TELECOMUNICACIÓN

TRABAJO FIN DE GRADO

SISTEMA DE REPORTE SEGURO FRENTE A ROOTKITS
DE ESPACIO DE USUARIO

Autor: Noel Rodríguez Pérez

Tutor: Gorka Guardiola Múzquiz

Curso académico 2025-2026



Este trabajo se distribuye bajo los términos de la licencia internacional CC BY-NC-ND International License (Creative Commons Licencia Atribución/Reconocimiento-NoComercial-SinDerivados 4.0 International). Usted es libre de *compartir*: copiar y redistribuir el material en cualquier medio o formato. La licenciante no puede revocar estas libertades en tanto usted siga los términos de la licencia:

- *Atribución*. Usted debe dar crédito de manera adecuada, brindar un enlace a la licencia, e indicar si se han realizado cambios. Puede hacerlo en cualquier forma razonable, pero no de forma tal que sugiera que usted o su uso tienen el apoyo de la licenciante.
- *No Comercial*. Usted no puede hacer uso del material con fines comerciales.
- *Sin Derivadas*. Si remezcla, transforma o crea a partir del material, no podrá distribuir el material modificado.

No hay restricciones adicionales — No puede aplicar términos legales ni medidas tecnológicas que restrinjan legalmente a otras a hacer cualquier uso permitido por la licencia.

No tiene que cumplir con la licencia para elementos del material en el dominio público o cuando su uso esté permitido por una excepción o limitación aplicable. No se dan garantías. La licencia podría no darle todos los permisos que necesita para el uso que tenga previsto. Por ejemplo, otros derechos como publicidad, privacidad, o derechos morales pueden limitar la forma en que utilice el material.

Agradecimientos

En primer lugar, me gustaría agradecer a mi tutor, **Gorka Guardiola Múzquiz**, por su constante apoyo, paciencia y dedicación durante el desarrollo de este proyecto. Todo el conocimiento que me ha transmitido ha sido fundamental no solo para este trabajo, sino para saber el interés real que tengo por este campo de la Ciberseguridad. Gracias por enseñarme tanto.

A todos los docentes de mi carrera, por dotarme de conocimientos que me permiten ahora cerrar este ciclo y empezar mi carrera profesional de la mejor manera posible.

También, a todas las personas de mi entorno, por haberme acompañado durante todo este proceso y mostrado su apoyo a lo largo de este tiempo.

Quiero agradecer especialmente a mi familia y pareja, por darme la educación, motivación y apoyo que me han hecho llegar a donde estoy y ser la persona que soy hoy en día. Y a mi grupo de la universidad, por haber hecho este ciclo mucho más ameno y haberme hecho disfrutar del proceso, incluso en los momentos más difíciles.

Muchas gracias por todo.

*A mi familia, amigos y pareja,
mi mayor apoyo en toda esta etapa.*

Madrid, 31 de mayo de 2026

Noel Rodríguez Pérez

Resumen

La seguridad en los sistemas modernos es un área en constante evolución, donde la integridad de los datos y el control de su funcionamiento juegan un papel fundamental. Esto es debido a la presencia de amenazas y agentes externos que trabajan con el fin malicioso de comprometer dicha información, haciendo necesario el desarrollo de mecanismos de protección cada vez más robustos.

En este ámbito, una de las amenazas más críticas para los sistemas de información actualmente son los *rootkits*, software malicioso diseñado para ocultar y manipular recursos o actividad del sistema, como procesos, ficheros o conexiones de red. Estas amenazas pueden operar tanto en *espacio de usuario* como en *espacio del kernel*, comprometiendo la información que recibe tanto el administrador como el usuario, impidiendo disponer de una visión verídica del estado real del sistema. Esto provoca una falta de integridad en las tareas de monitorización y control del mismo, pudiendo llegar a confiar en fuentes de información potencialmente comprometidas.

Para abordar esta problemática, en este proyecto se ha diseñado e implementado un *módulo del kernel* (*LKM*, *Loadable Kernel Module*) en un sistema *Linux* con el objetivo de proporcionar una fuente autenticada de información crítica del sistema frente a posibles manipulaciones. Esta solución introduce un mecanismo de autenticación criptográfica basado en HMAC (*Hash-based Message Authentication Code*) del contenido proporcionado por el *kernel*, mediante una clave simétrica secreta compartida únicamente entre el administrador y el módulo. Este planteamiento se apoya en una cadena de confianza en la que se asume que el *kernel* y el *LKM* no han sido modificados, por ejemplo mediante mecanismos de firmado del *kernel* y del módulo, reforzando así la integridad de la información ofrecida.

Los experimentos realizados aproximan un entorno real de operación donde la información sensible es solicitada desde el *espacio de usuario* hacia el *espacio del kernel*. Con este fin, se define el contenido sensible del sistema a autenticar mediante un

listado de todos los procesos en ejecución y sus respectivos metadatos, generado en el propio *módulo del kernel*, sirviendo como ejemplo de información crítica que puede ser manipulada por un *rootkit*. Sobre este contenido, se calcula el HMAC con la respectiva clave secreta y se adjunta a la información proporcionada al usuario, de forma que este pueda verificar su integridad y autenticidad.

Para validar esta propuesta, se han desarrollado componentes de usuario que prueban la funcionalidad. Un primer componente genera un fichero cuyo descriptor es transferido al *kernel* para que este lo referencie, estableciendo así el canal de comunicaciones seguras con el núcleo del sistema. Por otro lado, la integridad de los datos reportados se confirma mediante un segundo componente de verificación que analiza la información escrita por el *kernel*, calculando localmente el HMAC del contenido con la misma clave secreta y contrastándolo posteriormente con el recibido para asegurar que coinciden y que el contenido es verídico.

Acrónimos

API *Application Programming Interface*

CIA *Confidentiality, Integrity and Availability*

CLI *Command Line Interface*

CPU *Central Processing Unit*

RAM *Random Access Memory*

DRAM *Dynamic Random Access Memory*

FD *File Descriptor*

GNU *GNU's Not Unix*

GPL *GNU General Public License*

OS *Operating System*

MAC *Message Authentication Code*

HMAC *Hash-based Message Authentication Code*

SHA *Secure Hash Algorithm*

ID *Identifier*

LKM *Loadable Kernel Module*

PCB *Process Control Block*

US *User Space*

KS *Kernel Space*

NS *Namespace*

UID *User Identifier*

PID *Process Identifier*

GID *Group Identifier*

PS *Process Status*

VM *Virtual Memory*

VFS *Virtual File System*

FS *File System*

FUSE *Filesystem in Userspace*

FHS *Filesystem Hierarchy Standard*

MMU *Memory Management Unit*

BSS *Block Started by Symbol*

PC *Program Counter*

SP *Stack Pointer*

SYS *System*

BUF *Buffer*

GFP *Get Free Pages*

UTS *UNIX Time-Sharing*

IPC *Inter-Process Communication*

EDR *Endpoint Detection and Response*

HIDS *Host-based Intrusion Detection System*

PoLP *Principle of Least Privilege*

RAT *Remote Administration Tool*

RCU *Read-Copy-Update*

Índice general

1. Introducción	1
1.1. Ciberseguridad	2
1.1.1. Vulnerabilidades y Ataques	4
1.1.2. Defensas	6
1.1.3. Rootkits	7
2. Marco Tecnológico	10
2.1. Sistemas operativos <i>Linux</i>	10
2.1.1. Arquitectura y anillos de privilegios	12
2.1.2. Memoria: <i>Espacio de Usuario</i> y <i>Espacio del Kernel</i>	14
2.1.3. Ficheros y Sistemas de Ficheros (<i>FS</i>)	16
2.1.4. Espacios de nombres (<i>Namespaces</i>)	19
2.1.5. Módulos del Kernel de <i>Linux</i> (<i>LKMs</i>)	24
2.1.6. Entorno de desarrollo	26
3. Estado del arte	30
3.1. Rootkits de Espacio de Usuario	31
3.2. Rootkits de espacio del Kernel	32
4. Planteamiento del problema y objetivos	34
4.1. Descripción del problema	34
4.2. Modelo de amenaza	36
4.2.1. Contexto de operación y supuestos de seguridad	36
4.2.2. Capacidades y limitaciones del atacante	37
4.2.3. Cobertura defensiva	38
4.3. Objetivos	38
4.3.1. Objetivos funcionales	39
4.3.2. Objetivos no funcionales	40
4.4. Metodología	41

4.5. Plan de trabajo	43
5. Diseño y Desarrollo	45
5.1. Prototipos, Implementaciones y Validaciones	45
5.1.1. <i>Hello World</i>	46
5.1.2. Fichero virtual <i>/proc/fdev</i>	48
5.1.3. <i>File Handoff</i>	50
5.1.4. Mecanismo criptográfico de autenticación <i>HMAC-SHA256</i>	53
5.1.5. Sistema de reporte seguro de los procesos en ejecución del sistema	57
5.2. Problemas y Errores encontrados	62
5.2.1. Escritura sobre el fichero de transferencia en modo <i>append</i> . . .	63
5.2.2. Integración del mecanismo <i>HMAC-SHA256</i> entre <i>Kernel-Space</i> y <i>User-Space</i>	64
5.2.3. Recorrido seguro de la lista de procesos mediante <i>RCU</i>	66
6. Conclusiones	69
6.1. Cumplimiento de objetivos	70
6.1.1. Cumplimiento de los objetivos funcionales	70
6.1.2. Cumplimiento de los objetivos no funcionales	72
6.2. Impacto social, económico, temporal, medioambiental y ético	74
6.3. Competencias adquiridas y utilizadas	76
6.4. Trabajo futuro	78
A. Código y validación de los prototipos	83
A.1. Prototipos, Implementaciones y Validaciones	83
A.1.1. <i>Hello World</i>	83
A.1.2. Fichero virtual <i>/proc/fdev</i>	87
A.1.3. <i>File Handoff</i>	92
A.1.4. Mecanismo criptográfico de autenticación <i>HMAC-SHA256</i>	100
A.1.5. Sistema de reporte seguro de los procesos en ejecución del sistema	115
Bibliografía	128

Índice de figuras

1.1. Principales tipos de <i>malware</i>	3
1.2. Propiedades características de un <i>rootkit</i>	7
2.1. Árbol genealógico de los sistemas operativos <i>Unix</i>	11
2.2. Anillos de privilegios en sistemas operativos modernos.	12
2.3. Estructura de un sistema operativo moderno.	13
2.4. División lógica de memoria en <i>Espacio de Usuario</i> y <i>Espacio del Kernel</i>	15
2.5. Estructura jerárquica estándar de directorios en <i>Linux</i> (<i>FHS</i>).	17
2.6. Gestión de sistemas de ficheros en <i>Linux</i>	18
2.7. Punto de montaje: múltiples sistemas de ficheros en un único árbol lógico.	19
2.8. Tipos de <i>Namespaces</i>	21
2.9. Ejemplo de arquitectura <i>Docker</i>	23
2.10. Sistema físico empleado en el proyecto.	27
2.11. Logo de la distribución Armbian para Raspberry Pi 4 Model B.	27
3.1. Clasificación general de <i>rootkits</i> en sistemas <i>Linux</i>	30
4.1. Mecanismo de reporte autenticado entre <i>espacio de usuario</i> y <i>Espacio de kernel</i>	35
4.2. Esquema de la metodología tipo <i>Spiral</i> (espiral) empleada en el proyecto.	42
5.1. Flujo básico de carga, ejecución y descarga de un <i>LKM</i>	46
5.2. Comunicación básica entre <i>espacio de usuario</i> y <i>espacio del kernel</i> mediante un fichero virtual en <i>/proc</i>	48
5.3. Transferencia de un <i>File Descriptor</i> de un fichero real al <i>LKM</i> para establecer una vía de comunicación desde <i>espacio del kernel</i> hacia <i>espacio de usuario</i>	51
5.4. Incorporación de un mecanismo de autenticación <i>HMAC-SHA256</i> al sistema de <i>File Handoff</i>	54

5.5. Generación y transferencia autenticada de un reporte de los procesos activos del sistema desde <i>espacio del kernel</i>	58
--	----

Listado de códigos

2.1. Ejemplo de salida del comando <code>ps aux</code> en un terminal Linux.	20
2.2. Comandos básicos de gestión de LKMs.	25
2.3. Instalación del entorno de desarrollo.	28
5.1. Ejemplo de la estructura final del fichero de transferencia autenticado. .	61
A.1. <i>Makefile</i> genérico empleado para compilar <i>LKMs</i>	84
A.2. Cabeceras y metadatos del módulo <i>Hello World</i>	85
A.3. Funciones de inicialización/carga y salida/descarga del módulo <i>Hello World</i>	85
A.4. Asociación de los puntos de entrada/carga y salida/descarga del módulo.	86
A.5. Compilación, carga, consulta y descarga del prototipo <i>Hello World</i> . . .	87
A.6. Cabeceras adicionales para el prototipo basado en <code>/proc</code>	87
A.7. Referencia a la entrada virtual y tabla de operaciones asociadas en <code>/proc</code> .	88
A.8. <i>Handlers</i> con transferencia de datos entre <i>espacio de usuario</i> y <i>espacio del kernel</i>	90
A.9. Validación básica del prototipo basado en <code>/proc/fdev</code>	92
A.10. Cabeceras adicionales para el mecanismo de <i>File Handoff</i>	92
A.11. Función de lectura para el mecanismo de <i>File Handoff</i>	93
A.12. Función auxiliar de escritura sobre un fichero desde el <i>LKM</i>	94
A.13. Función auxiliar para la validación de permisos de un fichero desde el <i>LKM</i>	94
A.14. Función de escritura para el mecanismo de <i>File Handoff</i>	96
A.15. Programa <code>mk_fhandoff.c</code> de <i>espacio de usuario</i> para activar el <i>File Handoff</i>	98
A.16. Validación del mecanismo de <i>File Handoff</i>	99
A.17. Cabeceras adicionales y clave embebida para el mecanismo de autenticación <i>HMAC-SHA256</i>	100

A.18.Script <code>gen_k_embedded.sh</code> para generar la cabecera <code>k_embedded.h</code> con la clave embebida.	101
A.19.Cálculo del <i>HMAC-SHA256</i> para una cadena de <i>bytes</i> de datos.	102
A.20.Conversión del <i>HMAC</i> binario a <i>Base64</i>	103
A.21.Escritura conjunta del contenido y su <i>HMAC</i> en <i>Base64</i>	104
A.22.Integración conjunta del mecanismo de autenticación.	105
A.23.Manejador de escritura adaptado al mecanismo de autenticación de contenido.	106
A.24.Cabeceras, constantes y clave simétrica necesarias para la verificación. .	107
A.25.Deserialización del contenido del fichero autenticado en <i>espacio de usuario</i> . 109	
A.26.Cálculo local del <i>HMAC</i> y su codificación en <i>Base64</i>	111
A.27.Comparación del autenticador y flujo principal del verificador.	112
A.28.Validación del mecanismo de autenticación mediante <i>HMAC-SHA256</i> . .	114
A.29.Cabeceras y constantes para la generación del listado de procesos. . . .	115
A.30.Funciones auxiliares para el formateo y la construcción segura del contenido serializado.	117
A.31.Construcción del listado global de procesos en memoria del <i>kernel</i>	118
A.32.Serialización de los metadatos de un proceso.	120
A.33.Identificadores de usuario de cada proceso.	121
A.34.Identificadores de proceso de cada tarea.	122
A.35.Identificadores de grupo y nombre del comando asociado a cada proceso. 124	
A.36.Validación del sistema de reporte seguro de metadatos de los procesos activos de manera autenticada mediante <i>HMAC-SHA256</i>	125

Capítulo 1

Introducción

«*El mayor enemigo de la seguridad es la complejidad.*»

Bruce Schneier (*A Plea for Simplicity*, 1999) [1].

Hoy en día, la seguridad de los sistemas informáticos es un aspecto fundamental a tener en cuenta, cobrando cada vez más importancia a medida que alojamos mayor cantidad de información sensible en ellos y se convierten en sistemas mucho más complejos. Por lo tanto, podrían quedar expuestos a un gran número de amenazas si no se protegen debidamente. Como advierte el ensayo citado al inicio del capítulo, la inherente complejidad de estas infraestructuras facilita la proliferación de *malware* diseñado para operar e infiltrarse desde la más estricta invisibilidad.

Un sistema operativo moderno no es una estructura o proceso simple, sino un entrelazado conjunto de capas o niveles concéntricos de abstracción que actúan respecto a un sistema de hardware. Se puede describir como un sistema realmente complejo, con una gran cantidad de componentes y funcionalidades. Esta complejidad hace que sea difícil de proteger y monitorizar, requiriendo de un gran conocimiento sobre el mismo para saber cómo actuar ante posibles ataques.

A su vez, por parte del atacante, se requiere de un gran conocimiento técnico para diseñar un ataque efectivo, pero también de una gran creatividad y habilidad para encontrar vulnerabilidades y las formas de explotarlo sin ser detectado, dependiendo de su finalidad.

Esto hace de la ciberseguridad una disciplina realmente amplia y compleja, donde atacantes y defensores se encuentran en una constante carrera por encontrar nuevas formas de comprometer o proteger los sistemas.

Este proyecto se centra en el ámbito de la seguridad de los sistemas operativos *Linux*, concretamente en el diseño de un mecanismo defensivo que permita ofrecer información crítica del sistema de forma autenticada y verificable frente a posibles manipulaciones.

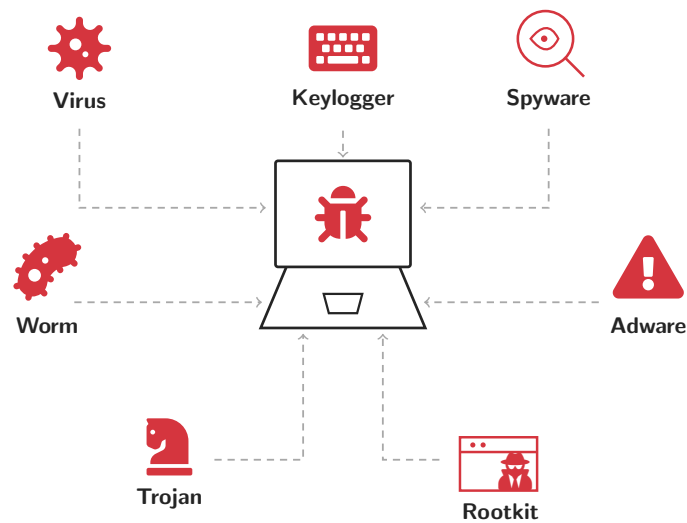
Para comprender en detalle el escenario descrito y contextualizar la temática del proyecto, a continuación se establecerán conceptos básicos sobre la ciberseguridad y sus principales finalidades, especialmente sobre la amenaza principal en la que se centra el proyecto, los *rootkits*.

1.1. Ciberseguridad

En el ámbito de los sistemas de información, la ciberseguridad comprende el conjunto de arquitecturas, tecnologías y prácticas diseñadas para proteger los sistemas, datos y redes frente a ataques que puedan comprometer su integridad, confidencialidad o disponibilidad, ya sean intencionados o no. No obstante, técnicamente debe entenderse como un equilibrio entre el valor de los activos a proteger, el riesgo asociado a su compromiso y el coste de las medidas necesarias para garantizar una protección proporcional.

Desde este punto de vista, proteger un sistema no consiste en maximizar indefinidamente las defensas, sino en aplicar mecanismos proporcionales al impacto real de un posible ataque.

El principal flujo de estas amenazas se materializa mediante ***software* malicioso** (***malware***), concepto que engloba cualquier código diseñado para infiltrarse en un sistema, dañarlo o robar datos sin el consentimiento de su propietario [2].



Fuente: Elaboración propia basada en Akamai Technologies, *¿Qué es el malware?* [3], usando iconografía vectorial de Font Awesome Free [4].

Figura 1.1: Principales tipos de *malware*.

Actualmente, existen diversas familias de *malware* clasificadas principalmente según su comportamiento y objetivo. Por un lado, destacan las amenazas orientadas a la rápida infiltración y propagación, como los **virus**, **gusanos** (*worms*) y **troyanos**, los cuales aprovechan vulnerabilidades técnicas o el engaño psicológico del usuario para acceder al sistema. Por otro lado, encontramos variantes focalizadas en el espionaje y/o el lucro económico, como el **ransomware**, que cifra datos críticos o personales exigiendo un posterior rescate económico, y el **spyware** junto a los **keyloggers**, diseñados para vigilar y extraer información sensible del sistema de forma inadvertida.

No obstante, dentro de este ecosistema ofensivo existe una categoría que destaca por su extrema criticidad y que constituye el foco principal de este proyecto: los **rootkits**. A diferencia de otras familias de *malware* centradas en la propagación, el espionaje o el lucro económico directo, los **rootkits** se caracterizan por su capacidad para ocultar y manipular recursos y actividad del sistema, comprometiendo la integridad de la información que este proporciona. Estas amenazas pueden operar tanto en el *espacio de usuario* como en el *espacio del kernel*, siendo una gran amenaza para la seguridad de los sistemas de información actuales. Más adelante se profundizará con mayor detalle en su funcionamiento.

Sin embargo, para que este *malware* logre infiltrarse en la infraestructura del sistema, los atacantes necesitan métodos de acceso y mecanismos de persistencia. A su

vez, precisan de un correcto análisis, identificación y explotación de las vulnerabilidades del sistema a través de las cuales poder infiltrarse [5].

1.1.1. Vulnerabilidades y Ataques

En el contexto de la ciberseguridad de sistemas, una **vulnerabilidad** se define como una debilidad en el diseño, implementación o configuración de un sistema que puede ser aprovechada por un agente malicioso para comprometer su integridad, confidencialidad o disponibilidad [6].

En este contexto, el marco **MITRE ATT&CK** adquiere una gran importancia, ya que es una fuente de información estandarizada que clasifica las técnicas y tácticas empleadas por los atacantes durante una intrusión. Su utilidad reside en que permite describir amenazas con un lenguaje común y evaluar de forma estructurada la capacidad de detección, monitorización, robustez y respuesta frente a estas amenazas [5].

Estas brechas de seguridad constituyen el principal motor de entrada a intrusos y pueden clasificarse fundamentalmente en tres grandes grupos según su naturaleza:

- **Vulnerabilidades de *software* y *hardware*:** Fallos a nivel de código fuente del sistema operativo, aplicaciones o bibliotecas, así como en componentes físicos o en la interacción entre ambos. En el caso del *software*, pueden aparecer por una validación insuficiente de entradas, inyecciones de código o errores de memoria como los desbordamientos de *buffer* (*Buffer Overflow*). En el caso del *hardware*, ataques como *Rowhammer* aprovechan efectos físicos de la memoria *DRAM* para modificar bits en zonas de memoria cercanas. Por otro lado, ataques como *Spectre* explotan la interacción entre el *software* y optimizaciones internas del procesador, como la ejecución especulativa, para inferir información sensible mediante canales laterales.
- **Vulnerabilidades humanas:** Debidas a la falta de concienciación, errores o negligencias de los usuarios propietarios del sistema, pudiendo ser el eslabón más débil de la cadena de seguridad.
- **Vulnerabilidades de configuración:** Diseño o implementaciones inseguras de

la propia arquitectura del sistema, como el uso de credenciales por defecto, servicios de red expuestos o una asignación incorrecta de privilegios.

Para aprovechar estas vulnerabilidades y realizar la intrusión del *malware*, los atacantes hacen uso de diversas **herramientas de ataque** y técnicas específicas para cada escenario. Algunos ejemplos¹:

- **Ataques de ingeniería social:** Orientados a explotar las vulnerabilidades humanas. Destacan el *phishing* y sus variantes (*spear phishing*), cuyo objetivo es la manipulación psicológica del usuario mediante comunicaciones fraudulentas que permitan revelar credenciales o ejecutar *software* malicioso de forma inadvertida.
- **Exploits:** Para atacar mediante fallos de configuración o de *software*, se emplean *exploits*. Estos son fragmentos de código diseñados para aprovechar una debilidad técnica y forzar un comportamiento no previsto en la máquina. Su principal objetivo suele ser inyectar una carga útil detonante (*payload*), como **backdoors** o herramientas de administración remota (**RATs**).
- **Ataques de escala de privilegios:** Una vez que el atacante logra comprometer mínimamente el sistema, utiliza herramientas ofensivas para aprovechar vulnerabilidades secundarias del mismo y elevar sus permisos hasta incluso obtener el control de administrador (*root*).
- **Persistencia y ocultación:** Tras obtener acceso al sistema, el atacante puede emplear técnicas para mantener su presencia y dificultar su detección. En este contexto se sitúan los *rootkits*, capaces de ocultar procesos, ficheros o conexiones manipulando la información visible para el usuario.

En definitiva, independientemente de la vulnerabilidad detectada y la técnica ofensiva empleada, existe una necesidad crítica de implementar mecanismos de defensa robustos y una monitorización de estado segura. Esto, añadido a una adecuada educación y concienciación de los usuarios, resulta fundamental para mitigar los riesgos y poder proteger en gran medida al sistema.

¹Para un detalle pormenorizado de fases y técnicas de explotación puede consultarse MITRE ATT&CK [5].

1.1.2. Defensas

Para mitigar y protegerse ante las principales amenazas de este ámbito, se creó el enfoque estratégico basado en el principio de **defensa en profundidad** (*Defense in Depth*). Este concepto consiste en desplegar múltiples capas de seguridad superpuestas, de modo que si un control falla o es evitado, el atacante pueda encontrarse con otro más adelante y complicar el acceso. Principalmente mediante este enfoque, podemos dividir los métodos defensivos en dos categorías [7].

En primer lugar, las **medidas preventivas** buscan reducir la superficie de ataque y evitar que la intrusión llegue a materializarse. Entre las prácticas más destacadas se encuentran:

- **Concienciación y capacitación** (*Security Awareness*): Educar a los usuarios para identificar y reportar ataques de ingeniería social, reduciendo el factor de error humano.
- **Bastionado de sistemas** (*Hardening*): Consiste en mantener las aplicaciones y el sistema operativo actualizados para mitigar posibles *exploits*. Por otro lado, implica también la configuración segura del sistema, como deshabilitar servicios innecesarios o restringir el tráfico de red mediante el uso de *firewalls*.
- **Principio de Menor Privilegio** (*PoLP*): Garantiza que los usuarios, procesos y aplicaciones operen únicamente con los permisos mínimos e indispensables que precisen para cumplir su función. Esto limita drásticamente el impacto de una brecha inicial de acceso y dificulta la escalada de privilegios.

Por otro lado, asumiendo que las barreras preventivas pueden ser superadas, es indispensable implementar mecanismos de **detección, respuesta y monitorización** continua [8]:

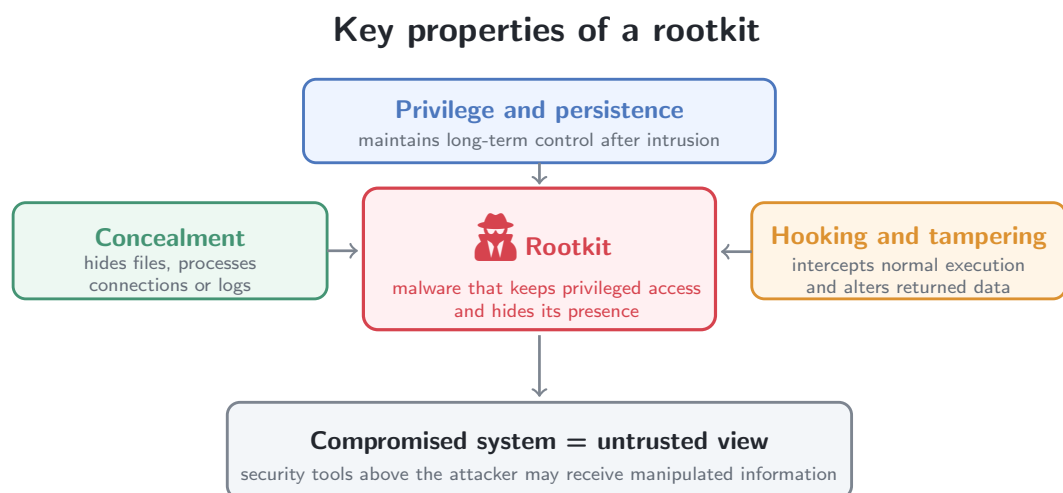
- **Sistemas de Detección de Intrusos basados en Host** (*HIDS*): Herramientas que monitorizan la actividad de la máquina, analizan los registros de eventos (*logs*) y detectan alteraciones no autorizadas registradas en ficheros críticos (*File Integrity Monitoring*).
- **Plataformas EDR** (*Endpoint Detection and Response*): Evolución del concepto de antivirus tradicional. En lugar de basarse únicamente en firmas,

analizan el comportamiento de los procesos en memoria en tiempo real para poder así detectar y bloquear ejecuciones anómalas.

Cabe destacar que el ámbito de la ciberseguridad se encuentra en constante evolución. La continua sofisticación y mejora de las técnicas ofensivas exige un gran esfuerzo por parte de la industria para investigar y diseñar nuevos métodos de protección. De este modo, los paradigmas de defensa empleados actualmente, como los explicados en este apartado, deben adaptarse de forma dinámica al contexto temporal de cada entorno en el que se utilicen, pudiendo quedar obsoletos y ser ineficaces a corto plazo.

1.1.3. Rootkits

En este contexto de ciberseguridad, un **rootkit** se define como un conjunto de fragmentos de *malware* diseñados específicamente para mantener un acceso privilegiado en un sistema, y a su vez, ocultar su presencia y las actividades que prefiera (como procesos, conexiones de red o ficheros) ante los administradores y el *software* de seguridad [9].



Fuente: Elaboración propia basada en National Institute of Standards and Technology (NIST), Rootkit [10].

Figura 1.2: Propiedades características de un *rootkit*.

A diferencia de otras amenazas, su principal objetivo no es propagarse o causar un daño destructivo inmediato, sino lograr una **persistencia indetectable** a largo plazo tras una intrusión inicial en el sistema objetivo. Para alcanzar este nivel de evasión, los *rootkits* alteran el flujo normal de ejecución del sistema operativo mediante técnicas de interceptación y modificación de código, típicamente conocidas como **hooking**.

Dependiendo de la capa en la que operen y del nivel de privilegios que puedan llegar a escalar, se pueden dividir en dos grandes categorías en sistemas *Linux*:

- **Rootkits de Modo Usuario (*User-Space*):** Operan en la región de menor privilegio (*Ring 3*). La técnica más común consiste en capturar las funciones de la biblioteca estándar de C (`libc`) mediante mecanismos como la carga dinámica de bibliotecas a través de la variable `LD_PRELOAD`. De esta forma, logran interceptar las peticiones a dichas funciones (*hooks*) como `open` o `read`, antes de llegar al *kernel*, ocultando y filtrando los resultados en la salida hacia el usuario. Investigaciones recientes demuestran que las herramientas de detección clásicas que operan en el nivel de privilegios de usuario normal resultan fácilmente evadibles, generando una falsa sensación de seguridad [9].
- **Rootkits de Modo Kernel (*Kernel-Space*):** Representan la amenaza más crítica, ya que operan en el nivel de máximo privilegio (*Ring 0*). Suelen implementarse mediante *módulos del kernel* (*LKMs*). Al residir en el *kernel* del sistema, adquieren privilegios de administrador sobre las estructuras de datos críticos y la Tabla de Llamadas al Sistema (*sys_call_table*). Esto les permite manipular el sistema desde dentro, volviendo completamente inútiles las herramientas de monitorización que operan por encima de ellos con menos privilegios.

El desafío fundamental que plantean estos tipos de amenazas radica en un axioma crítico en ciberseguridad: ***un sistema comprometido no puede proporcionar información fiable sobre sí mismo***. Con base en estos argumentos, intentar detectar o mitigar la acción de un *rootkit* utilizando mecanismos de seguridad que operan en un nivel de privilegios inferior al del atacante es una estrategia totalmente ineficaz. Por esta razón, se precisa diseñar arquitecturas defensivas y de monitorización robustas en las capas más profundas del sistema, con el fin de garantizar la integridad de los datos y el correcto funcionamiento del entorno.

Tomando como referencia el complejo panorama de la ciberseguridad descrito y la gran amenaza que suponen los *rootkits*, el presente proyecto propone el diseño y desarrollo de un sistema de reporte seguro para entornos *Linux*, enfocado en proporcionar información crítica del sistema de forma autenticada y verificable frente a posibles manipulaciones de este tipo de *malware*. Antes de ello, en el siguiente capítulo se establecerá el marco tecnológico de los sistemas operativos *Linux* actuales, donde se detallará su diseño, arquitectura y funcionamiento, facilitando así la comprensión del proyecto en cuestión. Posteriormente, se abordarán los objetivos y metodologías adoptadas para el diseño, integración y validación de este sistema de reporte seguro.

Capítulo 2

Marco Tecnológico

Para poder diseñar e implementar una solución de seguridad efectiva frente a las amenazas que representan los *rootkits*, es necesario un análisis detallado previo del entorno tecnológico en el que operan. Por ello, en el presente capítulo se analizará en primer lugar la arquitectura y estructura de los sistemas operativos *Linux*, detallando sus mecanismos internos, como su gestión de ficheros, memoria y modularidad.

2.1. Sistemas operativos *Linux*

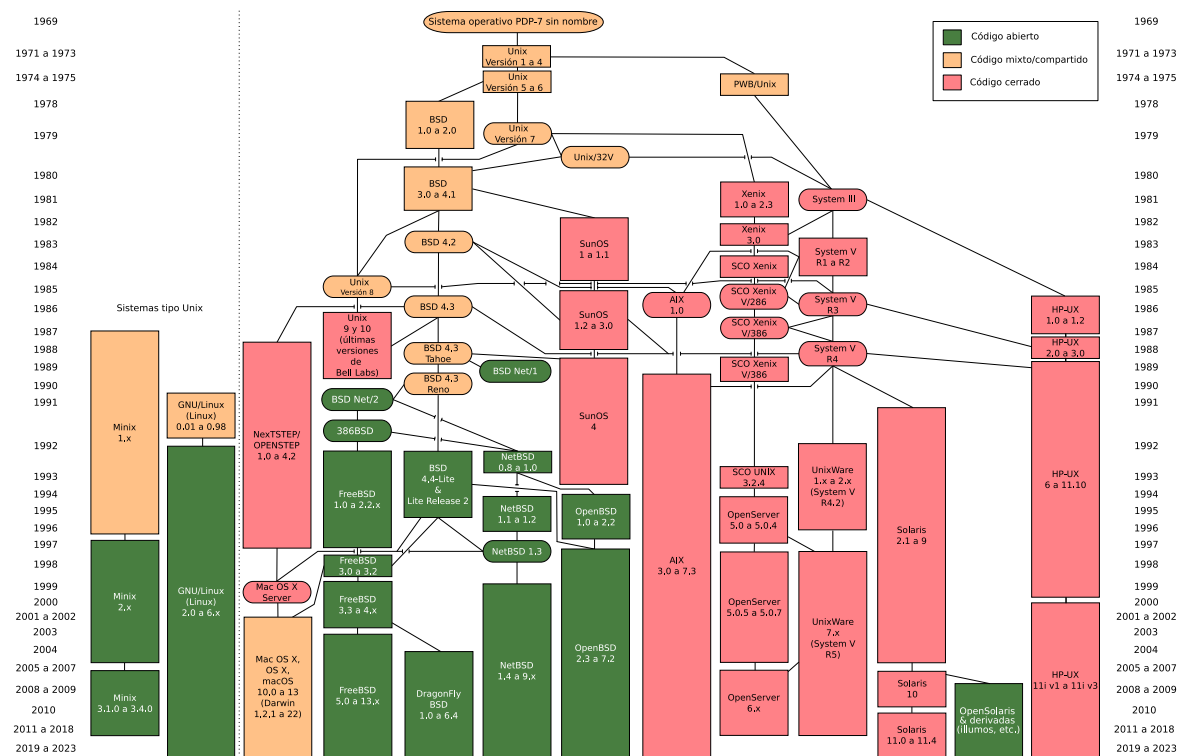
No existe una definición canónica y universal para el concepto de sistema operativo, tratándose de una idea hasta cierto punto relativa y subjetiva dependiendo de su cometido y contexto de uso [11].

Una posible definición puede ser: El conjunto de programas que permite utilizar la máquina como sistema, haciendo lo necesario para que puedan ejecutarse múltiples procesos sobre el mismo *hardware*, multiplexando la máquina en tiempo (administrando uso de *CPU*) y espacio (gestionando la memoria *RAM*).

Otra posible definición sería: El programa o conjunto de los mismos que actúan como intermediarios entre el usuario y el *hardware*, proporcionando una interfaz como abstracción de procesos e instrucciones internas, para poder interactuar con la máquina y sus recursos de manera eficiente y robusta.

En el mercado tecnológico actual existen diferentes sistemas operativos de propósito general, destacando opciones comerciales de código cerrado como *Microsoft Windows* o *macOS*, orientados generalmente a ordenadores, y por otra parte, *Android* e *iOS*, principalmente diseñados para dispositivos móviles. Sin embargo, dentro del ámbito

de la administración de infraestructuras, ciberseguridad y desarrollo de *software*, *GNU/Linux* (de la familia *Unix*) se considera como el estándar de referencia absoluto.



Fuente: VARGUX, File:Unix history-simple es.svg (2023) [12].

Figura 2.1: Árbol genealógico de los sistemas operativos *Unix*.

Como ilustra la imagen anterior, *Unix* es el sistema pionero creado en los años 70 que sentó las bases de la informática moderna del que *Linux* nació como un clon, desarrollado íntegramente bajo un modelo de código abierto. También *macOS* proviene de esta familia, sin embargo, este sistema operativo ha evolucionado bajo un modelo comercial semi-privativo y de código parcialmente cerrado.

La hegemonía de *Linux* (*GNU/Linux*) en estos sectores se debe principalmente a su filosofía de código abierto (*open-source*)¹. Disponer del código fuente de manera pública, ya sea mediante la instalación local de cabeceras y fuentes del sistema (típicamente situados en entornos de desarrollo en `/usr/src`) o en repositorios públicos en línea (*Git*), permite a una gran comunidad mundial de desarrolladores auditar, depurar y mejorar el código del mismo de forma continua. Esta transparencia garantiza un nivel

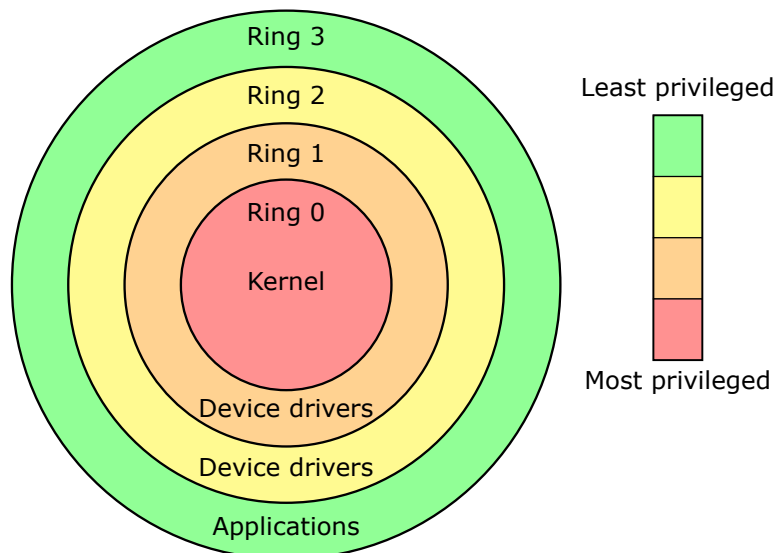
¹El código abierto (*open-source*) se centra en la disponibilidad del código fuente, su revisión y el desarrollo colaborativo. Y por otro lado, el software libre (*free software*) pone el foco en las libertades del usuario para ejecutar, estudiar, modificar y redistribuir el código, aunque ambos enfoques suelen coincidir en muchos proyectos.

de adaptabilidad y fiabilidad muy superior al de los sistemas comerciales, permitiendo una configuración del entorno a medida según las necesidades requeridas.

Por estos motivos, se considera *Linux* el sistema operativo óptimo para desarrollar este proyecto. A partir de este punto se asumirá por defecto que nos encontramos en un entorno *Linux*, aunque los principios fundamentales se pueden extrapolar a cualquier sistema operativo moderno.

2.1.1. Arquitectura y anillos de privilegios

Para garantizar su estabilidad y seguridad, los sistemas operativos modernos no confían únicamente en restricciones lógicas de *software*, sino que se apoyan de forma entrelazada en mecanismos de protección implementados a nivel de *hardware* por el propio procesador. En arquitecturas de sistemas predominantes como la familia x86, este mecanismo de aislamiento se implementa típicamente a través de dominios concéntricos de protección jerárquicos, comúnmente conocidos como anillos de protección (*Protection Rings*) [11].



Fuente: Colaboradores de Wikipedia, Anillo (seguridad informática) (2026) [13].

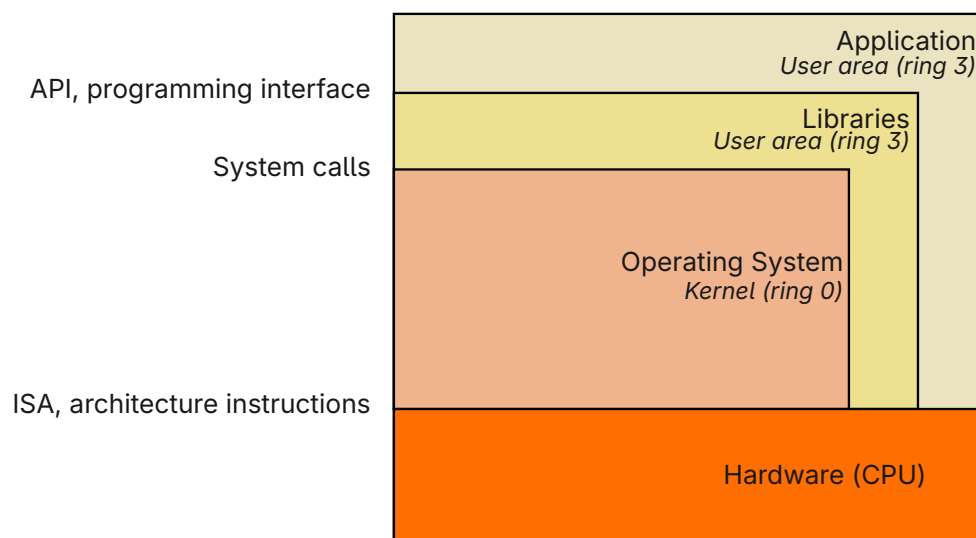
Figura 2.2: Anillos de privilegios en sistemas operativos modernos.

Esta arquitectura determina principalmente cuatro niveles de privilegios concéntricos, comúnmente llamados “anillos”, numerados del 0 al 3. El nivel de

privilegio implica qué instrucciones de la CPU puede ejecutar un proceso y a qué regiones de la memoria puede acceder al ejecutarse, distinguiendo estos cuatro anillos [11]:

- **Ring 0:** Nivel de máximo privilegio, donde reside típicamente el núcleo (*kernel*). Los programas ejecutados en este anillo tienen prácticamente acceso directo e irrestricto a todo el *hardware*, la memoria y las instrucciones del procesador.
- **Ring 1 y 2:** Originalmente diseñados para servicio y control del propio sistema con privilegios intermedios, aunque en la práctica han sido mayormente descartados por la mayoría de sistemas operativos, debido a su complejidad y bajo rendimiento.
- **Ring 3:** Nivel de menor privilegio. Los procesos lanzados en este anillo no pueden interactuar directamente con el *hardware*, debiendo delegar cualquier operación crítica a niveles de mayor privilegio.

Por motivos de eficiencia y portabilidad a otras arquitecturas de procesadores (como *ARM*, que normalmente no posee tantos niveles), *Linux* simplifica este modelo y utiliza exclusivamente dos de estos anillos. Por un lado, se aísla el *kernel* (núcleo del sistema), ejecutándolo íntegramente en el *Ring 0*, y por otro lado, se confina el resto de programas de usuario en la región del *Ring 3* [14, 11].



Fuente: Guardiola Múzquiz, Gorka and Soriano Salvador, Enrique, *Fundamentos de Sistemas Operativos: Una aproximación práctica usando Linux* (2025) [11].

Figura 2.3: Estructura de un sistema operativo moderno.

Básicamente, la estructura del sistema operativo se puede describir como una división física de privilegios que se traduce directamente en una arquitectura lógica estructurada en capas construidas unas encima de otras [11].

En la base se encuentra el *hardware* físico de la máquina, con el cual el núcleo interactúa directamente a través de las instrucciones de la arquitectura (ISA) disponibles. Inmediatamente por encima, operando con privilegios absolutos sobre el sistema, en el *Ring 0*, reside el *kernel*. Este es el corazón del sistema operativo, responsable de gestionar los recursos de la máquina y de actuar como puente fundamental entre el *hardware* y las aplicaciones de usuario. Posteriormente, en las capas más externas del mismo, se encuentran las bibliotecas del sistema, encargadas de proporcionar la ***interfaz de programación de aplicaciones (API)***, y las aplicaciones finales, las cuales se encuentran confinadas en el área de usuario (*Ring 3*).

Debido a esta separación, si algún proceso de esta región de usuario requiere realizar una acción privilegiada (como leer un archivo del disco, enviar un paquete de red o reservar memoria), no puede acceder al *hardware* directamente por sí mismo. En su lugar, debe solicitar dicha operación privilegiada al *kernel* atravesando la frontera del *Ring 0* mediante un mecanismo de comunicación estandarizado y controlado, comúnmente conocido como ***llamadas al sistema*** (*System Calls* o *syscalls*).

Esta barrera arquitectónica no solo define qué instrucciones se pueden ejecutar y dónde, sino que obliga al sistema operativo a dividir la memoria principal de rápido acceso (*RAM*) en dos grandes regiones virtuales completamente aisladas en ***espacio de usuario*** y ***espacio de kernel***, garantizando así controlar que ningún proceso de usuario pueda corromper datos sensibles internos del sistema [14, 11].

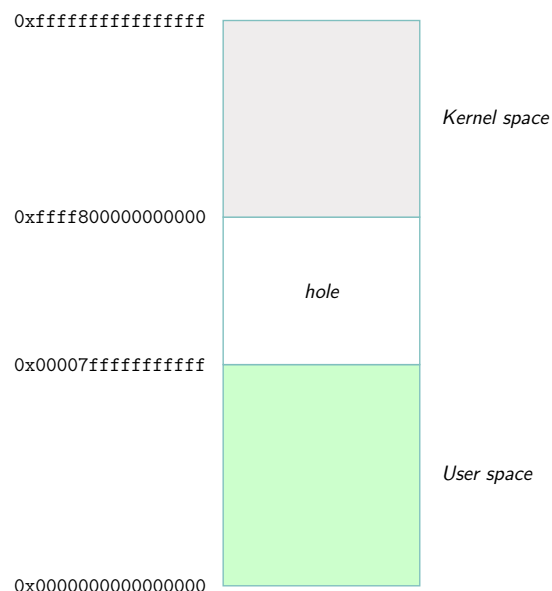
2.1.2. Memoria: *Espacio de Usuario y Espacio del Kernel*

Para comprender cómo se materializa la división de privilegios descrita en el apartado anterior realmente en la memoria física de la máquina (*RAM*), es necesario introducir el concepto de *memoria virtual*.

En los sistemas *Linux* modernos, los procesos no acceden directamente a las direcciones físicas de la memoria *RAM*. En su lugar, el *kernel* y la unidad de gestión

de memoria (*MMU*) de la *CPU* proporcionan a cada proceso la ilusión de tener espacio exclusivo y contiguo a través de la **memoria virtual**. Para lograrlo, la *MMU* traduce dinámicamente las direcciones virtuales de memoria a ubicaciones físicas reales y viceversa, consultando una **tabla de páginas**. De esta forma, se garantiza que múltiples procesos puedan ejecutarse simultáneamente de forma aislada y segura, cada uno con su propio mapa de direcciones gestionado íntegramente por el *kernel* [15].

A su vez, este espacio de *memoria virtual* se divide de forma estricta en dos grandes regiones, fuertemente relacionadas con la arquitectura de anillos de protección del procesador:



Fuente: Guardiola Múzquiz, Gorka and Soriano Salvador, Enrique, *Fundamentos de Sistemas Operativos: Una aproximación práctica usando Linux (2025)* [11].

Figura 2.4: División lógica de memoria en *Espacio de Usuario* y *Espacio del Kernel*.

- **Espacio de Usuario (User-Space):** Es la porción inferior de la memoria virtual, asociada al *Ring 3*. En esta región se cargan y ejecutan las aplicaciones de usuario, sus bibliotecas y los datos pertinentes. Si un proceso comete un error “crítico” en esta región (como intentar acceder a memoria que no le pertenece), el sistema operativo interviene para finalizar dicho proceso y genera un error (típicamente conocido como *Segmentation Fault*), evitando que el fallo afecte al resto del sistema.
- **Espacio del Kernel (Kernel-Space):** Es la porción superior de la memoria virtual, reservada exclusivamente para el **kernel** con sus módulos y los

controladores de dispositivos (*drivers*). Al estar asociado al *Ring 0*, el código que reside aquí tiene acceso total a la memoria física y, por consiguiente, al *hardware*. Un fallo de programación o una intrusión maliciosa en este nivel compromete todo el sistema, pudiendo provocar una caída total del mismo (*Kernel Panic*).

De la misma manera que la *memoria virtual* abstrae la complejidad de organización de la memoria física (*RAM*) para garantizar el aislamiento y la ejecución segura de los procesos, el sistema operativo proporciona a su vez una abstracción equivalente para la gestión del almacenamiento persistente y la interacción con el resto de componentes del sistema. Esta importante tarea recae sobre otro gran pilar estructural de un sistema operativo, en este caso *Linux*: su *sistema de ficheros* (*file system*).

2.1.3. Ficheros y Sistemas de Ficheros (*FS*)

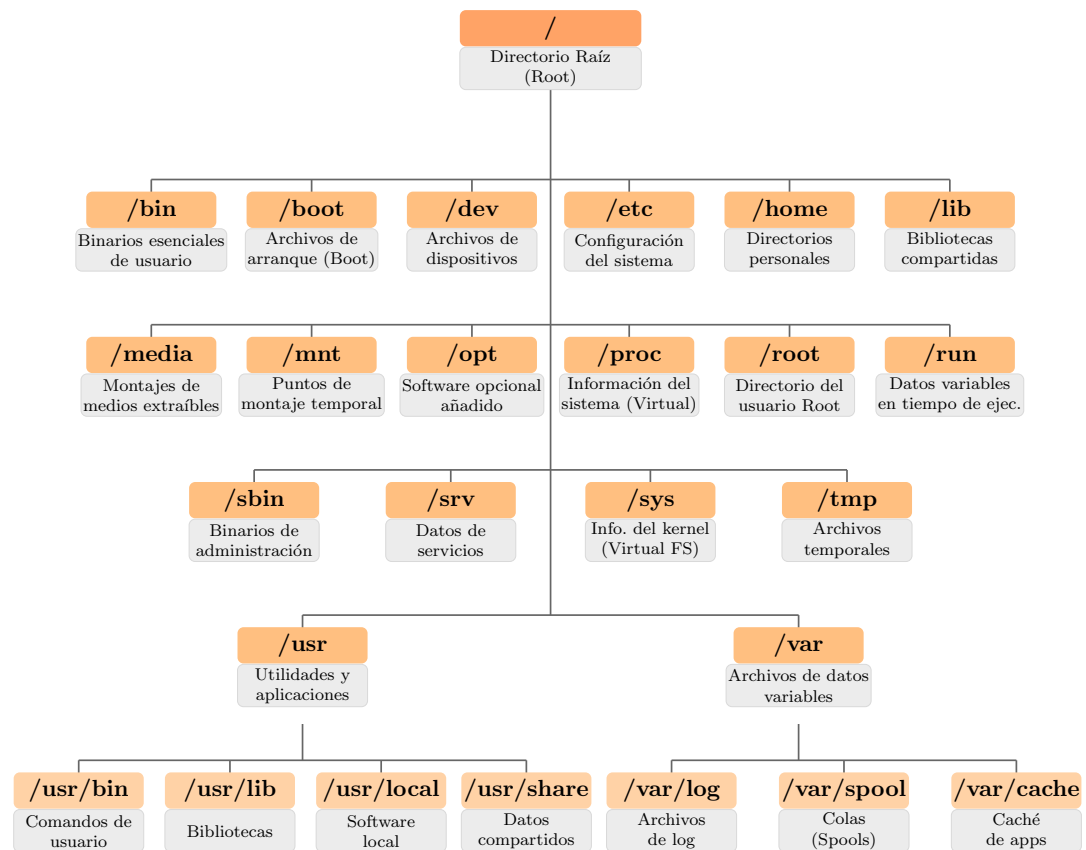
Una de las características más destacadas de la arquitectura de *Linux*, heredada directamente de la familia original de *Unix*, es el principio fundamental de que «todo es un fichero»². En este tipo de sistemas operativos, no solo los documentos de texto o los programas ejecutables binarios son tratados como ficheros, sino también los directorios, las conexiones de red (*sockets*), los dispositivos de *hardware* (como terminales o discos duros), así como la información dinámica de los procesos en ejecución y del propio sistema (como los ficheros virtuales situados en el directorio */proc*) [14].

Para estructurar, acceder y organizar toda esta enorme cantidad de información, el sistema operativo emplea un **Sistema de Ficheros (*FS*)**, que se puede definir como el conjunto de reglas, métodos y jerarquías lógicas encargados de gestionar cómo los ficheros se nombran, se almacenan y se recuperan en un medio lógico.

Debido a la gran cantidad de datos y recursos tanto físicos como lógicos que maneja la máquina, es necesario mantener un orden universal de los mismos para que el sistema operativo sepa exactamente dónde se encuentran. Como solución a esta necesidad organizativa, *Linux* adopta el estándar ***FHS (Filesystem Hierarchy Standard)***,

²Un fichero no se limita a un documento almacenado en disco, sino que representa una interfaz uniforme para acceder a recursos del sistema. En *Plan 9*, sucesor conceptual de *Unix*, esta filosofía fue llevada aún más lejos, haciendo que prácticamente todos los recursos fuesen visibles como ficheros y directorios. El sistema */proc* sigue esta misma filosofía, principalmente desde *Plan 9*.

que define una jerarquía global de directorios raíz donde se distribuyen los distintos tipos de ficheros del sistema, como binarios, bibliotecas, ficheros de configuración, datos persistentes y recursos dinámicos, según su función, tipo y alcance.



Fuente: Elaboración propia basada en Albritton, William McDaniel, "Module 2, Session 7 - Linux File System Structure" (2016) [16].

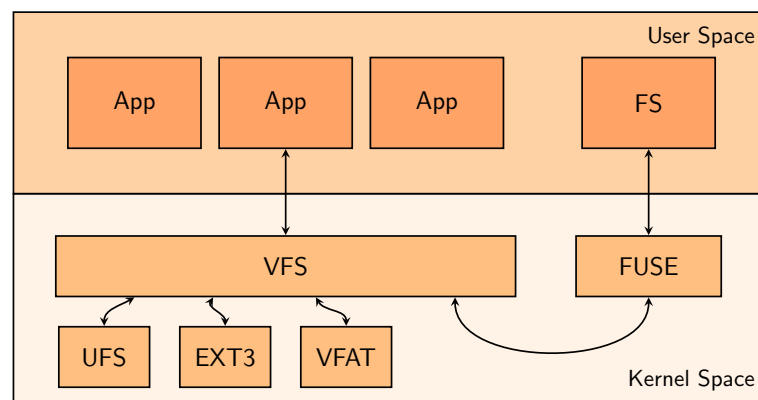
Figura 2.5: Estructura jerárquica estándar de directorios en *Linux* (FHS).

Dentro de esta estructura, uno de los directorios más singulares es `/proc`, ya que no almacena datos persistentes en disco, sino que constituye un pseudo-sistema de ficheros generado dinámicamente por el *kernel* en memoria (*RAM*) mediante **ficheros virtuales**. Su principal propósito es exponer, a través de una interfaz basada en ficheros, información sobre parámetros del sistema, procesos en ejecución y determinados estados internos del *kernel*, como `/proc/meminfo` o `/proc/cpuinfo`. El acceso a estas entradas se realiza mediante operaciones estándar sobre ficheros, como `read()` o `write()`, que el *kernel* redirige a los manejadores correspondientes según la naturaleza y propósito de cada fichero virtual.

De esta manera, es posible crear interfaces de comunicación entre el *espacio de usuario* y el *espacio del kernel* mediante *módulos del kernel* que creen nuevas entradas

de fichero virtual dentro de este directorio, permitiendo asociar operaciones de lectura y escritura a manejadores específicos para solicitar o reportar información sensible del sistema de forma controlada.

Para gestionar esta gran heterogeneidad de recursos y permitir la coexistencia de múltiples sistemas de ficheros físicos (como *ext4* o *NTFS*) en un mismo sistema anfitrión, el *kernel* implementa una capa de abstracción denominada **Sistema de Ficheros Virtual (VFS)**, situada entre la capa de llamadas al sistema (*syscalls*) y los *drivers* de bajo nivel. Su función es proporcionar una *API* unificada que permite a las aplicaciones del *espacio de usuario* utilizar llamadas estándar (como `read()` o `write()`) sobre cualquier fichero o dispositivo, abstrayendo por completo los detalles de la máquina [11].



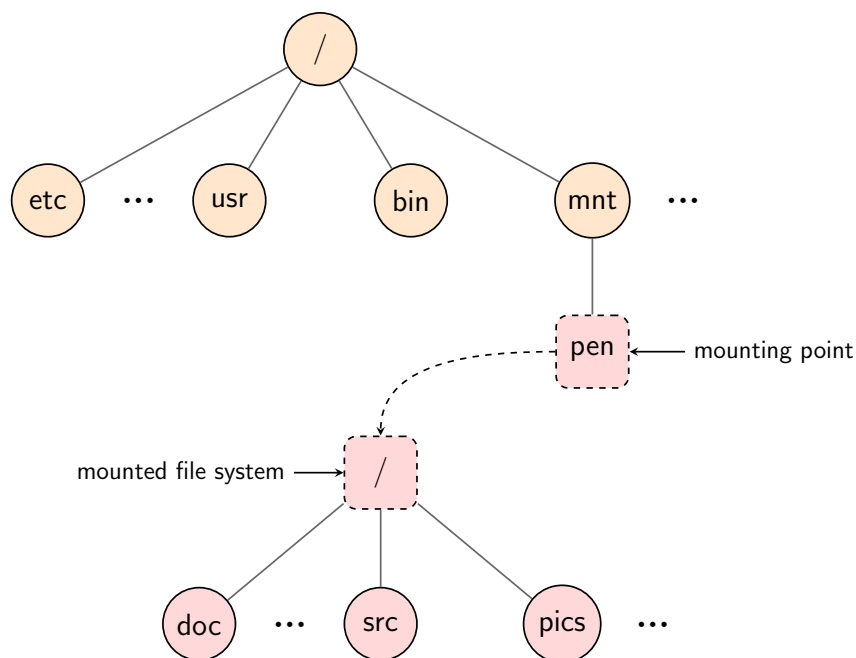
Fuente: Guardiola Múzquiz, Gorka and Soriano Salvador, Enrique, *Fundamentos de Sistemas Operativos: Una aproximación práctica usando Linux* (2025) [11].

Figura 2.6: Gestión de sistemas de ficheros en *Linux*.

Principalmente, el *VFS* actúa como un enrutador central entre el *espacio de usuario* y el *espacio del kernel*, recibiendo las peticiones de las aplicaciones y redirigiéndolas de forma transparente al *driver* correspondiente. Además, los sistemas modernos soportan mecanismos avanzados como *FUSE* (*Filesystem in Userspace*), que permite al *VFS* delegar la gestión de ciertos sistemas de ficheros a procesos que se ejecutan de forma segura directamente en el *espacio de usuario*.

Para integrar distintos sistemas de ficheros dentro del sistema anfitrión, *Linux* prescinde de unidades aisladas y utiliza la tecnología de **montaje** (*mounting*). Este mecanismo unificador “engancha” lógicamente cualquier sistema de ficheros externo a un directorio existente (por ejemplo, `/mnt` o `/media`), considerado **punto de**

montaje (*mount point*). De este modo, se crea una estructura de directorios continua y transparente que abstrae al usuario de la naturaleza del medio externo subyacente [14].



Fuente: Guardiola Múzquiz, Gorka and Soriano Salvador, Enrique, *Fundamentos de Sistemas Operativos: Una aproximación práctica usando Linux (2025)* [11].

Figura 2.7: Punto de montaje: múltiples sistemas de ficheros en un único árbol lógico.

Como se ha expuesto en los apartados anteriores, la arquitectura de *Linux* se fundamenta en recursos fuertemente centralizados: un único *kernel*, memoria física compartida, una jerarquía de directorios global, entre otros. Para romper con esta centralización y dotar al sistema de capacidades de aislamiento, el *kernel* implementa la tecnología de los *Espacios de nombres* que permite el confinamiento de procesos y recursos, tanto físicos como lógicos, dentro de un entorno aislado y controlado. Este concepto se explica en detalle en el siguiente apartado [11, 14].

2.1.4. Espacios de nombres (*Namespaces*)

Para poder explicar en profundidad cómo funcionan los *Namespaces*, es necesario definir previamente cómo gestiona y clasifica el sistema operativo los programas que se ejecutan en la máquina.

A todo programa que se encuentre en ejecución y alojado en la memoria virtual se

le denomina **proceso**. Para administrar de forma eficiente la gran cantidad de procesos que pueden coexistir simultáneamente en el sistema, el *kernel* asigna a cada uno de ellos una serie de atributos e identificadores críticos, considerados de gran relevancia en cuanto al registro del estado del sistema:

- **PID (*Process ID*)**: Es un número entero único asignado dinámicamente a cada proceso activo que lo identifica de forma inequívoca dentro del sistema.
- **UID (*User ID*)**: Es el identificador numérico que asocia el proceso a un usuario concreto de la máquina. Este atributo permite dictaminar los permisos y privilegios principales que tiene el proceso para interactuar con el sistema de ficheros, enviar señales o ejecutar ciertas llamadas al sistema (*syscalls*).
- **GID (*Group ID*)**: Es el identificador numérico del grupo al que pertenece el proceso. Se utiliza de forma complementaria al *UID* para determinar y refinar los permisos de acceso a recursos compartidos del sistema.
- **Comando**: Es el nombre o referencia directa del fichero ejecutable binario que dio origen al proceso (por ejemplo, `/bin/ls` o `/usr/bin/python3`).

```
$localhost:$ ps aux
```

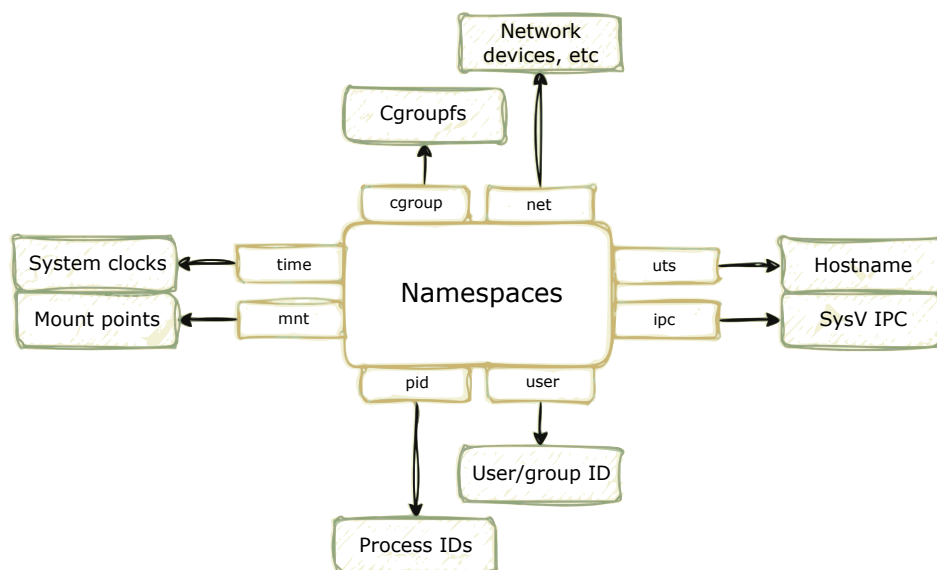
USER	PID	%CPU	%MEM	VSZ	RSS	TTY	STAT	START	TIME	COMMAND
root	1	0.7	0.3	22208	12444	?	Ss	12:37	0:04	/sbin/init
root	2	0.0	0.0	0	0	?	S	12:37	0:00	[kthreadd]
root	3	0.0	0.0	0	0	?	S	12:37	0:00	[pool_workque
root	4	0.0	0.0	0	0	?	I<	12:37	0:00	[kworker/R-kv
root	5	0.0	0.0	0	0	?	I<	12:37	0:00	[kworker/R-rc
root	7	0.0	0.0	0	0	?	I<	12:37	0:00	[kworker/R-sl
root	8	0.0	0.0	0	0	?	I<	12:37	0:00	[kworker/R-ne
root	11	0.1	0.0	0	0	?	I	12:37	0:00	[kworker/0:1-
...										
noel	3039	53.0	14.7	1217632252	573236	?	Sl	12:39	3:52	/usr/share/co
noel	3145	0.7	2.0	1215834136	78112	?	Sl	12:40	0:02	/usr/share/co
noel	3206	7.9	0.8	82304	31816	?	Sl	12:41	0:23	/home/noel/.v
root	3327	0.0	0.0	0	0	?	I<	12:42	0:00	[kworker/0:0H
root	3330	0.2	0.0	9760	3676	pts/0	S	12:42	0:00	su
root	3332	0.8	0.1	14356	7184	pts/0	S	12:42	0:01	sh
root	3619	0.0	0.0	0	0	?	I	12:44	0:00	[kworker/u20:
noel	3668	3.1	0.5	4260844	21540	?	Sl	12:45	0:02	/home/noel/.v
root	3683	0.7	0.0	0	0	?	I	12:45	0:00	[kworker/u16:
root	3726	700	0.1	10852	4264	pts/0	R+	12:46	0:00	ps aux

Código 2.1: Ejemplo de salida del comando `ps aux` en un terminal *Linux* mostrando el listado de procesos actuales en ejecución del sistema.

Si no consideramos, de momento, el concepto de *Namespace*, se puede asumir que todos estos recursos y atributos son de carácter **global**. Esto significa que si, por ejemplo, un proceso tiene asignado el *PID* 3039, ese identificador es único, absoluto y visible para cualquier otra herramienta de monitorización de la máquina (como los ficheros virtuales de */proc*). Del mismo modo, el usuario *root* (cuyo *UID* es 0) tiene privilegios absolutos sin restricciones sobre todos los procesos del sistema, a diferencia de un usuario estándar con un *UID* distinto (como el 1000), que estará mayormente restringido y solo podrá gestionar los procesos que él mismo haya iniciado.

Es en este punto donde la tecnología de *Namespaces* entra en escena, alterando por completo este paradigma. Un *Namespace* funciona como una capa de abstracción lógica que envuelve un recurso global del sistema y lo expone a un grupo de procesos como si fuera un recurso aislado y privilegiado, siendo el *kernel* el encargado de traducir estas identidades, atributos, permisos y recursos de forma totalmente transparente.

En la práctica, esto implica que un proceso ejecutándose dentro de un *Namespace* aislado puede tener asignado el *PID* 1, típicamente asociado al primer proceso ejecutado en la máquina, *systemd* o *init*. Sin embargo, desde la perspectiva exterior del *kernel*, ese mismo proceso es simplemente un proceso regular más con el *PID* 4500, por ejemplo.



Fuente: Gmkziz, *Understanding Linux Namespaces* (2024) [17].

Figura 2.8: Tipos de *Namespaces*.

Para lograr este nivel de flexibilidad, el *kernel* no aplica un aislamiento monolítico sobre todo el sistema operativo de una única vez. En su lugar, proporciona una arquitectura modular basada en distintos **tipos de *Namespaces***. Cada tipo se encarga de aislar un recurso global específico, permitiendo a los administradores (o a los atacantes) crear combinaciones de los mismos de forma independiente según la finalidad de su uso [18].

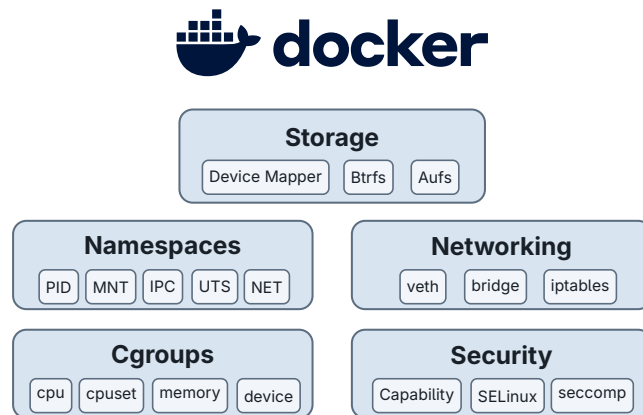
Estos son los principales tipos de *Namespaces* a implementar en un sistema *Linux* moderno:

- ***Mount* (mnt)**: Aísla los puntos de montaje del sistema de ficheros (concepto directamente relacionado con la Figura 2.7). Los procesos dentro de este *Namespace* poseen su propia visión de la jerarquía de directorios (*FHS*). Esto implica que montar o desmontar una unidad extraíble (como un dispositivo *USB*) en este espacio aislado no afectará ni será visible para el resto del sistema anfitrión.
- ***PID* (pid)**: Proporciona una jerarquía de procesos independiente. Como se puso de ejemplo antes, permite que el primer proceso ejecutado en ese espacio reciba el *PID* 1, aislando por completo la visibilidad hacia fuera del *Namespace*. De esta manera, se impide que los procesos internos a este espacio puedan interactuar con los procesos externos al mismo.
- ***User* (user)**: Aísla los identificadores de usuario (*UID*) y de grupo (*GID*). En este espacio se permite que un proceso tenga privilegios absolutos de *root* (*UID* 0) dentro del mismo, mientras que en el exterior siga siendo tratado y restringido como un usuario estándar sin privilegios.
- ***Network* (net)**: Virtualiza y aísla toda la pila de red del sistema. Cada *Namespace* de este tipo posee sus propias interfaces de red (incluyendo su propio *loopback*), puertos, direcciones *IP*, reglas de *firewall* y tablas de enrutamiento (*iptables* o *nftables*).
- ***UTS* (uts)**: Permite que un grupo de procesos tenga su propio nombre de máquina (*hostname*) y nombre de dominio *NIS*, de forma independiente al nombre real del servidor físico de la máquina.
- ***IPC* (ipc)**: Aísla los recursos de comunicación entre procesos, como las colas de mensajes (*POSIX*), los segmentos de memoria compartida y los semáforos.

Impide que un proceso de este espacio interfiera o intercepte los mensajes de procesos externos.

- **Cgroup** (cgroup): Encapsula la vista del directorio raíz de los grupos de control (*Control Groups*). Esto oculta la jerarquía de políticas y límites de recursos (como *CPU* o memoria) establecidos por el *kernel*, evitando que un proceso aislado los conozca o intente modificarlos.
- **Time** (time): Aísla los relojes internos del sistema, especialmente el reloj de arranque (*boot clock*) y el reloj monótonico (*monotonic clock*). Esto permite que este espacio aislado tenga su propia línea temporal, lo cual es muy útil para migrar contenedores entre diferentes servidores físicos sin alterar su percepción del tiempo de actividad (*uptime*).

La combinación estratégica de estos bloques es, precisamente, el motor tecnológico que hace posible la existencia de tecnologías de virtualización modernas como los contenedores (*Docker*), cuya arquitectura típica se ilustra en la siguiente Figura 2.9.



Fuente: Elaboración propia a partir de Docker, Inc., *What is Docker?* (2026) [19], usando el logotipo oficial de Docker [20].

Figura 2.9: Ejemplo de arquitectura *Docker*.

Todas estas tecnologías de aislamiento y abstracción, desde la gestión de memoria de la propia máquina hasta los *Namespaces*, residen y se gestionan en el núcleo del sistema operativo. Sin embargo, pese a su robustez, el *kernel* no es una entidad monolítica e inalterable. Para dotarlo de una alta flexibilidad y permitir la extensión de sus funcionalidades en tiempo real (o la alteración maliciosa de su comportamiento), este dispone de una arquitectura dinámica que permite inyectar y cargar código con máximos privilegios, como se explicará en el siguiente apartado.

2.1.5. Módulos del Kernel de *Linux* (*LKMs*)

Un **módulo del kernel** (*LKM*) es código objeto compilado (cuyo código fuente está escrito en lenguaje *C*) que puede ser cargado y descargado del espacio de memoria del núcleo (*kernel*) en tiempo de ejecución. Estos módulos permiten al *kernel* extender sus funcionalidades (como incorporar nuevos sistemas de ficheros) sin necesidad de reiniciar el sistema ni recompilar el *kernel* completo [21].

El desarrollo de *LKMs* presenta diferencias drásticas frente a la programación convencional en *espacio de usuario*. Al operar directamente en el nivel de máximos privilegios, los módulos no pueden acceder a la biblioteca estándar de *C* (`libc`) y, por consiguiente, a funciones típicas como `printf()` o `malloc()`. Además de esto, al formar parte del propio *kernel*, tampoco pueden utilizar llamadas al sistema (*syscalls*) a través de la interfaz estándar, sino que invocan directamente las funciones internas del *kernel*.

Por otro lado, no existe una salida estándar por consola como ocurre en *espacio de usuario*, y para suplirla, el *kernel* proporciona su propia macro de impresión de logs: `printk()`. Las trazas generadas mediante esta función se almacenan en un *buffer* circular interno y se registran en ficheros, típicamente en `/var/log/kern.log`. Asimismo, `printk()` exige clasificar cada traza con un nivel de prioridad (*log level*), abarcando eventos críticos o de error (`KERN_EMERG` y `KERN_ERR`), como también informativos o de depuración (`KERN_INFO` y `KERN_DEBUG`).

En cuanto a su compilación, se requieren las cabeceras del *kernel* (paquetes `linux-headers`) que coincidan con la versión exacta del sistema. Mediante un fichero `Makefile`, el código se compila generando un fichero objeto especial empaquetado con extensión `.ko` (*kernel object*). Una vez compilado, la gestión de su ciclo de vida se resume en cuatro fases básicas:

- **insmod**: Inserción del código en *Espacio de kernel*.
- **modinfo**: Reporte de los metadatos y dependencias del módulo.
- **dmesg**: Inspección de las trazas de ejecución del *kernel*.

- `rmmod`: Extracción del módulo y liberación de los recursos de memoria.

```
# 0. Root privileges
$ su

# 1. Compile the module using Makefile
$ make
make -C /lib/modules/6.12.41-current-bcm2711/build M=/home/lkms modules
make[1]: Entering directory '/usr/src/linux-headers-6.12.41-current-bcm2711'
  CC [M] /home/lkms/hw.o
  MODPOST /home/lkms/Module.symvers
  CC [M] /home/lkms/hw.mod.o
  LD [M] /home/lkms/hw.ko
make[1]: Leaving directory '/usr/src/linux-headers-6.12.41-current-bcm2711'

# 2. Load the module (.ko) into the kernel
$ insmod hw.ko

# 3. Inspect the module metadata
$ modinfo hw.ko
filename:      /home/lkms/hw.ko
version:       0.1
description:    A simple Hello world LKM
author:        Noel Rodriguez Perez
license:        GPL
srcversion:     2F2B1B95DA1F08AC18B09BC
depends:
vermagic:      6.12.41-current-bcm2711 SMP mod_unload modversions

# 4. Check the generated logs by printk
$ dmesg

# 5. Unload the module freeing its resources
$ rmmod hw
```

Código 2.2: Comandos básicos para la gestión de un *LKM*.

A nivel de código fuente, la estructura lógica de cualquier *LKM* se asienta sobre dos funciones principales. En primer lugar, una **función de inicialización o de carga** (`module_init()`), la cual se ejecuta automáticamente al insertar el módulo en el *kernel* (mediante `insmod`) y se encarga de reservar los recursos necesarios o crear los ficheros virtuales implicados. En segundo lugar, una **función de limpieza o de descarga** (`module_exit()`), invocada al eliminar el módulo cargado (`rmmod`) con el fin de liberar la memoria utilizada y evitar fugas o estados inconsistentes en el sistema.

Finalmente, para poder establecer un canal o interfaz de comunicación bidireccional

entre el *espacio de usuario* y el *espacio del kernel*, una técnica eficiente es la creación de ficheros virtuales a través del pseudo-sistema de ficheros de `/proc`. De esta manera, el módulo instancia un fichero virtual (por ejemplo, `/proc/mydev`) y define manejadores personalizados (*handlers*) que interceptan las llamadas al sistema que actúan sobre él, como `read` o `write`, y así poder declarar específicamente cómo interactuar con el *espacio de usuario*. Para implementar esta metodología en el *LKM*, se precisa del uso de funciones específicas de la *API* del *kernel* que aseguran su correcto funcionamiento de forma segura, como `copy_to_user()` y `copy_from_user()` para la transferencia de información entre espacios de memoria distintos [22].

En conclusión, el conjunto de todas las características estructurales y funcionales expuestas anteriormente constituye la base tecnológica de un sistema operativo *Linux*. Como se ha comentado previamente, este ha sido el entorno de trabajo elegido para desarrollar este proyecto, el cual se ha desplegado sobre una máquina física independiente y controlada. Esto permite garantizar un acceso directo a las capas más profundas del sistema operativo, como el *kernel* y sus módulos, mitigando cualquier riesgo sobre máquinas de producción o uso real.

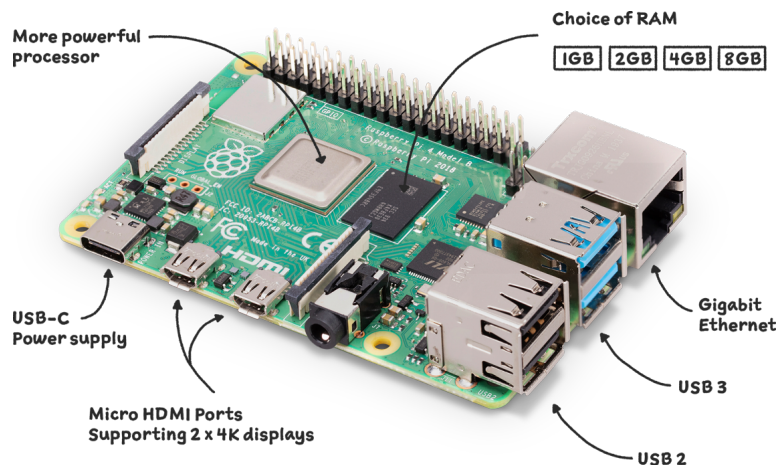
2.1.6. Entorno de desarrollo

Teniendo en cuenta todas las funcionalidades descritas en entornos *Linux*, se eligió realizar el desarrollo de este proyecto sobre una máquina física dedicada **Raspberry Pi 4 Model B de 4 GB de RAM**, perteneciente a la familia de arquitecturas *ARM*.

La elección de esta plataforma y de un sistema basado en *Linux* se debe principalmente a su versatilidad y al alto grado de control que ofrece sobre el sistema operativo, el *kernel* y las herramientas de bajo nivel necesarias para compilar, cargar y depurar *módulos del kernel*. Además, al tratarse de una máquina física aislada, se reducen los riesgos asociados a las pruebas con código en *Espacio del kernel*, evitando corromper un equipo de uso cotidiano y facilitando una experimentación más segura sobre un entorno real [23].

No obstante, la elección de esta placa no condiciona conceptualmente el mecanismo diseñado. La *Raspberry Pi* se emplea como plataforma experimental controlada, portable y adecuada para las pruebas. Pero las ideas desarrolladas se apoyan en

interfaces y mecanismos propios de *Linux*, por lo que podrían trasladarse a otros sistemas compatibles con pocas adaptaciones. Es un sistema operativo bastante flexible y portable, por lo que no habría que realizar apenas cambios en el código.



Fuente: Imagen oficial de Raspberry Pi 4 Model B [24].

Figura 2.10: Sistema físico empleado en el proyecto.

Sobre esta placa se instaló la distribución **Armbian**, en concreto una imagen para Raspberry Pi 4 basada en *Ubuntu Noble*, orientada a placas *single-board computer* y optimizada para arquitecturas *ARM*. Su elección se debe a que proporciona un entorno configurable, ligero y especialmente adecuado para este tipo de dispositivos, manteniendo al mismo tiempo la comodidad que ofrecen entornos *Ubuntu/Debian* y el gestor de paquetes `apt` [25, 26].



Fuente: Imagen oficial de Armbian [27].

Figura 2.11: Logo de la distribución Armbian para Raspberry Pi 4 Model B.

Y por otro lado, la preparación del entorno comenzó con la instalación externa de dicha imagen de la distribución *Armbian* en una tarjeta *microSD* de 32 GB y el arranque inicial de la placa tras introducirla. Una vez iniciado el sistema, se

actualizó y se instalaron las herramientas necesarias para la compilación en *C* (*gcc* y *make*), el desarrollo de *módulos del kernel* (*kmod*) y las cabeceras del *kernel* necesarias para su compilación (*linux-headers*), el uso de *OpenSSL* (*openssl* y *libssl-dev*) y el despliegue de contenedores para crear nuevos espacios de nombres con *Docker* (*docker.io*).

```
$ uname -r
6.12.41-current-bcm2711

$ sudo apt update
$ sudo apt install gcc make linux-headers-$(uname -r) \
> kmod openssl libssl-dev docker.io
```

Código 2.3: Instalación del entorno de desarrollo en *Armbian* basado en *Ubuntu Noble*.

Establecida la base tecnológica de los sistemas *Linux* y definido el entorno de desarrollo, a continuación se presentará formalmente el problema de seguridad que explotan amenazas como los *rootkits* y se plantearán los requisitos e implementaciones para la solución propuesta.

Capítulo 3

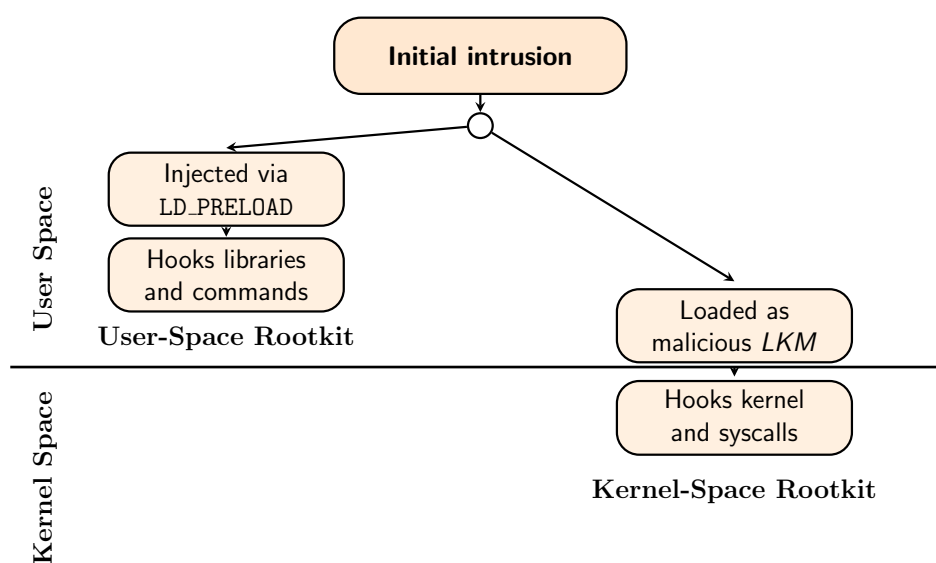
Estado del arte

«A collection of files that is installed on a host to alter the standard functionality of the host in a malicious and stealthy way.»

NIST CSRC, *Rootkit - Glossary* [28].

A partir de la definición expuesta y de lo mencionado en la introducción de la memoria, un *rootkit* es un tipo de *malware* diseñado para ocultar y manipular recursos del sistema, como procesos, ficheros, conexiones de red o mecanismos de persistencia. Su peligrosidad no reside únicamente en la escalada de privilegios que puede alcanzar, sino en su capacidad para falsear la visión del estado real de la máquina, comprometiendo la fiabilidad de su monitorización.

En sistemas *Linux*, los *rootkits* pueden clasificarse principalmente en dos grandes categorías según la capa en la que operen:



Fuente: Elaboración propia.

Figura 3.1: Clasificación general de *rootkits* en sistemas *Linux*.

3.1. Rootkits de Espacio de Usuario

Los *rootkits* de *espacio de usuario* actúan sobre programas, bibliotecas dinámicas y herramientas de usuario del sistema operativo. Su técnica más habitual consiste en aprovechar el mecanismo de enlazado dinámico LD_PRELOAD para cargar bibliotecas de forma maliciosa y así interceptar funciones (*hooking*) de uso común (como las de `libc`), alterando de esta forma el comportamiento de comandos como `ps`, `ls`, `top` o `netstat` [9]. Por otra parte, estas amenazas no se limitan únicamente a este mecanismo de actuación, pueden también reemplazar comandos legítimos por versiones manipuladas o apoyarse en técnicas de aislamiento y virtualización para presentar una visión alterada del sistema.

Si el *espacio de usuario* ya está comprometido, pretender detectar un *rootkit* de esta región con herramientas que se ejecutan también en *espacio de usuario* presenta importantes limitaciones e incluso puede llegar a ser totalmente ineficaz. Según el diseño del *rootkit*, puede llegar a evadir mecanismos de detección especializados en ocultación de procesos e incluso manipular la propia *shell* que los lanza o la información expuesta a través del sistema de ficheros `/proc` mediante *namespaces* y *bind mounts*.

Entre los ejemplos más destacados de esta categoría se encuentran varias familias de *rootkits* basados en LD_PRELOAD, como **Jynx** y **Jynx2**, que popularizaron este enfoque de ocultación en *espacio de usuario* (*userland*). Posteriormente, **Azazel** amplió estas técnicas con mecanismos de anti-depuración, ocultación de procesos, conexiones remotas y evasión frente a mecanismos de detección como `unhide` u otros comandos de monitorización como `ps` o `netstat`. Paralelamente, **BEURK** se presenta como un *preload rootkit* focalizado en anti-detección y ocultación de procesos, ficheros y accesos; mientras que **vlany** añade además persistencia, modificación del enlazador dinámico y funcionalidades de *backdoor*.

En este contexto, conviene destacar las herramientas comunes de detección, principalmente **unhide**, herramienta forense utilizada para detectar procesos ocultos mediante la búsqueda de discrepancias entre distintas vistas del sistema, por ejemplo, comparando la información proporcionada en `/proc` con la reportada por herramientas

como **ps**. Sin embargo, como se ha comentado anteriormente, una vez esta región se encuentre comprometida, su utilidad queda severamente limitada, ya que estas herramientas operan en el mismo nivel de privilegio (*espacio de usuario*) y a igualdad de condiciones, por lo que los atacantes tienen ventaja [29].

3.2. Rootkits de espacio del Kernel

Los *rootkits* de *espacio del kernel* se infiltran en el nivel de máximo privilegio del sistema y suelen materializarse mediante *módulos del kernel* (*LKMs*) maliciosos o a través de modificaciones directas del propio núcleo. Su posición privilegiada les permite alterar estructuras internas, ocultar procesos, ficheros o tráfico de red, interceptar llamadas al sistema sensibles, e incluso falsear la información suministrada a las herramientas de usuario.

Por ello, su detección desde capas superiores resulta especialmente compleja, y cualquier mecanismo defensivo que opere con menos privilegios parte de una posición bastante desventajosa. Por esta razón existe la necesidad de diseñar arquitecturas de monitorización autenticadas, que operen en regiones cercanas al *kernel* o, al menos, que no dependan de la información ya proporcionada por el propio entorno de usuario.

Herramientas como **chkrootkit** siguen incluyendo detección para familias de este tipo de amenazas como **Adore LKM**, **knark**, **SucKIT**, **Fu** o **Syslogk**, reflejando el modelo *LKM* como vector de ocultación y persistencia. El marco *MITRE ATT&CK* documenta casos como **REPTILE**, implementado como *rootkit Linux* de código abierto basado en *LKMs*, y **Drovorub**, un conjunto de *malware* para *Linux* que incluye *LKMs* para persistencia y control remoto [30].

En conclusión, los *rootkits* no constituyen una amenaza homogénea que actúa siempre de la misma manera, sino una familia de técnicas de ocultación distribuida en varias capas del sistema.

A la luz de lo expuesto, este proyecto se enfoca principalmente en la problemática de los *rootkits* de *espacio de usuario*, debido a su capacidad para falsear la información suministrada por herramientas que se ejecutan en esa misma región. La solución propuesta frente a esta limitación se centra en desplazar la obtención de información crítica del sistema a una capa de mayor confianza, implementando un *módulo del kernel* que genera y autentica dicha información, de forma que su integridad pueda verificarse posteriormente desde *espacio de usuario*.

Capítulo 4

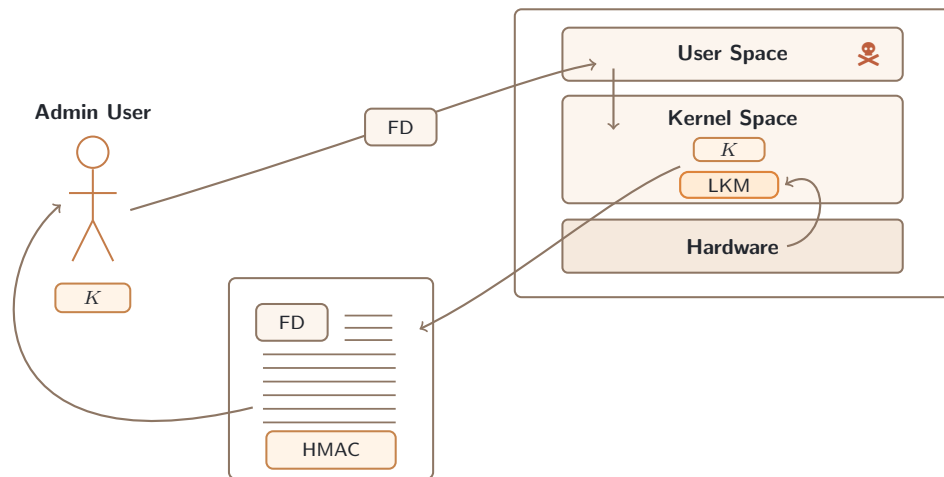
Planteamiento del problema y objetivos

Tras establecer previamente el contexto necesario sobre el entorno de los sistemas *Linux* y la amenaza que representan los *rootkits*, en este capítulo se describe la problemática concreta que motiva el presente proyecto y los objetivos que se persiguen con el mismo.

4.1. Descripción del problema

Uno de los problemas fundamentales en el ámbito de la seguridad de sistemas operativos consiste en determinar hasta qué punto la información crítica ofrecida al administrador sobre el estado de una máquina puede considerarse fiable una vez que el entorno ha sido comprometido, en particular, el *espacio de usuario*. Con base en lo expuesto en el estado del arte, los *rootkits* de esta región pueden interceptar funciones de bibliotecas fundamentales y manipular la salida de herramientas como `ps`, `ls` o `netstat`, falseando la visión que el operador tiene del sistema y dificultando la detección de actividad maliciosa [9].

Este escenario resulta especialmente problemático en sistemas *Linux*, debido a que gran parte de sus tareas de monitorización y administración se apoyan en utilidades de *espacio de usuario*, donde la información se expone a través de interfaces de alto nivel. Si dicha región se encuentra comprometida, confiar en ella para verificar el estado del sistema deja de ser una estrategia sólida. Consecuentemente, aparece la necesidad de obtener información sensible desde una capa con mayores privilegios, dificultando su manipulación y, además, dotar a esa información de un mecanismo que permita comprobar posteriormente su integridad y autenticidad.



Fuente: Elaboración propia.

Figura 4.1: Mecanismo de reporte autenticado entre *espacio de usuario* y *Espacio de kernel*.

En este proyecto, se aborda un caso concreto y representativo, tomando como información crítica un listado de todos los procesos del sistema junto con sus metadatos más relevantes, entre ellos, identificadores globales y locales de los procesos, de los usuarios que los ejecutan y referencias a sus *namespaces*.

Esta elección no es arbitraria, ya que mediante esta información se puede obtener una visión global del estado del sistema y, por tanto, compararla con los datos obtenidos por otras herramientas para detectar posibles indicios de manipulación y ocultación en dicha región. Sin embargo, si un atacante consigue manipular la información mostrada al administrador, podría ocultar procesos maliciosos o alterar la percepción del estado real del sistema, dificultando así su detección.

En conclusión, el problema del proyecto puede formularse del siguiente modo: **cómo proporcionar al administrador de un sistema *Linux* una visión fiable de información crítica del sistema cuando el *espacio de usuario* puede haber sido comprometido por *malware* con capacidades de persistencia y ocultación, como los *rootkits*.**

4.2. Modelo de amenaza

El presente proyecto aborda específicamente la amenaza de los *rootkits* de **espacio de usuario** en sistemas *Linux*. Estas son herramientas de *malware* diseñadas para mantener el control sobre esta región del sistema, pudiendo alterar la salida de herramientas de monitorización críticas mediante técnicas de interceptación y modificación de código [9].

Esta capacidad de manipulación permite al atacante ocultar:

- **Procesos maliciosos:** Ocultar la ejecución de herramientas de control remoto o *backdoors*.
- **Ficheros y conexiones de red:** Enmascarar conexiones sospechosas hacia servidores remotos o ficheros de persistencia.
- **Accesos a recursos:** Falsear registros de acceso a ficheros o información de permisos.
- **Actividad de usuarios:** Falsificar información sobre sesiones activas o comandos ejecutados.

En este contexto, el modelo de amenaza se centra en un *espacio de usuario* potencialmente comprometido, capaz de ocultar o falsear información visible para el administrador. A continuación, se detallan los supuestos de seguridad considerados.

4.2.1. Contexto de operación y supuestos de seguridad

El proyecto asume un escenario en el que el *espacio de usuario* ha sido comprometido, permitiendo al atacante ejecutar *malware* con capacidades de persistencia y ocultación. En este contexto, se plantean los siguientes supuestos:

- **El *espacio de usuario* está bajo control del atacante.** El administrador no puede confiar en la salida de herramientas de monitorización como `ps`, `ls` o `netstat`, ya que pueden haber sido interceptadas y manipuladas por un *rootkit*.

- **El *espacio del kernel* permanece íntegro.** El proyecto asume que no hay compromiso de *rootkits* de *espacio del kernel*, manteniendo la integridad del código ejecutado en *Ring 0*.
- **El administrador es de confianza.** Se asume que la clave simétrica compartida para la autenticación HMAC es conocida únicamente por el administrador de confianza del sistema.
- **La comunicación es unidireccional desde el *kernel* a *espacio de usuario*.** El *kernel* no confía en datos recibidos desde *espacio de usuario* salvo el identificador del fichero de salida, siendo el vector principal de entrada.
- ***Hardware* y *Firmware* íntegros.** Se asume que tanto el *hardware* como el *firmware* del sistema no han sido comprometidos.
- **Cadena de confianza de arranque.** Se concibe una cadena de confianza que permite mantener la integridad del *kernel* y del *LKM*, por ejemplo mediante mecanismos de firmado, evitando la ejecución de componentes no autorizados dentro de la base fiable del sistema.

4.2.2. Capacidades y limitaciones del atacante

Capacidades del atacante dentro del escenario modelado:

- Interceptar y modificar la salida de herramientas de usuario mediante *LD_PRELOAD* y *hooking* de funciones de bibliotecas como *libc*.
- Ocultar procesos, ficheros, conexiones de red y actividad maliciosa desde herramientas de monitorización de *espacio de usuario*.
- Ejecutar código malicioso con privilegios arbitrarios en *espacio de usuario*.
- Manipular información en */proc* mediante *bind mounts* y *namespaces*.

Limitaciones del atacante dentro del escenario modelado:

- No puede modificar el *kernel* ni cargar *LKMs* maliciosos (no accede a *Ring 0*).
- No puede interceptar llamadas al sistema ni modificar la *Tabla de Llamadas del Sistema* (*sys_call_table*).

- No puede falsificar el HMAC-SHA256 sin conocer la clave simétrica compartida entre el *kernel* y el administrador del sistema.
- La clave simétrica se carga en el *kernel* durante el proceso de arranque sin que el atacante tenga acceso a ella.

4.2.3. Cobertura defensiva

Lo que el sistema protege:

- **Ocultación de procesos:** Proporciona visibilidad desde *kernel*, imposible de manipular desde *espacio de usuario*.
- **Integridad de información:** Mediante autenticación criptográfica HMAC-SHA256, verificable matemáticamente mediante la misma clave con la que se codifica.
- **Detección de manipulación:** Permite comparar información autenticada del *kernel* con herramientas comprometidas de *espacio de usuario*.

Lo que el sistema NO protege:

- Amenazas de *rootkits* de *espacio del kernel* que operen en *Ring 0*.
- Compromisos post-carga del *LKM* mediante modificación de memoria del *kernel*.
- Exposición o interceptación de la clave simétrica compartida entre el *kernel* y el administrador.
- Procesos o actividad maliciosa anterior a la carga del *LKM*.

4.3. Objetivos

El objetivo principal del presente proyecto consiste en **diseñar un mecanismo de reporte seguro para sistemas *Linux* capaz de proporcionar al administrador una visión más fiable de información crítica del sistema, concretamente del**

estado de sus procesos activos, especialmente en entornos comprometidos por *rootkits* de *espacio de usuario*. Para ello, la solución propuesta desplaza la obtención de la información crítica al *Espacio de kernel* mediante un *LKM*, incorporando además un mecanismo criptográfico que permita verificar posteriormente su integridad y autenticidad desde *espacio de usuario* por el propio administrador.

Con base en este objetivo general, el proyecto se concreta en los siguientes objetivos:

4.3.1. Objetivos funcionales

Los objetivos funcionales que describen **qué debe hacer** la solución propuesta son:

- **Obtener información crítica del sistema desde *espacio del kernel*.** Se precisa del diseño de un mecanismo de reporte seguro desde el *Espacio de kernel*, mediante un *LKM*, capaz de recopilar información de los procesos activos sin depender de herramientas de *espacio de usuario*.
- **Construir una vista global y estructurada de los procesos del sistema.** Dicho mecanismo de reporte debe generar un listado estructurado de los procesos activos del sistema, que incluya los metadatos más relevantes de los mismos: *UID* global, *UID* local, identificador del espacio de nombres de usuario, *PID* global, *PID* local, identificador del espacio de nombres de procesos, *GID* y comando asociado.
- **Transferir la información obtenida desde el *Espacio de kernel* a un canal accesible desde *espacio de usuario*, cuando el administrador lo solicite.** Se debe habilitar un mecanismo de comunicación controlado entre ambas regiones, de modo que el *kernel* pueda detectar la solicitud de información y transferirla. En este caso, se articula la transferencia de información enviando la referencia de un fichero fijado por el administrador, mediante su descriptor de fichero, desde *espacio de usuario* a *espacio del kernel*.
- **Autenticar criptográficamente el contenido generado en *Espacio de kernel*.** El reporte producido debe ir acompañado de un mecanismo que demuestre su autenticidad e integridad, implementado en este proyecto mediante un *HMAC* basado en *SHA-256*, calculado mediante una clave simétrica *K* a partir de la información del listado de procesos.

- **Permitir una verificación posterior de la información solicitada al *kernel* desde *espacio de usuario*.** El administrador debe poder verificar la autenticidad e integridad del reporte generado por el *LKM*, pudiendo realizar dicha comprobación posteriormente en otra máquina no comprometida tras copiar los datos obtenidos. Para ello, se crea un *marco de confianza* en el que tanto el administrador como el *kernel* deben compartir una clave simétrica *K*, y en este mismo contexto, se implementa en *espacio de usuario* un mecanismo que recalcula el *HMAC* correspondiente y comprueba si el resultado coincide con el valor autenticado emitido por el *kernel*.
- **Facilitar la detección de amenazas de ocultación y manipulación.** El sistema de reporte desarrollado debe permitir comparar la información autenticada obtenida desde *espacio del kernel* con la ofrecida por herramientas de monitorización de *espacio de usuario*, posibilitando la identificación de indicios de ocultación o manipulación.
- **Validar el funcionamiento del prototipo.** El mecanismo diseñado debe probarse en un entorno *Linux* real, incluyendo procesos y usuarios pertenecientes a distintos *namespaces*, con el fin de demostrar que el administrador obtiene correctamente la información crítica y que su autenticación puede verificarse satisfactoriamente.

4.3.2. Objetivos no funcionales

Los objetivos no funcionales que describen **cómo debe hacerlo** el sistema diseñado son:

- **Integridad y autenticidad verificables de la información crítica.** El sistema de reporte no debe limitarse a exponer datos del *kernel* únicamente, sino que debe garantizar que dichos datos puedan comprobarse posteriormente mediante un mecanismo criptográfico de autenticación, en este caso un *HMAC* basado en SHA-256.
- **Robustez en la comunicación entre espacios.** La interfaz entre *espacio del kernel* y *espacio de usuario* debe incorporar validaciones de toda la información que entre dentro del *kernel*, ya sean sobre permisos, descriptores de fichero, modos de apertura o límites de memoria, reduciendo el riesgo de usos incorrectos o comportamientos inseguros que puedan comprometer la estabilidad del sistema.

- **Seguridad.** El sistema implementado debe minimizar la confianza depositada en el *espacio de usuario*, de forma que la información crítica sea generada prioritariamente en una región con mayor privilegio, como el *espacio del kernel*, y se acompañe de mecanismos criptográficos y autenticables que permitan detectar modificaciones no autorizadas.
- **Modularidad y bajo impacto en el *kernel* del sistema.** El mecanismo de reporte seguro debe integrarse como un *módulo del kernel* (*LKM*), evitando modificar el núcleo de forma permanente y facilitando tanto su despliegue controlado como su descarga del sistema, además de cumplir con el objetivo anterior de generar la información crítica desde una región con altos privilegios para favorecer su integridad.
- **Portabilidad con entornos *Linux*.** El diseño debe apoyarse en interfaces y mecanismos estándar del ecosistema *Linux*, como */proc*, la *Crypto API* del *kernel*, utilidades convencionales de compilación en lenguaje C y las cabeceras necesarias para la compilación de *LKMs*.
- **Reproducibilidad del prototipo.** El prototipo debe poder compilarse, cargarse, descargarse y verificarse mediante un procedimiento claro y preciso, de forma que los resultados obtenidos durante el desarrollo puedan repetirse y analizarse posteriormente.
- **Formato de salida.** La información proporcionada por el mecanismo de reporte debe presentarse en una estructura legible y ordenada, facilitando su inspección, análisis y su comparación con la salida de herramientas habituales de monitorización del sistema.

4.4. Metodología

La metodología seguida en este proyecto ha sido principalmente de tipo **Espiral** (*Spiral*). Esta elección se debe a que, aunque la problemática a abordar estaba claramente definida desde el inicio, la solución técnica concreta a la misma no lo estaba. Durante las primeras fases del proyecto todavía no se conocían con precisión varios aspectos necesarios para poder diseñar una solución adecuada al problema. Por esta razón, se optó por una metodología en espiral, la cual permite investigar, diseñar,

implementar y validar de forma iterativa y progresiva los conocimientos requeridos para desarrollar una serie de prototipos funcionales que sirvan como solución a la problemática planteada.

En primer lugar, se estudió a fondo el problema planteado y el marco tecnológico donde se desarrolla el prototipo, así como los fundamentos técnicos necesarios para empezar a diseñar el *LKM*. Posteriormente, se fueron construyendo varios prototipos incrementales, incorporando en cada vuelta nuevos elementos y funcionalidades sobre la base validada del prototipo previo, lo que permitió construir progresivamente el componente hasta lograr las funcionalidades definidas. Finalmente, en cada iteración se realizaron las validaciones necesarias en el propio entorno *Linux* de desarrollo, apoyándose además en dos componentes de *espacio de usuario* que permitieron comprobar el correcto funcionamiento del sistema de reporte seguro creado.

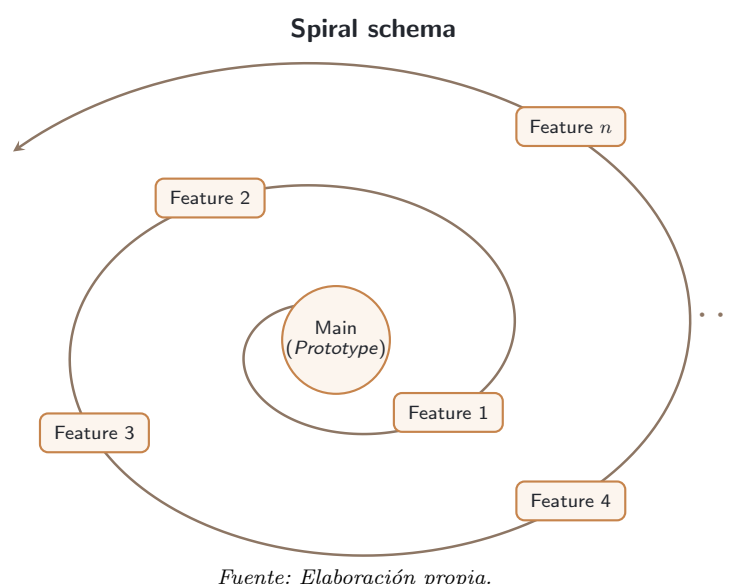


Figura 4.2: Esquema de la metodología tipo *Spiral* (espiral) empleada en el proyecto.

En conclusión, puede considerarse que el proyecto ha seguido una metodología en espiral con refinamiento progresivo sobre lo desarrollado en iteraciones anteriores, ya que el diseño final del prototipo se fue consolidando a medida que se avanzaba en el estudio técnico, la implementación y la validación de cada nuevo prototipo.

4.5. Plan de trabajo

La planificación del trabajo seguido en el proyecto ha estado directamente alineada con la metodología en espiral descrita en el apartado anterior, avanzando mediante iteraciones sucesivas que dependían de los conocimientos y resultados obtenidos en las anteriores. Durante todo el desarrollo del proyecto se mantuvieron reuniones semanales con el tutor, de una duración media aproximada de entre 30 y 60 minutos, en las que se revisaba el trabajo realizado en la iteración en curso, se resolvían dudas y se proponían las correcciones pertinentes para refinar cada prototipo. Una vez completada una iteración, se fijaban los objetivos parciales a realizar en la siguiente.

La primera fase comenzó en julio de 2025 y estuvo centrada en el estudio del entorno de desarrollo (*Linux*), del problema a abordar (*rootkits de espacio de usuario*) y de la base técnica necesaria para empezar a plantear una solución práctica viable (programación de *LKMs* y mecanismos criptográficos). Ya consolidada esta base, en septiembre de 2025 se inició la implementación del componente principal del proyecto. Desde ese momento hasta enero de 2026 se fue desarrollando progresivamente el prototipo final del sistema de reporte seguro a medida que se profundizaba más en los conocimientos técnicos requeridos, incorporando poco a poco las distintas funcionalidades definidas, realizando las validaciones necesarias y refinando el código mediante ajustes, comentarios y correcciones hasta alcanzar una versión funcional.

Una vez el prototipo estuvo finalizado, se empezó con la elaboración de la memoria, lo que requirió continuar investigando en varios de los temas que se desarrollan en la misma, tanto para fundamentar correctamente el problema y la solución propuesta como para documentar con rigor técnico el trabajo realizado. Durante el desarrollo de la memoria, se utilizó adicionalmente de forma auxiliar una herramienta de IA generativa, concretamente *Gemini 3 Pro*, para generar algunas imágenes vectoriales esquemáticas e ilustrativas, a partir de los conceptos previamente explicados mediante un *prompt* bien estructurado, así como para detectar y corregir errores de redacción y gramática de dicha memoria. La memoria fue terminada a finales del mes de mayo del año 2026.

En conclusión, el proyecto ha seguido una planificación progresiva y ordenada, con seguimiento semanal y consecución de objetivos parciales hasta completar el resultado final del TFG, incluyendo las validaciones necesarias.

Una vez definido el problema de seguridad a abordar, establecidos los objetivos del proyecto y descrita la metodología seguida para alcanzarlos, el siguiente capítulo se centrará en el desarrollo y explicación técnica de la solución propuesta. En él se detallarán el diseño de los distintos prototipos desarrollados hasta obtener la versión final funcional del sistema de reporte seguro, las decisiones de implementación adoptadas y el proceso seguido para construir y validar el prototipo final.

Capítulo 5

Diseño y Desarrollo

Una vez definida la problemática abordada, los objetivos del proyecto y el marco tecnológico en el que se desarrolla, en este capítulo se describe el proceso seguido para construir la solución propuesta. Se presentan los prototipos desarrollados hasta alcanzar la versión final funcional, describiendo su implementación y validación, así como los errores encontrados y las soluciones adoptadas.

Cabe resaltar que el código fuente oficial del *kernel* de *Linux* se encuentra disponible en <https://www.kernel.org>, y que la versión del *kernel* utilizada en el proyecto es la 6.12.41-current-bcm2711. Esta precisión resulta importante, ya que el desarrollo y las referencias consultadas se basan en el código fuente de esta versión, pudiendo variar tanto la funcionalidad como la localización de algunos recursos respecto a otras versiones del *kernel*.

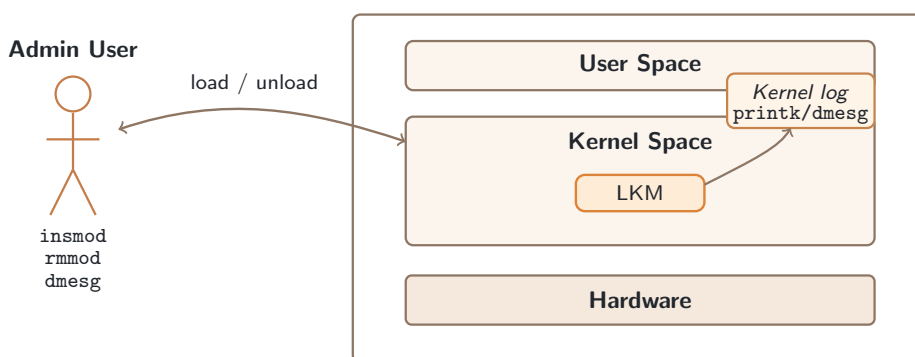
5.1. Prototipos, Implementaciones y Validaciones

En esta sección se describen los prototipos desarrollados durante el proyecto desde una perspectiva conceptual, explicando los conocimientos teóricos aplicados en cada etapa y la forma en la que dichos conceptos se integran progresivamente en la solución final. El objetivo no es reproducir aquí el código fuente, sino justificar el diseño seguido, detallar el funcionamiento de cada mecanismo y relacionar cada prototipo con la necesidad técnica que resuelve dentro del sistema de reporte seguro propuesto.

La implementación en detalle, el código y scripts utilizados junto con las salidas de validación generadas se recogen en el Apéndice A. Por otra parte, el código fuente completo del proyecto se encuentra también disponible en el repositorio público de *GitHub* <https://github.com/nowelito28/TFG>.

5.1.1. *Hello World*

El primer prototipo desarrollado consistió en un *LKM* mínimo de tipo *Hello World*. Se trata de una primera toma de contacto con programación dentro del *kernel*, validando el flujo básico de trabajo necesario para desarrollar código ejecutado en esta región.



Fuente: Elaboración propia.

Figura 5.1: Flujo básico de carga, ejecución y descarga de un *LKM*.

El componente de *LKM* es la principal herramienta utilizada en el proyecto, ya que todos los prototipos se construyen como módulos externos y mantienen el mismo ciclo general de compilación, carga, prueba y descarga sin tener que reiniciar el sistema cada vez que se quiera ejecutar.

La compilación del módulo se apoya en *kbuild*, el sistema de construcción utilizado por el propio *kernel* de *Linux*. En lugar de compilar el fichero fuente como un programa de usuario convencional, el *Makefile* delega la construcción en el árbol de cabeceras del *kernel* activo mediante la ruta `/lib/modules/$(uname -r)/build`. De esta forma, el módulo se compila con la misma versión del *kernel* que lo va a cargar, en este caso `6.12.41-current-bcm2711`. El detalle del fichero de compilación utilizado se recoge en el Código A.1 del Apéndice A.

Desde el punto de vista estructural, el módulo necesita incluir las cabeceras que exponen las primitivas básicas de desarrollo en *espacio del kernel*. Entre ellas se encuentran las necesarias para declarar módulos (*LKMs*), utilizar funciones del núcleo y registrar las rutinas de inicialización y salida. Además, el prototipo define metadatos mediante macros como la licencia, el autor, la descripción y la versión. Estos metadatos no son puramente informativos, ya que la licencia indicada afecta a cómo el *kernel* considera el módulo cargado. Concretamente, declarar una licencia compatible con *GPL* evita que el núcleo marque el módulo como *tainted* por incompatibilidad de licencia, lo cual advierte que el *kernel* ha sido modificado o ejecuta componentes que pueden dificultar el diagnóstico de errores. La declaración concreta de cabeceras y metadatos se muestra en el Código A.2 del anexo.

El comportamiento del módulo se organiza principalmente mediante dos puntos de entrada. El primero es la función de inicialización (asociada a la macro `module_init()`), ejecutada automáticamente al cargar el módulo mediante `insmod`. Y el segundo es la función de salida (asociada a la macro `module_exit()`), ejecutada cuando el módulo se descarga del *kernel* mediante `rmmmod`. Estos puntos representan el ciclo de vida mínimo de un *LKM*: reservar o inicializar recursos al entrar en el *kernel* (*carga*), y liberar o finalizar el trabajo al salir (*descarga*). En este primer prototipo todavía no se reserva memoria ni se crean interfaces de comunicación, el objetivo es únicamente demostrar cómo funcionan ambas fases del módulo y su flujo de ejecución. Esta lógica se detalla en A.3 y A.4.

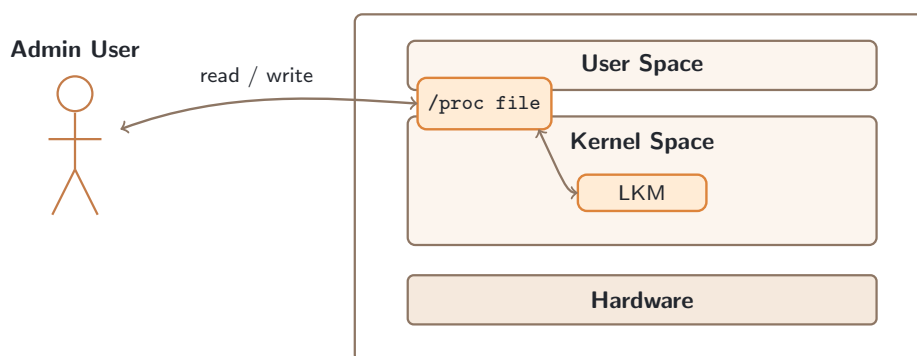
Para validar que el código se ejecuta realmente dentro del *kernel*, se utiliza `printk`. Esta función es el mecanismo habitual para registrar mensajes/trazas desde *espacio del kernel*, de forma análoga a cómo un programa de usuario podría escribir por la salida estándar, pero integrado en el sistema de registros del núcleo. Cada mensaje puede incluir un nivel de prioridad dependiendo de la traza a escribir y su función, como `KERN_INFO`, utilizada para trazas meramente informativas, o `KERN_ERR`, para registrar errores. Posteriormente, dichas trazas pueden consultarse desde *espacio de usuario* mediante herramientas como `dmesg` o el fichero de *logs* del *kernel* `/var/log/kern.log`, accesible desde *espacio de usuario*. Estos métodos de validación son importantes porque permiten confirmar simultáneamente estos aspectos: que el módulo se carga o descarga, que el *kernel* ejecuta sus rutinas de inicialización y salida correctamente, y si fuera necesario, ver trazas de depuración de la lógica implementada en dicho módulo.

Finalmente, una vez programado el *LKM*, se procede a validar su funcionamiento, empezando por su compilación, carga en el *kernel*, comprobar los mensajes generados (*logs*), consultar sus metadatos con `modinfo` y descargarlo verificando la ejecución de la rutina de salida. El procedimiento completo de comandos y salidas se encuentra recogido en el Código A.5.

Con ello, se desarrolló y validó la base técnica sobre la que se construyen los siguientes prototipos.

5.1.2. Fichero virtual `/proc/fdev`

Una vez validado el ciclo de vida básico de un *LKM*, el siguiente prototipo incorporó una interfaz de comunicación entre *espacio de usuario* y *espacio del kernel*. El objetivo de esta etapa fue exponer un punto de interacción controlado que permitiese enviar datos al módulo y dar pie a una gestión personalizada de los mismos dentro del *kernel*. Para ello se utilizó el sistema de ficheros virtual `/proc` como interfaz de entrada al *LKM*, mediante el fichero `/proc/fdev`.



Fuente: Elaboración propia.

Figura 5.2: Comunicación básica entre *espacio de usuario* y *espacio del kernel* mediante un fichero virtual en `/proc`.

En primer lugar, para poder crear y gestionar este fichero virtual fue necesario ampliar las cabeceras del módulo con soporte para dos aspectos principales: la creación

de entradas en el sistema de ficheros virtual `/proc` y la transferencia controlada de datos entre *espacio de usuario* y *espacio del kernel*. El primer grupo permite registrar el fichero virtual y asociarle operaciones propias, mientras que el segundo proporciona primitivas específicas para copiar información entre ambas regiones de memoria sin desreferenciar directamente punteros procedentes de *espacio de usuario*. Esta separación es importante, ya que el *kernel* y los procesos de usuario trabajan en contextos de memoria distintos, por lo que el acceso directo a direcciones de usuario desde el *kernel* debe realizarse de forma correcta, evitando accesos inseguros o corruptos. El conjunto concreto de cabeceras incorporadas en este prototipo se recoge en el Código A.6 del Apéndice A.

En este prototipo se crea una entrada específica dentro de `/proc` mediante el fichero `/proc/fdev`, que se registra durante la carga del módulo mediante la función de inicialización. Conceptualmente, el módulo solicita al subsistema `/proc` que cree un nuevo nodo visible desde *espacio de usuario* y que asocie a dicho nodo una tabla de operaciones. Esta tabla actúa como el contrato entre las operaciones de fichero invocadas desde *espacio de usuario* y las funciones internas del módulo. Así, una lectura sobre `/proc/fdev` se redirige al *handler* de lectura, mientras que una escritura sobre esa misma ruta se redirige al *handler* de escritura. Al descargar el módulo, la rutina de salida elimina la entrada para no dejar referencias inválidas dentro de `/proc`. La referencia a la entrada virtual, su tabla de operaciones y el registro mediante `proc_create` se detallan en el Código A.7.

En relación con lo anterior, se puede considerar que el núcleo de este prototipo se encuentra en los *handlers* de lectura y escritura. El *handler* de escritura recibe los bytes proporcionados desde *espacio de usuario*, los copia a un *buffer* temporal en memoria del *kernel* y los interpreta como valores de prueba. En sentido contrario, el *handler* de lectura construye una respuesta dentro del *kernel* y la devuelve al *buffer* de usuario. Para realizar estas transferencias se emplean funciones específicas como `copy_from_user` y `copy_to_user`, que además permiten detectar fallos de acceso y devolver errores en lugar de provocar accesos inseguros. La implementación concreta de estos manejadores se encuentra en el Código A.8.

Otro aspecto relevante de esta implementación es el uso del puntero de posición del fichero. Aunque `/proc/fdev` no representa un fichero persistente, las operaciones de lectura y escritura reciben igualmente una posición asociada a la apertura. El prototipo

utiliza dicha posición para imponer una semántica sencilla de una única operación efectiva por apertura (*single-shot*), evitando que se realicen varias lecturas consecutivas o que una escritura se procese varias veces dentro del mismo flujo. Esta decisión simplifica la validación y hace que el comportamiento observado desde herramientas como `cat` sea controlado y predecible.

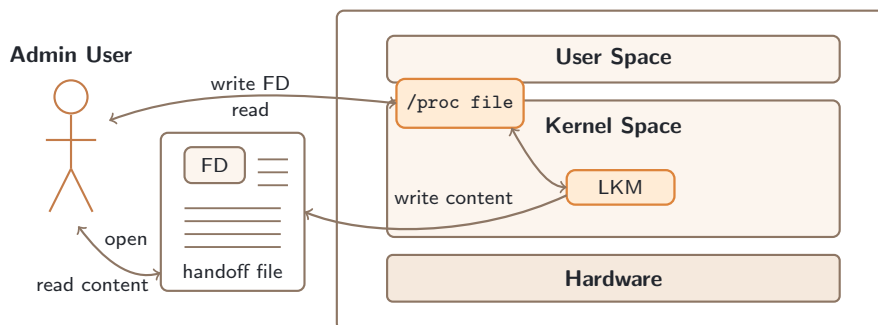
Para finalizar este prototipo, se realizó un proceso de validación que consistió en compilar el módulo, cargarlo, comprobar la creación de `/proc/fdev`, escribir valores desde *espacio de usuario*, leerlos de nuevo, revisar las trazas generadas en el registro del *kernel* y descargar el *LKM*. Con ello se comprobó que la entrada virtual se creaba y eliminaba correctamente, que las operaciones de lectura y escritura eran redirigidas a los *handlers* del módulo correspondientes, y que la transferencia de información entre ambas regiones funcionaba de forma controlada. El procedimiento completo de comandos y salidas se recoge en el Código A.9.

En conclusión, este prototipo establece la primera interfaz funcional entre el administrador y el *módulo del kernel*. A partir de este punto, los siguientes prototipos reutilizan `/proc/fdev` como canal de entrada para transferir información desde *espacio de usuario* hacia el *LKM*, ampliando progresivamente la lógica ejecutada dentro del módulo.

5.1.3. *File Handoff*

Una vez establecida la comunicación básica mediante el fichero virtual `/proc/fdev`, el siguiente prototipo incorporó un mecanismo de *File Handoff*. El objetivo de esta etapa fue permitir que el *LKM* no se limitase a recibir datos simples desde *espacio de usuario*, sino que pudiera resolver la referencia de un descriptor de fichero (*FD*) abierto por el administrador y utilizarlo como vía de comunicación hacia *espacio de usuario*.

Un *FD* es un identificador entero de un fichero asociado a la tabla de ficheros abiertos del proceso que lo posee, pudiendo referenciar un fichero regular, una conexión de red, etc. Por tanto, cuando un proceso en *espacio de usuario* abre un fichero, el sistema le devuelve un entero que sirve como referencia local a una estructura interna del *kernel* que representa dicho fichero. El mecanismo de *File Handoff* consiste en



Fuente: Elaboración propia.

Figura 5.3: Transferencia de un *File Descriptor* de un fichero real al *LKM* para establecer una vía de comunicación desde *espacio del kernel* hacia *espacio de usuario*.

enviar ese identificador al módulo a través de `/proc/fdev`, para que el *kernel* pueda resolverlo y operar sobre el fichero referenciado.

Para soportar este mecanismo fue necesario ampliar las cabeceras del módulo con funcionalidades relacionadas con el subsistema de ficheros, la resolución de descriptors, la conversión segura de cadenas a enteros y la validación de permisos. Este conjunto de dependencias permite pasar de una comunicación basada en texto a una interacción con objetos reales del *VFS* del *kernel*. Las cabeceras concretas incorporadas en este prototipo se recogen en el Código A.10 del Apéndice A.

En este prototipo, la función de lectura mantiene un papel auxiliar. En lugar de devolver valores de prueba como en el prototipo anterior, informa al administrador de que el módulo se encuentra preparado para recibir un *FD* y ejecutar el mecanismo de *File Handoff*. De esta manera, la lectura de `/proc/fdev` funciona como una comprobación sencilla del estado del *LKM*, mientras que la lógica principal queda concentrada en la operación de escritura. La implementación concreta de esta función se muestra en el Código A.11.

Por otro lado, la escritura sobre `/proc/fdev` es el punto de entrada principal del prototipo. El administrador abre o crea el fichero destino en modo escritura y añadido al final (modo *append*) desde *espacio de usuario*, obtiene su *FD*, lo convierte a texto y lo escribe en `/proc/fdev`. Desde el punto de vista del *LKM*, esa cadena de texto escrita

se recibe como una secuencia de *bytes*, que se copia a memoria del *kernel*, se interpreta como un entero y se comprueba si el *FD* extraído es válido. El programa auxiliar de *espacio de usuario* encargado de abrir el fichero destino y enviar el *FD* al módulo se recoge en el Código A.15.

Una vez recibido el descriptor, el módulo utiliza una primitiva del *kernel* para transformar el entero en una referencia interna de tipo `struct file`. Este paso es importante porque el *FD* por sí solo no permite escribir directamente sobre el fichero; primero se debe resolver mediante la tabla de ficheros abiertos del proceso que invoca la operación. Si la resolución tiene éxito, el *LKM* obtiene una referencia válida al objeto `struct file` del fichero y debe liberarla posteriormente cuando deje de utilizarla. Esta lógica principal de recepción del descriptor, conversión, resolución y liberación se detalla en el Código A.14.

Antes de escribir desde *espacio del kernel*, el prototipo comprueba que el fichero cumple la política definida. En concreto, se exige que el fichero haya sido abierto con capacidad de escritura y en modo *append*. Esta decisión evita sobrescrituras accidentales y problemas con el control del puntero de escritura, haciendo así que la información generada por el módulo se añada siempre al final del fichero destino. Además, se comprueban los permisos efectivos sobre el fichero para verificar que la operación de escritura y anexo está autorizada. La función auxiliar encargada de estas comprobaciones se referencia en el Código A.13.

La escritura efectiva sobre el fichero no se realiza con las funciones habituales de *espacio de usuario*, sino mediante la funcionalidad de escritura disponible dentro del *kernel* que utilice datos en memoria en esta misma región (`kernel_write`). Para ello se implementa una función auxiliar (`write_full`) que encapsula una única llamada a `kernel_write` y centraliza la comprobación de errores, verificando que la operación no falle y que el número de *bytes* escritos coincida con el tamaño solicitado. La función auxiliar de escritura se encuentra en el Código A.12.

Finalmente, la validación del prototipo consistió en compilar y cargar el módulo, comprobar que `/proc/fdev` quedaba disponible, crear el fichero destino mediante la utilidad de usuario, enviar su *FD* al módulo y verificar que el contenido de prueba era escrito desde *espacio del kernel* correctamente. También se revisaron las trazas del *kernel* y se descargó el módulo al finalizar la prueba. El procedimiento completo se

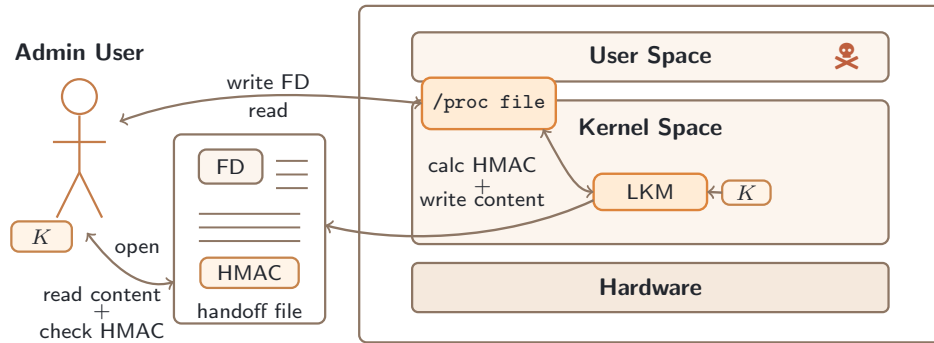
recoge en el Código A.16.

En conclusión, este prototipo amplía la interfaz creada en la etapa anterior permitiendo que el administrador indique dinámicamente la referencia al fichero donde se depositará la información generada por el *LKM*. A partir de este punto, los siguientes prototipos reutilizan este mecanismo como canal principal de salida para escribir contenido generado en *espacio del kernel*, añadiendo posteriormente autenticación criptográfica y el reporte final de procesos del sistema.

5.1.4. Mecanismo criptográfico de autenticación *HMAC-SHA256*

Una vez validado el mecanismo de *File Handoff*, el siguiente prototipo incorporó autenticación criptográfica sobre el contenido generado en *espacio del kernel* y escrito en el fichero referenciado en *espacio de usuario* mediante un *HMAC-SHA256*.

El objetivo de esta etapa fue que el fichero escrito por el *LKM* no contuviese únicamente datos, sino también una etiqueta de autenticación que permitiese comprobar posteriormente si dicho contenido había sido autenticado con la clave secreta esperada y si no había sido modificado tras su escritura.



Fuente: Elaboración propia.

Figura 5.4: Incorporación de un mecanismo de autenticación *HMAC-SHA256* al sistema de *File Handoff*.

Se empleó una etiqueta criptográfica *HMAC-SHA256* como mecanismo de autenticación de mensajes, basado en una clave simétrica secreta (K) y en la función resumen *SHA-256* cuyo tamaño de bloque es de 512 *bits*. Es importante destacar que este mecanismo no cifra el contenido, por lo que los datos escritos en el fichero siguen siendo legibles. Su finalidad es generar un valor de autenticación dependiente simultáneamente del contenido y de la clave secreta K , por lo que, para un único contenido, solo es posible calcular una única etiqueta *HMAC-SHA256* para una clave simétrica K dada. Así, cualquier modificación del contenido, o cualquier intento de generar una etiqueta sin conocer dicha clave, provocaría que la verificación posterior no coincidiese, asegurando de esta manera la integridad del contenido a transferir.

Para que este mecanismo tenga sentido, cabe destacar que la clave K debe ser secreta y compartida únicamente entre el *LKM* en *espacio del kernel* y la utilidad de verificación del contenido por parte del administrador del sistema en *espacio de usuario*. Como se explicó en el modelo de amenaza del Apartado 4.2, en el entorno en el que se envuelve el sistema de reporte seguro propuesto se asume que el atacante no tiene acceso a esta clave simétrica secreta (K) según la cadena de confianza descrita, por lo que no puede generar etiquetas válidas para contenidos falsificados ni modificar el contenido sin invalidar la etiqueta. En caso contrario, este mecanismo no aportaría integridad alguna al contenido a transferir.

En este proyecto se genera una clave simétrica K con tamaño óptimo de 64 *bytes*, por

lo que encaja directamente con el tamaño de bloque usado por *SHA-256* sin necesitar reducción si fuese de mayor tamaño, teniendo que calcular $SHA-256(K)$ dando una salida de 32 *bytes* y teniendo que rellenar con ceros (0x00) por la derecha hasta llegar al tamaño. Ni utilizar relleno por otro lado si fuese de menor tamaño, teniendo que cumplimentar con ceros por la derecha.

Una vez ya calculada la etiqueta *HMAC-SHA256* binaria mediante el resumen por bloques de 64 *bytes* del contenido y la clave simétrica de tamaño óptimo (K), el prototipo la codifica en *Base64* para escribirla posteriormente en el fichero de transferencia. Esta codificación agrupa los datos binarios de entrada en bloques de 3 *bytes* (24 *bits*), donde a su vez se dividen en 4 grupos de 6 *bits*, pudiendo así representar dicha información codificando cada bloque de 6 *bits* en un carácter de los 64 del alfabeto *ASCII* limitado (A-Z, a-z, 0-9, +, /). Por ello, los 32 *bytes* de la etiqueta *HMAC-SHA256* se transforman en 44 caracteres en *Base64*, incluyendo el carácter de relleno final (=) cuando se aplica *padding* a los grupos incompletos, como en este caso para el tamaño de 32 *bytes*, ya que no forma un número entero de grupos de 3 *bytes*. Esta conversión no modifica la seguridad criptográfica del mecanismo, sino únicamente su representación para poder almacenarlo y procesarlo como texto.

Para integrar este mecanismo en el *LKM* fue necesario incorporar soporte de la *Crypto API* del *kernel*, la codificación en *Base64* y la clave simétrica embebida. La clave no fue escrita manualmente en el código, sino que se generó mediante un script auxiliar que crea un valor aleatorio de 64 *bytes* y lo transforma en una cabecera C importable al módulo (.h). De esta forma, tanto el *kernel* como el administrador pueden operar con la misma clave secreta durante la prueba. Las cabeceras añadidas y el proceso de generación de la clave se recogen en los Códigos A.17 y A.18 del Apéndice A.

El cálculo de la etiqueta *HMAC* se delega en la *Crypto API* del *kernel* mediante una instancia síncrona de *hash* (*shash*) asociada al algoritmo *hmac(sha256)*. Conceptualmente, el módulo solicita dicha instancia para una *hash*, le asocia la clave simétrica secreta K previamente cargada en el *kernel* dentro de la cadena de confianza considerada, reserva memoria para el resultado y ejecuta el cálculo sobre el bloque de datos recibido. La llamada de alto nivel abstrae las operaciones internas de *HMAC*. La implementación concreta del cálculo se muestra en el Código A.19.

Una vez calculado el *HMAC* binario de 32 *bytes*, el prototipo lo convierte a *Base64*

siguiendo el proceso anteriormente explicado. Se pasa de una etiqueta binaria de 32 *bytes* a una cadena de 44 caracteres *Base64*: diez grupos completos generan 40 caracteres y los dos bytes restantes generan cuatro caracteres adicionales con relleno. Al tratarse de un fichero generado para ser revisado y validado desde *espacio de usuario*, codificar la etiqueta a *Base64* evita escribir bytes binarios directamente y facilita su extracción posterior por parte del programa de comprobación. La función auxiliar encargada de esta conversión se recoge en el Código A.20.

El formato final escrito en el fichero de *handoff* se organiza mediante separadores textuales. En primer lugar se escribe un marcador que identifica el inicio del contenido generado por el *kernel*, después se añade el contenido protegido, posteriormente se escribe un separador específico para la etiqueta, y finalmente, se incorpora el *HMAC* codificado en *Base64*. Esta estructura hace que el fichero sea determinista y permite que la utilidad de verificación pueda separar sin ambigüedad la información útil transferida de su autenticador. La lógica de escritura conjunta del contenido y su etiqueta *HMAC* se detalla en el Código A.21.

En este prototipo, la función principal de integración genera todavía un contenido fijo de prueba, calcula su *HMAC-SHA256*, lo codifica en *Base64* y escribe el conjunto completo en el fichero real recibido a través del mecanismo de *File Handoff*. Por tanto, la lógica de recepción del *FD*, resolución mediante el objeto `struct file` y validación del fichero se mantiene igual respecto al prototipo anterior, pero la escritura en dicho fichero desde el *kernel* se sustituye por una escritura autenticada. Esta integración se muestra en A.22 y A.23.

Para cerrar el ciclo de validación fue necesario implementar también una utilidad adicional en *espacio de usuario*, con la cual el administrador pudiese validar (en otra máquina) la integridad y autenticidad del contenido transferido. Esta herramienta abre el fichero de transferencia, localiza los separadores, extrae por un lado el contenido o *payload* y por otro la etiqueta *HMAC* en *Base64*, recalcula localmente dicha etiqueta de autenticación con *OpenSSL* usando la misma clave *K* con respecto al contenido extraído, vuelve a codificar el resultado y compara ambas etiquetas. La comparación se realiza con una primitiva criptográfica específica, evitando una comparación ordinaria de cadenas y así reducir la exposición a ataques de canales laterales basados en el tiempo (*time side-channel attacks*). Las partes principales de esta utilidad se recogen en A.24, A.25, A.26 y A.27.

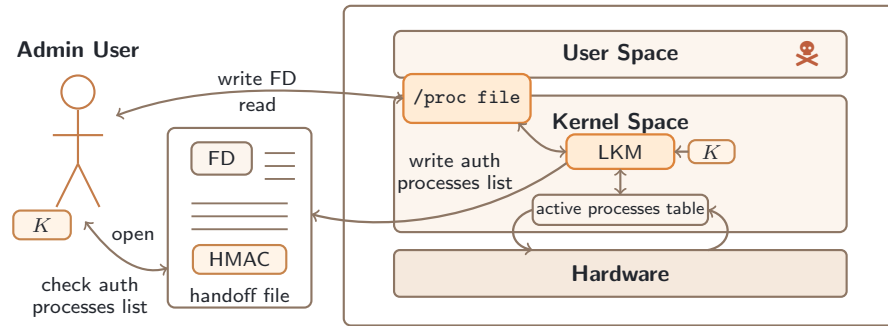
Finalmente, la validación práctica confirmó que el módulo podía generar un contenido autenticado en *espacio del kernel*, escribirlo en el fichero recibido desde *espacio de usuario* y verificar posteriormente que la etiqueta recibida coincidía con la recalculada por la utilidad de usuario. El procedimiento completo de compilación, ejecución y verificación se recoge en el Código A.28.

Esta implementación se utilizará como mecanismo de autenticidad e integridad en la solución final del proyecto, proporcionando una herramienta robusta a prueba de ataques de *rootkits* de *espacio de usuario* siguiendo el *Modelo de amenaza* definido. A partir de este punto, el siguiente y último paso consistió en sustituir el contenido fijo de prueba por información crítica real del sistema, generada dinámicamente en *espacio del kernel* y protegida mediante el mismo esquema *HMAC-SHA256*.

5.1.5. Sistema de reporte seguro de los procesos en ejecución del sistema

Una vez diseñado el canal de *File Handoff* y el mecanismo de autenticación mediante *HMAC-SHA256*, el prototipo final sustituyó el contenido fijo de prueba por información crítica real del sistema generada dinámicamente en *espacio del kernel*.

El objetivo de esta etapa fue construir un reporte de todos los procesos activos del sistema y transferirlo de forma autenticada mediante los mecanismos diseñados en los prototipos anteriores.



Fuente: Elaboración propia.

Figura 5.5: Generación y transferencia autenticada de un reporte de los procesos activos del sistema desde *espacio del kernel*.

Este prototipo unifica todas las capacidades desarrolladas en las etapas anteriores, añadiendo funcionalidades para la generación de reportes de procesos activos como contenido a transferir de manera autenticada. La comunicación entre el administrador y el módulo se sigue realizando mediante el fichero virtual `/proc/fdev`. El administrador continúa entregando un *FD* de un fichero real, el *LKM* resuelve dicho descriptor como un objeto `struct file`, y la escritura final mantiene la estructura formada por contenido, separador textual y etiqueta *HMAC-SHA256* codificada en *Base64*. La diferencia principal respecto al prototipo anterior es que el contenido protegido deja de ser una cadena fija y pasa a ser una tabla generada a partir de las estructuras internas del *kernel* con la información principal de todos los procesos activos en el sistema.

Para construir este reporte fue necesario ampliar las dependencias del módulo con cabeceras relacionadas con credenciales, procesos, identificadores, espacios de nombres y sincronización. Estas cabeceras permiten acceder a las credenciales de cada tarea en ejecución, recorrer la lista global de procesos, obtener identificadores globales y locales respecto a los espacios de nombres activos y proteger el recorrido mediante el mecanismo *RCU*, evitando condiciones de carrera sobre las estructuras compartidas del *kernel*. Además, se definieron constantes para acotar el tamaño máximo del reporte y así simplificar la implementación, y por otra parte, se reservan anchuras fijas para cada campo y se construye una cabecera bien espaciada con los metadatos que se desean mostrar de cada proceso para una mejor legibilidad. Las cabeceras, constantes y estructura inicial de la tabla se recogen en el Código A.29 del Apéndice A.

El formato elegido para el reporte está inspirado en la salida de herramientas como `ps`, pero con campos adaptados al objetivo del proyecto. Cada fila representa un proceso en ejecución e incluye el identificador de usuario *UID* global visible desde el *kernel* y el local relativo al espacio de nombres del proceso, el identificador del *namespace* de usuario, el identificador de proceso *PID* global y relativo a su *namespace* con el identificador del *namespace* de procesos, el identificador de grupo *GID* y el nombre del comando asociado. Esta combinación permite comparar la visión global del sistema vista desde el propio *kernel* con la visión local que pueden tener procesos ejecutados dentro de contenedores o espacios de nombres diferenciados.

Antes de recorrer los procesos, el módulo incorpora funciones auxiliares para construir el contenido de forma ordenada y controlada, pudiendo además mejorar su posterior interpretabilidad. Por un lado, una de ellas rellena cada campo con espacios hasta alcanzar una anchura fija, de manera que las columnas del reporte queden totalmente alineadas. Y por otro lado, hay otra función auxiliar que copia cada fragmento en el *buffer* global con un padding adicional si lo requiere, pero únicamente si existe espacio disponible, evitando desbordamientos (*overflows*) al ir concatenando filas. Estas operaciones de formateo y copia de datos segura se detallan en el Código A.30.

La función central de este prototipo final es la encargada de generar el contenido completo del reporte. En primer lugar, se reserva memoria en *espacio del kernel* para un *buffer* de tamaño máximo fijo, se añade la cabecera de la tabla y se entra en una sección de lectura protegida mediante *RCU* delimitada por `rcu_read_lock` y `rcu_read_unlock`. Dentro de esta sección, se recorre la lista global de tareas del sistema mediante `for_each_process`, comprobando antes de cada inserción que el *buffer* dispone todavía de espacio suficiente para la nueva fila más los separadores y la etiqueta *HMAC-SHA256*. Para cada proceso, delega en una función auxiliar la extracción y serialización de sus metadatos. La construcción general del listado en memoria se muestra en A.31.

El uso de *RCU* resulta bastante relevante porque la lista de procesos y las estructuras `task_struct` pueden ser consultadas mientras el sistema sigue ejecutándose y modificando su estado. La protección de lectura no detiene el sistema ni bloquea globalmente la creación o finalización de procesos, pero permite recorrer las referencias

de forma coherente durante la generación del reporte.

La serialización de cada fila se encapsula en una función específica que recibe la tarea actual y añade sus campos al *buffer* global del reporte. Esta función no calcula directamente todos los valores, sino que coordina llamadas auxiliares especializadas para obtener cada metadato, formatearlo como una cadena textual portable y añadirlo de forma secuencial. Si alguna extracción falla o si no existe espacio suficiente en el *buffer*, se aborta la generación para evitar producir un reporte inconsistente. La lógica de serialización de una fila completa se referencia en el Código A.32.

Los identificadores de usuario (*UIDs*) se tratan diferenciando entre el valor global y el valor local respecto a un espacio de nombres específico. El *UID* global se obtiene tomando como referencia el espacio de nombres inicial del sistema, por lo que representa la visión del *kernel*. El *UID* local, en cambio, se obtiene a partir del *namespace* asociado a las credenciales del proceso, permitiendo reflejar el valor del identificador que tendría dicho usuario dentro de su propio contexto. Además, se incluye el identificador interno del *namespace* de usuario, lo que permite distinguir procesos que pueden compartir valores locales pero pertenecer a espacios de nombres distintos. Estas funciones se muestran en A.33.

De forma análoga, los identificadores de proceso (*PIDs*) se separan en *PID* global y *PID* local respecto al *namespace* asociado a cada tarea. El *PID* global corresponde al identificador visible desde el *kernel*, mientras que el *PID* local se obtiene respecto al espacio de nombres en el que trabaja dicha tarea. Esta distinción es especialmente útil en entornos con contenedores, donde un proceso puede aparecer con *PID* local 1 dentro del contenedor, aunque tenga otro *PID* distinto desde la perspectiva global del sistema. La obtención de estos campos se recoge en el Código A.34.

El reporte se completa con el identificador de grupo (*GID*) y el comando asociado a cada proceso. Para el *GID* se parte de las credenciales del proceso y se obtiene su valor global respecto al espacio de nombres inicial del sistema. Y como último tipo de metadato, se obtiene el nombre del comando asociado a cada proceso, distinguiendo entre procesos de usuario con ejecutable asociado, hilos del *kernel* (recogidos entre corchetes []) y casos en los que solo está disponible el nombre almacenado en `task->comm`. Cuando existe un ejecutable asociado, se intenta resolver su ruta mediante las estructuras internas del fichero ejecutable, pero si no es posible, se utiliza el

nombre corto del proceso registrado en la propia tarea (`task->comm`). Estas funciones se referencian en A.35.

Una vez construido el contenido crítico real del reporte, el flujo vuelve a coincidir con el prototipo anterior. El módulo calcula el *HMAC-SHA256* sobre la tabla completa de procesos generada, codifica la etiqueta en *Base64* y escribe en el fichero de transferencia el marcador de inicio, la tabla de procesos como información crítica a transferir, el separador de la etiqueta y el *HMAC* final. De esta forma, la autenticación no protege únicamente una cadena de prueba, sino el listado completo de procesos en ejecución del sistema generado con sus metadatos más relevantes desde *espacio del kernel*. La escritura conjunta del contenido y su autenticador se apoya en la lógica descrita anteriormente en el Código A.21.

El resultado escrito en el fichero de transferencia mantiene una estructura textual clara, donde el bloque de procesos y la etiqueta de autenticación quedan delimitados por marcadores explícitos. De esta forma, el contenido final del fichero tendría una estructura como la mostrada a continuación, si dicho fichero estuviese vacío, ya que se utiliza el modo *append* de escritura:

```
--KERNEL-PS--
UID_KERNEL UID_LOCAL UID_NS   PID_KERNEL PID_LOCAL PID_NS   GID   COMMAND
0           0         4026531837 1           1         4026531836 0     /usr/lib/systemd/systemd
0           0         4026531837 2           2         4026531836 0     [kthreadd]
0           0         4026531837 3           3         4026531836 0     [pool_workqueue_]
...
113         113        4026531837 1440        1440      4026531836 119    /usr/sbin/chronyd
113         113        4026531837 1448        1448      4026531836 119    /usr/sbin/chronyd
0           0         4026531837 1484        1484      4026531836 0     /usr/sbin/gdm3
0           0         4026531837 1489        1489      4026531836 1000   /usr/libexec/gdm-session-worker
1000        1000      4026531837 1496        1496      4026531836 1000   /usr/lib/systemd/systemd
1000        1000      4026531837 1497        1497      4026531836 1000   /usr/lib/systemd/systemd-exe
...
0           0         4026531837 8211        8211      4026531836 0     /usr/bin/dockerd
0           0         4026531837 8212        8212      4026531836 0     [kworker/1:1]
0           0         4026531837 8239        8239      4026531836 0     [kworker/1:2H]
0           0         4026531837 8500        8500      4026531836 0     /usr/bin/containerd-shim-runc-v2
100000      0         4026532674 8520        1         4026532678 100000 /usr/bin/sleep
0           0         4026531837 8532        8532      4026531836 0     [kworker/2:0]
0           0         4026531837 8596        8596      4026531836 0     /usr/bin/containerd-shim-runc-v2
100000      0         4026532754 8617        1         4026532758 100000 /usr/bin/sleep
0           0         4026531837 8667        8667      4026531836 0     [kworker/2:2H]
0           0         4026531837 8678        8678      4026531836 0     /usr/bin/containerd-shim-runc-v2
100000      0         4026532834 8698        1         4026532838 100000 /usr/bin/sleep
0           0         4026531837 8741        8741      4026531836 0     [kworker/u20:2]
0           0         4026531837 8935        8935      4026531836 0     [kworker/0:0]
0           0         4026531837 8947        8947      4026531836 0     [kworker/u16:3]
0           0         4026531837 8989        8989      4026531836 0     [kworker/3:2]
0           0         4026531837 9033        9033      4026531836 0     /home/noel/Project/mk_fhandoff

--HMAC--
ckvLUckcgmt6RFfdGzuvgnAEgt9rLPJ5q9FYJoFKAW0=
```

Código 5.1: Ejemplo de la estructura final del fichero de transferencia autenticado.

La validación de este prototipo final se realizó compilando y cargando el módulo, generando de nuevo la clave embebida, habilitando el fichero de salida mediante la utilidad de *File Handoff* y verificando el resultado con el programa de comprobación de *HMAC* de *espacio de usuario*. Además, se ejecutaron varios contenedores *Docker* para forzar la presencia de procesos en distintos espacios de nombres y comprobar que el reporte reflejaba diferencias en campos como `UID_LOCAL`, `PID_LOCAL`, `UID_NS` y `PID_NS`, y también, respecto a sus identificadores globales. El procedimiento completo de validación, incluyendo la generación del fichero, la presencia de procesos de contenedores y la verificación criptográfica, se recoge en el Código A.36.

Finalmente, a partir de los resultados obtenidos, se pudo concluir que el *LKM* era capaz de generar desde *espacio del kernel* una tabla global de procesos activos del sistema, incluir metadatos relevantes de usuarios, procesos, grupos y espacios de nombres, transferir dicha información a un fichero de transferencia indicado por el administrador y adjuntar una etiqueta *HMAC-SHA256* verificable posteriormente desde *espacio de usuario*. Con ello queda completado el prototipo final del sistema de reporte seguro, integrando en una única solución la comunicación entre *espacio de usuario* y *espacio del kernel*, la generación de información crítica desde una región privilegiada y la autenticación criptográfica del contenido transferido para asegurar su integridad frente a *rootkits* de *espacio de usuario*.

5.2. Problemas y Errores encontrados

Durante el desarrollo del proyecto no solo fue necesario implementar progresivamente los prototipos descritos, sino también profundizar en matices realmente complejos y corregir decisiones iniciales que, aunque podían funcionar en pruebas sencillas, no eran adecuadas para una solución ejecutada dentro del *kernel*.

La principal dificultad estuvo en que cada mecanismo debía respetar las reglas propias del núcleo del sistema: el acceso a ficheros no podía tratarse como una escritura convencional de *espacio de usuario*, el cálculo criptográfico debía apoyarse en la infraestructura interna del *kernel* y el recorrido de procesos debía realizarse teniendo

en cuenta problemas de concurrencia del sistema. Por ello, las decisiones finales no se limitaron a conseguir una salida correcta, sino que buscaron evitar accesos inseguros, procurar seguir una metodología compatible con las funcionalidades del *kernel* y mantener una estructura verificable desde *espacio de usuario* de forma sencilla y práctica.

A continuación, se describen los tres problemas más relevantes encontrados durante el desarrollo del proyecto y la solución adoptada en cada caso.

5.2.1. Escritura sobre el fichero de transferencia en modo *append*

Uno de los primeros problemas importantes apareció al diseñar el mecanismo de *File Handoff*. El objetivo era que el administrador abriese un fichero real en *espacio de usuario*, enviase su *FD* al *LKM* y que el módulo escribiese en dicho fichero desde *espacio del kernel*. Inicialmente, esta operación podía parecer equivalente a una escritura ordinaria sobre un fichero abierto, pero al analizar el flujo real de escritura del *kernel* se comprobó que el manejo del puntero de posición del fichero (`f_pos`) era un punto realmente delicado a tener en cuenta.

En una escritura normal, la posición del puntero de escritura del fichero determina dónde se insertan los nuevos bytes. Si el módulo escribiese directamente usando el puntero asociado al objeto `struct file`, tendría que asumir la responsabilidad de controlar correctamente dicha posición, especialmente si el fichero pudiera ser utilizado por más de una ruta de ejecución. Esto habría añadido complejidad adicional al prototipo, ya que sería necesario razonar sobre actualizaciones del puntero, implementación de mecanismos de bloqueo sobre el puntero, posibles escrituras intermedias y sobrescrituras accidentales del contenido previo.

Además, como el contenido final se escribe en varias partes (marcador inicial, contenido, separador y etiqueta *HMAC*), una gestión incorrecta de la posición podría producir un fichero mal formateado con una difícil verificación posterior.

La solución adoptada fue exigir que el fichero recibido estuviese abierto con capacidad de escritura y en modo *append*. De esta forma, la política del prototipo queda restringida

a añadir el reporte al final del fichero de transferencia, evitando depender de una posición arbitraria proporcionada por el estado previo del descriptor y de su correcta gestión y modificación.

Esta decisión se implementó en la validación de metadatos del fichero, comprobando tanto `FMODE_WRITE` como el *flag* `O_APPEND`, y verificando además los permisos efectivos mediante `file_permission` con `MAY_WRITE` y `MAY_APPEND` dentro del *LKM*. El código asociado a esta lógica de validación se encuentra en A.13.

Por otra parte, se implementó la función auxiliar `write_full` para mantener encapsulada la operación de escritura desde el *LKM*. Esta función realiza una única llamada a `kernel_write` y comprueba tanto el posible código de error devuelto por el *kernel* como que el número de *bytes* escritos coincida con el tamaño solicitado, asegurando de esta forma que se realiza una escritura completa.

Con esta solución, el módulo mantiene una semántica de escritura más controlada, solo acepta ficheros compatibles con la política definida de escritura en modo *append*, escribiendo de esta manera siempre al final del fichero. El diseño final de esta parte se recoge en el Código A.12.

5.2.2. Integración del mecanismo *HMAC-SHA256* entre *Kernel-Space* y *User-Space*

Otro punto especialmente delicado fue la incorporación del mecanismo criptográfico de autenticación en el *LKM*. Las primeras pruebas permitieron validar el concepto desde *espacio de usuario*, generando una clave simétrica y calculando un *HMAC-SHA256* mediante *OpenSSL*.

Sin embargo, trasladar esa idea al *módulo del kernel* no consistía en reutilizar directamente las mismas funcionalidades de usuario, ya que dentro del *kernel* no se dispone de las bibliotecas ni del mismo modelo de memoria. Fue necesario apoyarse en la *Crypto API* del *kernel*, concretamente en la interfaz de *hash* síncrono (*shash*) asociada al algoritmo `hmac(sha256)`.

El cálculo exigía solicitar primero la instancia al *hash* (*shash*) con `crypto_alloc_shash`, asociarle la clave mediante `crypto_shash_setkey`, obtener el tamaño real del resumen con `crypto_shash_digestsize` y reservar memoria para la etiqueta resultante mediante `kmalloc`. Después, se debía construir el descriptor temporal requerido por la *API* con `SHASH_DESC_ON_STACK`, ejecutar el resumen con `crypto_shash_digest` y controlar en cada paso los errores y la liberación de recursos, sabiendo que la salida de todo este proceso es binaria. Esto hizo que el mecanismo fuese mucho más complejo en comparación a la facilidad de cálculo mediante *OpenSSL* en *espacio de usuario*.

Otro obstáculo importante encontrado en esta parte del proyecto fue coordinar correctamente tres aspectos: la clave compartida, el cálculo del autenticador en *espacio del kernel* y la verificación posterior desde *espacio de usuario*.

Durante el desarrollo se pasó de pruebas iniciales con claves creadas manualmente a una clave embebida generada previamente mediante un script auxiliar de manera aleatoria. Este cambio permitió que el *LKM* y la utilidad de verificación del administrador en *espacio de usuario* trabajasen con la misma clave *K* de forma práctica. La generación de la cabecera con la clave embebida se recoge en el Código A.18.

Por otro lado, fue necesario tratar correctamente la representación del *HMAC*. El resultado de *HMAC-SHA256* es una etiqueta binaria de 32 *bytes*, no una cadena de texto directamente legible. Escribir esos bytes sin codificación complicaría la inspección manual del fichero y su verificación. Por ello, el módulo convierte la etiqueta binaria a *Base64* antes de escribirla en el fichero de transferencia, generando una representación textual estable que puede ser leída, separada y recalculada por la utilidad de usuario de forma sencilla y muy práctica. Esta decisión obligó a cuidar la estructura del fichero, separando explícitamente el contenido protegido de la etiqueta mediante marcadores textuales, para que la verificación se realizase exactamente sobre los mismos bytes que habían sido autenticados por el *kernel*.

Finalmente, la utilidad de comprobación en *espacio de usuario* tuvo que reproducir el mismo proceso de forma coherente: extraer el contenido entre separadores, recalcular el *HMAC-SHA256* con la misma clave, codificarlo en *Base64* y compararlo con el valor escrito por el módulo. Además, la comparación se realiza mediante una primitiva criptográfica específica, evitando una comparación ordinaria de cadenas y reduciendo

así posibles fugas de información por diferencias de tiempo en la comparación. Con ello se consiguió cerrar el ciclo completo de autenticación: generación dentro del *kernel*, transferencia textual estructurada y verificación externa del contenido. La implementación de este mecanismo se detalla en A.19, A.20, A.21 y A.27.

5.2.3. Recorrido seguro de la lista de procesos mediante *RCU*

El último problema relevante durante el desarrollo del proyecto apareció al sustituir el contenido fijo de prueba por información real del sistema. El prototipo final debía recorrer la lista global de tareas del *kernel* y extraer los metadatos más relevantes de cada proceso, como identificadores de usuario, de proceso, de grupo, espacios de nombres y el nombre del comando asociado.

Esta operación no puede plantearse como un recorrido estático sobre una estructura inmutable, ya que la lista de procesos cambia continuamente mientras el sistema está en ejecución: pueden crearse nuevas tareas, finalizar procesos o modificarse estructuras internas relacionadas con ellos.

Una primera aproximación sin una protección específica habría expuesto al módulo a condiciones de carrera durante el recorrido. Por ejemplo, mientras se consulta una entrada del listado, el proceso correspondiente podría finalizar o cambiar su estado, dejando referencias inconsistentes o potencialmente inválidas.

En código ejecutado dentro del *kernel*, al tratarse de una región privilegiada del sistema, este tipo de errores es especialmente crítico, ya que no se limita a producir una salida incorrecta, sino que puede provocar lecturas incoherentes, accesos a memoria no válidos o fallos del propio módulo. Por este motivo, recorrer estructuras internas como la lista de procesos exige respetar los mecanismos de sincronización previstos por el *kernel*.

La solución adoptada fue proteger el recorrido mediante una sección de lectura *RCU* (*Read-Copy-Update*), delimitada por `rcu_read_lock` y `rcu_read_unlock`. *RCU* permite que el módulo recorra referencias compartidas de forma segura sin bloquear globalmente la creación o finalización de procesos. Esto encaja con el objetivo del proyecto, ya que no se necesita congelar por completo el sistema, sino obtener una

instantánea práctica y coherente de los procesos activos desde una región privilegiada. La macro `for_each_process` se ejecuta dentro de esta sección protegida, y cada tarea se serializa inmediatamente en el *buffer* del reporte mediante funciones auxiliares.

Esta solución no convierte el reporte en una fotografía transaccional perfecta del sistema, ya que el sistema operativo continúa ejecutándose y su estado puede cambiar durante la generación del listado. No obstante, sí evita recorrer la lista de procesos sin protección y reduce el riesgo de acceder a referencias inestables mientras se construye el reporte.

Además, en los puntos donde se consultan credenciales de las tareas, el módulo utiliza referencias controladas mediante las primitivas correspondientes, como `get_task_cred` y `put_cred`, para no depender de estructuras cuya vida útil no esté protegida. La construcción del listado bajo protección *RCU* se muestra en el Código A.31.

La solución final desarrollada no elimina por completo el riesgo asociado a un sistema comprometido, pero sí proporciona un mecanismo práctico para elevar la confianza en una fuente concreta de información crítica, reduciendo la dependencia exclusiva de herramientas de *espacio de usuario* cuando estas pueden haber sido alteradas por un *rootkit*. A continuación, se profundizará en todo lo relacionado con las conclusiones obtenidas tras la finalización del proyecto desde distintos puntos de vista.

Capítulo 6

Conclusiones

«You can't trust code that you did not totally create yourself.»

Ken Thompson (*Reflections on Trusting Trust*, 1984) [31].

Tras haber presentado la problemática abordada, el marco tecnológico necesario, el modelo de amenaza, los objetivos definidos y el desarrollo progresivo de la solución, este último capítulo recoge una valoración final del proyecto realizado.

El principal propósito es interpretar el resultado obtenido desde una perspectiva más amplia, considerando su impacto, las competencias aplicadas durante el trabajo, las limitaciones que permanecen abiertas y las posibles líneas de evolución.

El prototipo desarrollado demuestra que es posible construir un mecanismo de reporte seguro para sistemas *Linux* capaz de generar desde *espacio del kernel* información crítica, concretamente un listado de los procesos activos del sistema, transferirla de manera accesible para el administrador mediante un fichero de transferencia y autenticarla mediante una etiqueta *HMAC-SHA256* verificable posteriormente en *espacio de usuario*. Esta solución no elimina por completo los riesgos asociados a un sistema comprometido, pero sí aporta una vía práctica para reducir la dependencia exclusiva de herramientas de *espacio de usuario* cuando estas pueden estar manipuladas por un *rootkit* que opera en esta región. Por ello, las conclusiones del proyecto deben entenderse tanto desde el resultado técnico alcanzado como desde las implicaciones de seguridad, responsabilidad y sostenibilidad asociadas a este tipo de mecanismos.

6.1. Cumplimiento de objetivos

Una vez descrito el diseño progresivo de la solución adoptada, sus validaciones y los principales problemas encontrados durante el desarrollo, resulta necesario contrastar el resultado final con los objetivos definidos en el Capítulo 4. Este cierre permite comprobar si el prototipo implementado responde realmente a la problemática inicial y si proporciona al administrador una fuente de información crítica fiable cuando el *espacio de usuario* puede encontrarse comprometido por un *rootkit*.

El resultado final del proyecto consiste en un *módulo del kernel* capaz de generar desde *espacio del kernel* un listado estructurado de los procesos presentes en el sistema, incluyendo metadatos relevantes de usuarios, procesos, grupos, espacios de nombres y nombre del comando asociado. Dicho contenido se transfiere a un fichero de transferencia indicado por el administrador mediante un mecanismo de *File Handoff*, y se acompaña de una etiqueta *HMAC-SHA256* calculada con una clave simétrica compartida, permitiendo su verificación posterior desde *espacio de usuario* para comprobar la autenticidad del contenido.

Por tanto, el prototipo final cumple el objetivo general planteado, ya que desplaza la generación de la información crítica a una región de mayor privilegio y añade un mecanismo criptográfico para detectar modificaciones posteriores del reporte.

6.1.1. Cumplimiento de los objetivos funcionales

Respecto a los objetivos funcionales definidos en el Apartado 4.3.1, se puede considerar que la solución desarrollada cumple las capacidades principales previstas inicialmente.

- **Obtención de información crítica desde *espacio del kernel*.** Este objetivo se cumple mediante el *LKM* final, que recorre la lista de tareas del *kernel* y construye el reporte directamente desde una región privilegiada, sin depender de herramientas de monitorización de *espacio de usuario*. Esta decisión responde a la premisa principal del modelo de amenaza, donde dichas herramientas de usuario podrían estar manipuladas.

- **Construcción de una vista global y estructurada de procesos.** El reporte generado incluye los campos definidos en los objetivos: `UID_KERNEL`, `UID_LOCAL`, `UID_NS`, `PID_KERNEL`, `PID_LOCAL`, `PID_NS`, `GID` y `COMMAND`. Con ello se obtiene una visión más completa que una simple lista de procesos, ya que también se reflejan diferencias entre identificadores globales y locales, especialmente relevantes en entornos con *namespaces*.
- **Transferencia controlada hacia un canal accesible desde *espacio de usuario*.** Este objetivo se cumple mediante la entrada virtual `/proc/fdev` y la utilidad de usuario encargada de crear el fichero de transferencia y enviar su *File Descriptor* al *kernel*. A partir de esa referencia, el módulo consigue resolverla y así puede escribir el reporte directamente en el fichero seleccionado por el administrador, manteniendo controladas las validaciones sobre descriptor, permisos y modo de apertura.
- **Autenticación criptográfica del contenido generado.** El contenido del reporte se autentica mediante *HMAC-SHA256*, calculado dentro del *kernel* con la *Crypto API* y una clave simétrica *K* embebida previamente. De este modo, la etiqueta generada queda asociada al contenido exacto producido por el módulo.
- **Verificación posterior desde *espacio de usuario*.** Este objetivo se cumple mediante la utilidad de comprobación desarrollada en C, que extrae el contenido protegido, recalcula el *HMAC-SHA256* con la misma clave, codifica el resultado en *Base64* y lo compara con la etiqueta escrita por el *kernel*. La validación confirma que el administrador puede comprobar posteriormente la integridad y autenticidad del reporte recibido desde otra máquina no comprometida.
- **Facilitar la detección de ocultación o manipulación.** El sistema no implementa un motor automático de detección de *rootkits*, pero sí cumple con el objetivo de proporcionar una fuente autenticada de información generada desde el *kernel* que puede compararse con la salida de herramientas de *espacio de usuario*. Por tanto, facilita la detección de inconsistencias cuando un *rootkit* manipule la visión ofrecida por comandos convencionales como `ps`, dejando la decisión final de análisis en manos del administrador.
- **Validación del prototipo.** El prototipo se validó en un entorno *Linux* real sobre la versión de *kernel* utilizada durante el proyecto, compilando y cargando el módulo, generando la clave embebida, creando el fichero de transferencia, ejecutando procesos en contenedores *Docker* y verificando criptográficamente el

reporte resultante. Las pruebas muestran que el sistema genera el listado esperado y que la etiqueta *HMAC* puede comprobarse correctamente desde *espacio de usuario*.

6.1.2. Cumplimiento de los objetivos no funcionales

Los objetivos no funcionales del Apartado 4.3.2 también quedan cubiertos por las decisiones de diseño adoptadas durante el desarrollo.

- **Integridad y autenticidad verificables.** Se cumple mediante el uso de *HMAC-SHA256* y la verificación posterior con la misma clave simétrica, bajo la premisa de que tanto el *kernel* como el administrador tengan la misma clave simétrica *K* y que nadie más tenga acceso a ella. La codificación de la etiqueta en *Base64* facilita además su almacenamiento textual y su comprobación en el fichero de transferencia.
- **Robustez en la comunicación entre espacios.** El módulo valida la entrada recibida desde *espacio de usuario*, limita el tamaño de escritura sobre la entrada */proc*, transforma el descriptor recibido de forma controlada mediante *fget* y comprueba que el fichero de destino sea escribible y esté abierto en modo *append*. Además, el uso de funciones auxiliares como *write_full* y *safe_chunk* centraliza la comprobación de errores durante la escritura y reduce riesgos de desbordamientos del *buffer* de reporte.
- **Seguridad frente al modelo de amenaza definido.** La solución diseñada minimiza la confianza depositada en herramientas de usuario, ya que el contenido sensible se genera dentro del *kernel*. No obstante, esta garantía se mantiene dentro de los supuestos del modelo de amenaza: el *kernel*, el *hardware*, el *firmware* y la clave compartida deben permanecer íntegros. Por ello, el sistema no pretende proteger frente a *rootkits* de *espacio del kernel* ni frente a escenarios en los que la clave simétrica haya sido expuesta.
- **Modularidad y bajo impacto en el *kernel*.** El mecanismo se implementa como *LKM*, por lo que puede cargarse y descargarse sin modificar permanentemente el código fuente del núcleo del sistema operativo. Esta decisión facilita el despliegue controlado del prototipo y encaja con el carácter

experimental del proyecto, aunque no convierte todavía el mecanismo en una herramienta de producción completa con gestión avanzada de claves, integración con alertas o endurecimiento frente a cargas concurrentes más complejas.

- **Compatibilidad con mecanismos estándar de *Linux*.** La implementación se apoya en interfaces propias del ecosistema *Linux*, como `/proc`, `proc_create`, `kernel_write`, `fget`, *RCU*, la *Crypto API* del *kernel* y herramientas convencionales de compilación en C.
- **Reproducibilidad del prototipo.** El proyecto proporciona el código fuente del *LKM*, las utilidades de usuario, el script de generación de la clave embebida y las secuencias de validación recogidas en el anexo y en el repositorio de *GitHub*: <https://github.com/nowelito28/TFG>, lo que permite repetir y utilizar bajo licencia el flujo de compilación, carga, ejecución, verificación y descarga del módulo diseñado, siempre que se utilicen entornos *Linux* compatibles.
- **Portabilidad gestionada.** Aunque la validación se realizó sobre una Raspberry Pi 4 con arquitectura *ARM*, el diseño no queda ligado conceptualmente a dicha placa concreta. La solución se apoya en mecanismos propios de *Linux* y en herramientas estándar de compilación, por lo que podría trasladarse a otros entornos *Linux* compatibles realizando las adaptaciones necesarias al *kernel*, las cabeceras disponibles y la arquitectura objetivo.
- **Formato de salida legible y ordenado.** El fichero de transferencia contiene separadores explícitos, una tabla alineada de procesos y una etiqueta *HMAC* final en *Base64*. Esta estructura facilita la inspección manual, la extracción automática del contenido protegido y la comparación con otras fuentes de información del sistema.

Por todo ello, puede concluirse que el proyecto realizado alcanza los objetivos fijados y materializa una solución funcional, verificable y coherente con el problema planteado. No obstante, el resultado obtenido no debe entenderse como una solución definitiva o completa, sino como un prototipo que demuestra la viabilidad de la aproximación propuesta y que abre la puerta a futuras mejoras, ampliaciones y validaciones en escenarios más complejos o realistas.

6.2. Impacto social, económico, temporal, medioambiental y ético

El impacto social principal causado por el proyecto se sitúa en el ámbito de la ciberseguridad y la confianza en la información crítica de un sistema. En escenarios donde un atacante ha conseguido comprometer el *espacio de usuario*, las herramientas habituales de administración pueden dejar de ofrecer una visión fiable del estado real de la máquina. El mecanismo desarrollado contribuye a mitigar este problema proporcionando al administrador una fuente alternativa y autenticada de información sobre los procesos activos, generada desde una región de mayor privilegio y verificable mediante una clave simétrica compartida.

Desde esta perspectiva, el proyecto puede tener un impacto positivo en tareas de análisis, respuesta ante incidentes y auditoría técnica. Disponer de un reporte autenticado permite contrastar la información ofrecida por herramientas convencionales como **ps** con una visión detallada obtenida desde el *kernel*, facilitando la identificación de indicios de ocultación o manipulación. Este tipo de mecanismos resulta especialmente relevante en sistemas donde la continuidad del servicio, la trazabilidad de la actividad y la confianza en los datos sensibles de monitorización son aspectos críticos.

No obstante, debe tenerse en cuenta que el mecanismo propuesto está orientado a un perfil técnico. Su utilización requiere comprender conceptos de administración de sistemas *Linux*, programación en lenguaje **C**, funcionamiento básico del *kernel* y verificación criptográfica mediante *HMAC*, además de conocimientos generales en ciberseguridad. Por tanto, no se plantea como una herramienta destinada a usuarios finales sin conocimientos técnicos, sino como un apoyo para administradores, analistas o perfiles con formación en sistemas, programación y ciberseguridad.

Cabe destacar, por otro lado, que en entornos sujetos a políticas internas de seguridad o requisitos legales de trazabilidad y protección de la información, disponer de evidencias técnicas verificables puede facilitar la justificación de decisiones durante una auditoría o investigación de seguridad.

En cuanto al impacto económico, el valor del proyecto se relaciona principalmente con el coste potencial asociado a ataques con capacidades de ocultación y persistencia. Un

compromiso de este tipo puede provocar interrupciones del servicio, robo o filtración de información, costes derivados del análisis forense y la recuperación del sistema, daño reputacional y posibles incumplimientos normativos. En este contexto, disponer de un mecanismo capaz de aportar una fuente autenticada de información crítica permite detectar antes indicios de presencia o persistencia maliciosa, reduciendo el tiempo necesario para diagnosticar el incidente y limitando, en consecuencia, parte del impacto económico derivado de una respuesta tardía.

Desde el punto de vista temporal, el prototipo puede reducir el tiempo necesario para obtener una primera evidencia técnica fiable sobre los procesos activos de un sistema comprometido, ya que genera un reporte autenticado bajo demanda sin depender únicamente de herramientas de *espacio de usuario*. No obstante, su configuración, carga, verificación y análisis posterior siguen requiriendo intervención de un perfil técnico, por lo que el ahorro temporal se produce principalmente en escenarios de diagnóstico o respuesta ante incidentes, no en un uso automatizado por usuarios no especializados.

El impacto medioambiental directo del proyecto es reducido, ya que el desarrollo se ha limitado a un prototipo software ejecutado en un entorno local (*on-premise*) de pruebas. La validación se realizó sobre un sistema *Linux* real, utilizando únicamente los recursos necesarios para compilar, cargar y descargar el módulo, generar procesos de prueba y verificar el reporte resultante. Además, el diseño del prototipo evita una monitorización continua o intensiva del sistema. El reporte se genera bajo demanda cuando el administrador solicita la transferencia mediante el mecanismo de *File Handoff*, por lo que el sistema no necesita estar activo continuamente para funcionar, consumiendo así poca energía. Esto cambiaría, por ejemplo, si se modificase para monitorizar el sistema de forma continua o si se desplecase en la nube, ya que aumentaría el consumo energético del sistema y la necesidad de recursos operativos, como servidores o infraestructura de soporte.

Por último, desde el punto de vista de la responsabilidad ética y profesional, el proyecto exige tratar con especial cuidado el equilibrio entre seguridad defensiva y su potencial uso indebido. El desarrollo de mecanismos próximos al *kernel* implica operar en una zona crítica del sistema, donde errores de implementación pueden afectar a la estabilidad de la máquina y donde una mala gestión de permisos o claves puede reducir la protección esperada, además de poder aumentar el riesgo de vulnerabilidades. Por

ello, el diseño se ha planteado bajo un modelo de amenaza explícito, con validaciones sobre la entrada recibida desde *espacio de usuario*, comprobaciones sobre el fichero de destino, uso de mecanismos estándar del *kernel* y una delimitación clara de lo que el prototipo protege y de lo que queda fuera de su alcance.

6.3. Competencias adquiridas y utilizadas

La realización de este *Trabajo de Fin de Grado* ha permitido integrar distintas competencias propias del *Grado en Ingeniería en Tecnologías de la Telecomunicación*, especialmente aquellas relacionadas con sistemas operativos, programación de bajo nivel, seguridad informática, análisis técnico y documentación de soluciones de ingeniería. Aunque estas competencias ya fueron presentadas en el Capítulo 4 como parte del planteamiento del trabajo, en este punto se recoge una valoración final de cómo han sido aplicadas y reforzadas durante el desarrollo del proyecto.

De forma concreta, las competencias más relevantes aplicadas y adquiridas durante el desarrollo del proyecto son las siguientes:

- **Análisis de sistemas *Linux* y fundamentos de sistemas operativos.** El proyecto ha requerido comprender la separación entre *espacio de usuario* y *espacio del kernel*, el papel de los procesos, los identificadores de usuario, proceso y grupo, los *namespaces*, el sistema de ficheros virtual */proc* y los mecanismos de carga y descarga de módulos en una región privilegiada del sistema.
- **Programación de sistemas en lenguaje C.** El desarrollo del *LKM* exigió trabajar con estructuras y funciones propias del *kernel*, gestionar memoria con especial cuidado, validar datos procedentes de *espacio de usuario*, tratar descriptores de fichero mediante mecanismos internos del núcleo y escribir de forma controlada sobre un fichero seleccionado por el administrador.
- **Desarrollo en *espacio del kernel*.** Se adquirieron competencias específicas en programación de *módulos del kernel*, creación de entradas en */proc*, comunicación entre *espacio de usuario* y *espacio del kernel*, escritura desde el *kernel* hacia ficheros de usuario y uso tanto de mecanismos de sincronización y lectura segura (*RCU*) como de *APIs* específicas del código fuente del *kernel*, como *Crypto API*

para el manejo y cálculo de *HMAC*. Cabe destacar que para el correcto uso de estas funcionalidades, hubo que realizar previamente un análisis técnico del código fuente del *kernel* para determinar qué mecanismos debían utilizarse y cómo aplicarlos en cada caso.

- **Ciberseguridad y análisis de amenazas.** El trabajo permitió aplicar principalmente conocimientos sobre *rootkits* de *espacio de usuario*, manipulación de herramientas de monitorización, definición de un modelo de amenaza y diseño de una solución defensiva ajustada a unos supuestos de seguridad concretos.
- **Criptografía aplicada e integridad de información.** La solución incorpora autenticación criptográfica mediante *HMAC-SHA256*, calculada en el *kernel* y verificada después desde *espacio de usuario*. Esto permitió reforzar la comprensión práctica de conceptos como integridad, autenticidad, clave simétrica, codificación en *Base64* y límites de una protección basada en clave simétrica compartida.
- **Diseño incremental, validación y resolución de errores.** La construcción por prototipos permitió validar primero conceptos básicos, como la carga de un módulo o la comunicación mediante un fichero virtual en */proc*, antes de incorporar mecanismos más delicados como el traspaso de *File Descriptors*, el cálculo de *HMAC* y la generación del reporte de procesos en ejecución.
- **Comunicación técnica y documentación.** La memoria recoge el contexto del problema, la justificación del modelo de amenaza, el diseño progresivo de la solución, los problemas encontrados, las validaciones realizadas y las limitaciones de cada prototipo hasta la solución final, demostrando la capacidad de documentar y presentar una solución técnica de forma coherente y verificable.

Estas competencias no se han aplicado de forma aislada, sino de manera integrada durante todo el desarrollo del proyecto.

A su vez, se han aplicado conocimientos previos vistos en las siguientes asignaturas del plan de estudios del *Grado en Ingeniería en Tecnologías de la Telecomunicación*: *Sistemas Operativos*, *Seguridad en Redes de Ordenadores*, *Programación de Sistemas de Telecomunicación*, *Sistemas Telemáticos*, *Ingeniería de Sistemas Telemáticos* e *Ingeniería de Sistemas de Información*.

El resultado final no solo materializa una implementación funcional, sino que también refleja un proceso de aprendizaje técnico autónomo, razonamiento ingenieril, validación experimental y documentación ajustada al alcance de un proyecto final de ingeniería.

6.4. Trabajo futuro

Aunque el prototipo desarrollado cumple los objetivos definidos y demuestra la viabilidad del mecanismo propuesto, todavía existen distintas líneas de trabajo que permitirían evolucionarlo hacia una solución más completa, robusta y cercana a un entorno operativo real. Estas mejoras estarían orientadas tanto a ampliar la información crítica protegida y a dar la posibilidad de seleccionar la información sensible a solicitar del sistema como a fortalecer el despliegue, la gestión de claves y la integración con infraestructuras de monitorización.

Una primera línea de evolución consistiría en mejorar la gestión de la clave simétrica compartida K , uno de los aspectos más críticos. En el prototipo actual, la clave se genera previamente y se embebe en el módulo y en la utilidad a nivel de usuario para poder validar el funcionamiento del mecanismo *HMAC-SHA256* y así permitir su uso por parte del administrador durante la verificación. Sin embargo, una solución más realista debería evitar que la clave quedase incorporada de forma estática en el código o en el binario del *LKM*.

Para ello, podrían estudiarse mecanismos de provisión segura de claves, integración con almacenes protegidos, rotación periódica, uso de módulos de seguridad hardware (*HSM*) o apoyo en tecnologías como *TPM*, de forma que la autenticación ya no dependiese de una clave fija sin protección de acceso.

Otra línea importante sería adaptar el sistema a despliegues más amplios, tanto *on-premise* como en entornos distribuidos o de nube. El mecanismo podría evolucionar para que varios nodos generasen reportes autenticados de forma independiente y los enviaran a un componente central de verificación, permitiendo construir una visión agregada del estado de una infraestructura operativa.

En este escenario sería necesario definir protocolos de envío, identificación de nodos, políticas de confianza, almacenamiento seguro de reportes y mecanismos de correlación

temporal, especialmente si se desea aplicar el sistema en arquitecturas con contenedores, máquinas virtuales o múltiples servidores *Linux* en red.

También sería posible ampliar el tipo de información crítica obtenida desde *espacio del kernel* y permitir que esta se solicitase de forma dinámica y selectiva. En este trabajo se ha tomado como caso principal el listado de procesos activos, pero la misma aproximación podría extenderse para que el administrador solicitase en cada ejecución distintos tipos de información, como conexiones de red, módulos cargados, ficheros abiertos, información de usuarios, cambios en tablas internas o eventos de creación y finalización de procesos.

Estas ampliaciones permitirían construir un mecanismo de reporte más completo y flexible, aunque requerirían definir una interfaz de solicitud más expresiva y estudiar cuidadosamente el coste de recolección, la estabilidad del sistema y la cantidad de información transferida.

De cara a una aplicación práctica, también sería conveniente mejorar la automatización de la verificación. Actualmente, el administrador ejecuta la utilidad de comprobación sobre el fichero de transferencia generado desde *espacio de usuario*. Una evolución natural sería integrar esta verificación en una herramienta de análisis que comparase automáticamente el reporte autenticado del *kernel* con la salida de herramientas convencionales de *espacio de usuario*, señalando discrepancias relevantes y generando alertas cuando se detectasen posibles indicios de ocultación o manipulación.

Por último, sería necesario endurecer el prototipo desde el punto de vista operativo. Esto incluiría ampliar las pruebas en distintas versiones del *kernel*, revisar el comportamiento bajo cargas concurrentes, mejorar la gestión de errores, incorporar controles de acceso más estrictos sobre la interfaz */proc*, auditar el código con herramientas específicas y documentar procedimientos seguros de instalación, carga y descarga del módulo.

Estas líneas permitirían avanzar desde un prototipo funcional hacia una herramienta más madura, manteniendo siempre el modelo de amenaza definido y las limitaciones inherentes a confiar en la integridad del *kernel*.

En conclusión, estas ideas no agotan todas las posibles evoluciones del proyecto, sino que representan algunas implementaciones futuras especialmente relevantes para aumentar su seguridad, flexibilidad y aplicabilidad en escenarios reales.

A partir del prototipo desarrollado, podrían explorarse nuevas mejoras técnicas y operativas que no se recogen en este apartado, pudiendo así ampliar las capacidades y prestaciones del sistema diseñado de cara a un funcionamiento más robusto y a una posible puesta en producción.

Para concluir este proyecto, desde una perspectiva personal, me gustaría finalizar remarcando que este *Trabajo de Fin de Grado* me ha supuesto un recorrido intenso de aprendizaje, investigación profunda y diseño, implementación y validación exigentes, en el que se ha partido de un problema realmente complejo de abordar hasta construir una solución funcional y coherente con los objetivos planteados.

Más allá del resultado técnico obtenido, este periodo de desarrollo me ha permitido consolidar conocimientos y adquirir nuevas habilidades mientras disfrutaba del proceso, pudiendo trabajar sobre un tema que verdaderamente me genera un gran interés. Por ello, quiero expresar de nuevo mi agradecimiento a mi tutor por su orientación y acompañamiento durante todo el proceso, a todas las personas cercanas que me han apoyado a lo largo no solo de este proyecto, sino de toda la carrera, y, finalmente, a quienes han dedicado parte de su tiempo a la lectura de esta memoria.

Espero que haya resultado de gran interés y pueda aportar algún valor, ya sea como conocimiento o como punto de partida o apoyo para futuros proyectos.

Muchas gracias por todo.

Apéndice A

Código y validación de los prototipos

En este anexo se explica en detalle el código empleado en la implementación y validación de los prototipos desarrollados a lo largo del proyecto. Para ello, se incluyen los fragmentos de código más relevantes, los scripts auxiliares utilizados durante el desarrollo y las salidas obtenidas en las pruebas de validación realizadas.

El código fuente completo del proyecto también se encuentra disponible en el repositorio público <https://github.com/nowelito28/TFG>.

A.1. Prototipos, Implementaciones y Validaciones

El desarrollo principal del proyecto se articuló mediante varios prototipos de *LKM* contruidos progresivamente y apoyados en las funcionalidades validadas en los anteriores. Esta estrategia permitió avanzar desde una primera toma de contacto con la programación en *espacio del kernel* hasta la construcción del prototipo final capaz de generar un listado autenticado de los procesos activos del sistema desde dicha región privilegiada.

A.1.1. *Hello World*

El primer paso del desarrollo consistió en construir un *LKM* básico cuyo objetivo era verificar cómo funciona el flujo básico de trabajo asociado a los *módulos del kernel*.

Para ello, se diseñó inicialmente el fichero de compilación externa *Makefile* basado en *kbuild*, que delega el proceso de compilación en el sistema de construcción del propio *kernel* y se reutiliza en la compilación de los prototipos posteriores. En este fichero se

define la variable `obj-m`, que indica a *kbuild* qué fichero objeto (`.o`) debe compilarse como módulo cargable (`.ko`). Por otro lado, se declaran dos reglas principales que estructuran el flujo de compilación. La primera, `all`, invoca el sistema de construcción del *kernel* activo mediante la ruta `/lib/modules/$(shell uname -r)/build`, y especifica la ruta del código fuente del módulo a compilar en el directorio actual con `M=$(shell pwd)`, por lo que para realizar la compilación es necesario situar el *Makefile* en el mismo directorio que el código fuente del módulo y ejecutar la compilación desde ese mismo directorio. La regla `clean`, como segunda regla, elimina los ficheros intermedios generados durante la compilación. Por último, se declaran dichas reglas como *phony targets* para evitar conflictos con ficheros del mismo nombre. De este modo, el módulo puede compilarse de manera externa sin necesidad de modificar ni recompilar el núcleo del sistema.

```
obj-m := module_name.o

all: aux.sh
    make -C /lib/modules/$(shell uname -r)/build M=$(shell pwd) modules

clean:
    make -C /lib/modules/$(shell uname -r)/build M=$(shell pwd) clean

.PHONY: all clean
```

Código A.1: *Makefile* genérico empleado para compilar *LKMs*.

En caso de ser necesario proporcionar alguna dependencia previa a la compilación, como un fichero auxiliar, esta puede declararse en la regla `all`, como se muestra en el código anterior con el fichero `aux.sh`. En este prototipo inicial no fue necesario hacer uso de esta característica, por lo que la dependencia mostrada debe entenderse únicamente como un ejemplo general de cómo declararla en caso de que fuera necesario.

Una vez definida la herramienta de compilación mediante *Makefile* y *kbuild*, se implementó un módulo simple, `hello_world.c`, con el fin de validar el ciclo de vida básico de un *LKM*: carga del módulo en el *kernel*, ejecución de funciones controladoras de carga y descarga del módulo, registro de mensajes en el *log* del *kernel* y posterior descarga del módulo.

En primer lugar, se incluyeron las cabeceras necesarias para compilar el módulo y se declararon los metadatos básicos del componente:

```
#include <linux/module.h>
#include <linux/kernel.h>
#include <linux/init.h>

MODULE_LICENSE("Dual BSD/GPL");
MODULE_AUTHOR("Noel Rodriguez Perez");
MODULE_DESCRIPTION("A simple Hello world LKM!");
MODULE_VERSION("0.1");
```

Código A.2: Cabeceras y metadatos del módulo *Hello World*.

En este fragmento se incorporan las cabeceras necesarias para programar un *LKM* de *Linux* y, por otro lado, se definen varios metadatos descriptivos del componente, visibles posteriormente mediante la herramienta `modinfo`. Entre ellos, la licencia resulta especialmente importante, debido a que condiciona el comportamiento del módulo dentro del *kernel* y determina si este queda marcado como *tainted* al cargarse, no considerándose completamente limpio a efectos de depuración y soporte. En este caso, se elige una licencia dual *BSD/GPL*. Por un lado, la compatibilidad con *GPL*, empleada por el propio *kernel* de *Linux*, evita que el módulo quede marcado como *tainted*. Por otro lado, la licencia *BSD*, de carácter permisivo, permite el uso del módulo en entornos privativos sin tener que compartir el código fuente.

A continuación, el módulo necesita implementar sus funciones de inicialización y salida:

```
static int __init hello_start(void)
{
    printk(KERN_INFO "Loading hello world module...\n");
    printk(KERN_INFO "Hello world!!\n");
    return 0;
}

static void __exit hello_end(void)
{
    printk(KERN_INFO "Goodbye Mr.\n");
}
```

Código A.3: Funciones de inicialización/carga y salida/descarga del módulo *Hello World*.

La función `hello_start` se ejecuta al cargar el módulo en el *kernel* con `insmod <module_name>.ko`. En este prototipo, su única finalidad es escribir mensajes

en el *buffer* de *logs* del *kernel* mediante la función `printk`. Dichos mensajes pueden consultarse directamente mediante `dmesg` y, dependiendo de la configuración del sistema de registro, también quedan reflejados en ficheros de *logs* del *kernel* como `/var/log/kern.log`. Análogamente, la función `hello_end` se ejecuta al descargar el módulo con `rmmmod <module_name>`.

El uso de `printk` se verificó en el código fuente del registro de mensajes del *kernel*, expuesto en `include/linux/printk.h`, donde se definen la propia función y los distintos niveles de prioridad que pueden asociarse a cada traza, como `KERN_INFO`, `KERN_DEBUG` o `KERN_ERR`.

Finalmente, ambos controladores deben asociarse explícitamente al flujo de control del módulo:

```
module_init(hello_start);
module_exit(hello_end);
```

Código A.4: Asociación de los puntos de entrada/carga y salida/descarga del módulo.

Estas macros indican al *kernel* qué función debe ejecutarse durante la carga del módulo y cuál debe invocarse durante su descarga. Con ello queda definida la estructura mínima funcional de un *LKM* y puede procederse a validar su correcto funcionamiento.

Con permisos de administrador (*root*), se compiló el módulo, se cargó en el *kernel*, se inspeccionaron las trazas registradas en el log del *kernel*, se consultaron los metadatos del propio módulo y, finalmente, se descargó verificando también la ejecución de la rutina de salida:

```
$ make
make -C /lib/modules/6.12.41-current-bcm2711/build M=/home/LKM modules
make[1]: Entering directory '/usr/src/linux-headers-6.12.41-current-bcm2711'
  CC [M] /home/LKM/hello_world.o
  MODPOST /home/LKM/Module.symvers
  CC [M] /home/LKM/hello_world.mod.o
  LD [M] /home/LKM/hello_world.ko
make[1]: Leaving directory '/usr/src/linux-headers-6.12.41-current-bcm2711'
$ insmod hello_world.ko
$ dmesg -T | tail
...
[Fri Sep 5 19:45:58 2025] Loading hello world module...
[Fri Sep 5 19:45:58 2025] Hello world!!
$ modinfo hello_world.ko
filename:      /home/LKM/hello_world.ko
version:      0.1
```

```

description:  A simple Hello world LKM!
author:      Noel Rodriguez Perez
license:     Dual BSD/GPL
$ rmmod hello_world
$ dmesg -T | tail
...
[Fri Sep 5 19:46:30 2025] Goodbye Mr.

```

Código A.5: Compilación, carga, consulta y descarga del prototipo *Hello World*.

Mediante este flujo de trabajo se verificó que el módulo podía compilarse correctamente como un *LKM* externo, cargarse sin errores en el *kernel* ejecutando su rutina de inicialización, quedando correctamente asociados sus metadatos, ejecutar su rutina de salida al descargarse y registrar adecuadamente los mensajes mediante `printk` durante ambas fases.

A.1.2. Fichero virtual `/proc/fdev`

El siguiente paso consistió en dotar al módulo de una interfaz de comunicación entre *espacio de usuario* y *espacio del kernel*, implementado en `proc_fdev.c`. Para ello, se recurrió al sistema de ficheros virtual `/proc` para exponer información del *kernel* y permitir la interacción controlada con el *LKM* mediante la creación de un fichero virtual propio en este sistema, `/proc/fdev`.

El objetivo principal de esta fase fue comprobar que el módulo podía crear una entrada propia en `/proc`, asociarle funciones manejadoras (*handlers*) de lectura y escritura, y responder de manera controlada a estas operaciones invocadas desde *espacio de usuario*.

En primer lugar, fue necesario ampliar el conjunto de cabeceras incluidas en el módulo para poder trabajar con `/proc` y con las transferencias de datos entre *espacio de usuario* y *espacio del kernel*:

```

#include <linux/proc_fs.h>
#include <asm/uaccess.h>

```

Código A.6: Cabeceras adicionales para el prototipo basado en `/proc`.

Se incorporan `<linux/proc_fs.h>`, necesaria para crear y gestionar entradas dentro del sistema de ficheros virtual `/proc`, y por otro lado, `<asm/uaccess.h>`, que importa funcionalidades como `copy_to_user` y `copy_from_user`, utilizadas para transferir información de forma controlada y segura entre la memoria del *kernel* y la memoria de *espacio de usuario*.

A continuación, fue necesario definir una referencia a la entrada virtual y la tabla de operaciones asociada a dicha entrada en `/proc`:

```
static struct proc_dir_entry *ent;

static struct file_operations myops =
{
    .owner = THIS_MODULE,
    .read = myread,
    .write = mywrite,
};

static int simple_init(void)
{
    ent = proc_create("fdev", 0660, NULL, &myops);
    if (!ent)
        return -ENOMEM;
    printk(KERN_INFO "/proc/fdev created\n");
    return 0;
}

static void simple_cleanup(void)
{
    proc_remove(ent);
    printk(KERN_INFO "/proc/fdev removed\n");
}

module_init(simple_init);
module_exit(simple_cleanup);
```

Código A.7: Referencia a la entrada virtual y tabla de operaciones asociadas en `/proc`.

En este fragmento, el puntero `ent` almacena la referencia a la entrada que se crea en `/proc` al cargar el módulo, permitiendo posteriormente su gestión y eliminación al descargarlo. Por otro lado, la estructura `myops` define la tabla de operaciones del fichero virtual a crear, asociando las acciones de lectura y escritura sobre dicho fichero con las funciones manejadoras (*handlers*) `myread` y `mywrite`, respectivamente. Además,

el campo `.owner = THIS_MODULE` vincula estas operaciones al propio módulo cargado. Por último, `simple_init` crea la entrada virtual `/proc/fdev` mediante `proc_create` durante la carga del módulo, devolviendo `-ENOMEM` (*no memory error*) si la entrada no puede crearse, mientras que `simple_cleanup` la elimina con `proc_remove` durante su descarga.

El empleo de `struct proc_dir_entry`, `proc_create` y `proc_remove` se contrastó en el código fuente del sistema de ficheros `/proc` del *kernel*, definido en `include/linux/proc_fs.h` y desarrollado en el subsistema `fs/proc`, mientras que `struct file_operations` se implementa en la interfaz general de operaciones sobre ficheros expuesta en `include/linux/fs.h`.

Un aspecto a resaltar en el desarrollo de los prototipos de *módulos del kernel* es la conveniencia de usar *static* para declarar variables y funciones que no se van a utilizar fuera del propio módulo, restringiendo así su visibilidad al código fuente del módulo y favoreciendo una mejor encapsulación del código, por lo que este criterio se mantendrá en el resto del proyecto.

Una vez creada la entrada al fichero virtual `/proc/fdev` y asociada su tabla de operaciones, se implementaron los *handlers* de escritura y lectura de dicho fichero:

```
enum { BUFSIZE = 100 };

static int irq = 20;
static int mode = 1;

static ssize_t mywrite(struct file *file, const char __user *ubuf,
                      size_t count, loff_t *ppos)
{
    int num, c, i, m;
    char buf[BUFSIZE];

    if (*ppos > 0 || count >= BUFSIZE)
        return -EFAULT;

    printk(KERN_DEBUG "/proc/fdev: write handler\n");

    if (copy_from_user(buf, ubuf, count))
        return -EFAULT;

    num = sscanf(buf, "%d %d", &i, &m);
    if (num != 2)
        return -EFAULT;
```

```

    irq = i;
    mode = m;

    c = strlen(buf);
    *ppos = c;

    printk(KERN_DEBUG "/proc/fdev: %d bytes received from userspace\n", c);
    return c;
}

static ssize_t myread(struct file *file, char __user *ubuf,
                     size_t count, loff_t *ppos)
{
    char buf[BUFSIZE];
    int len = 0;

    if (*ppos > 0 || count < BUFSIZE)
        return 0;

    printk(KERN_DEBUG "/proc/fdev: read handler\n");

    len += sprintf(buf, "irq = %d\n", irq);
    len += sprintf(buf + len, "mode = %d\n", mode);

    if (copy_to_user(ubuf, buf, len))
        return -EFAULT;

    *ppos = len;

    printk(KERN_DEBUG "/proc/fdev: %d bytes sent to userspace\n", len);
    return len;
}

```

Código A.8: *Handlers* con transferencia de datos entre *espacio de usuario* y *espacio del kernel*.

La función manejadora de escritura, `mywrite`, recibe datos desde *espacio de usuario* mediante `copy_from_user`, trasladando `count` bytes del *buffer* de memoria de usuario (`ubuf`) a un *buffer* en memoria del *kernel* (`buf`). Análogamente, `myread` devuelve datos hacia *espacio de usuario* mediante `copy_to_user`, trasladando `len` bytes del *buffer* en memoria del *kernel* (`buf`) al *buffer* de memoria de usuario (`ubuf`). Además, mediante el puntero de posición del fichero (`*ppos`) se impuso una semántica de acceso de tipo *single-shot*.

El empleo de `copy_from_user` y `copy_to_user` se contrastó en la infraestructura `uaccess` del código fuente del *kernel*, concretamente en `include/linux/uaccess.h`, donde ambas funciones se apoyan sobre las primitivas internas `raw_copy_from_user` y `raw_copy_to_user`.

En este ejemplo, los valores `irq` y `mode` se emplean como variables enteras de prueba para validar el intercambio de información entre *espacio de usuario* y *espacio del kernel*, permitiendo comprobar su correcta interpretación a partir de los datos recibidos mediante `sscanf`, así como su formateo para la salida mediante `sprintf`, y retornando errores como `-EFAULT` (*bad address*) cuando no se cumplen dichas restricciones o cuando el formato de entrada no es el esperado. Por otro lado, se define una constante de tipo enumerado `BUFSIZE` para fijar de forma simple el tamaño máximo del *buffer* temporal utilizado, en este caso 100 bytes, tanto para la lectura como para la escritura.

Ya creado el fichero virtual `/proc/fdev` y sus funciones manejadoras de lectura y escritura, se procedió a comprobar el correcto funcionamiento del prototipo asumiendo que se dispone de permisos de administrador (`root`) sobre el sistema:

```
$ make
make -C /lib/modules/6.12.41-current-bcm2711/build M=/home/LKM modules
make[1]: Entering directory '/usr/src/linux-headers-6.12.41-current-bcm2711'
  CC [M] /home/LKM/proc_fdev.o
  MODPOST /home/LKM/Module.symvers
  CC [M] /home/LKM/proc_fdev.mod.o
  LD [M] /home/LKM/proc_fdev.ko
make[1]: Leaving directory '/usr/src/linux-headers-6.12.41-current-bcm2711'
$ insmod proc_fdev.ko
$ dmesg -T | tail
...
[Tue Oct 14 17:25:58 2025] /proc/fdev created
$ ls -l /proc/fdev
-rw-rw---- 1 root root 0 Oct 14 17:25 /proc/fdev
$ echo "7 3" > /proc/fdev
$ dmesg -T | tail
...
[Tue Oct 14 17:26:10 2025] /proc/fdev: write handler
[Tue Oct 14 17:26:10 2025] /proc/fdev: 4 bytes received from userspace
$ cat /proc/fdev
irq = 7
mode = 3
$ dmesg -T | tail
...
[Tue Oct 14 17:26:18 2025] /proc/fdev: read handler
[Tue Oct 14 17:26:18 2025] /proc/fdev: 17 bytes sent to userspace
```

```
$ rmmod proc_fdev
$ dmesg -T | tail
...
[Tue Oct 14 17:26:30 2025] /proc/fdev removed
```

Código A.9: Validación básica del prototipo basado en `/proc/fdev`.

Mediante esta validación se comprobó que toda la funcionalidad del módulo se ejecutaba correctamente, tanto la creación de la entrada virtual `/proc/fdev`, como sus operaciones personalizadas de lectura y escritura, asegurando una vía básica de comunicación entre *espacio de usuario* y *espacio del kernel*.

A.1.3. *File Handoff*

Una vez creada una entrada válida en `/proc` y logrado así el intercambio básico de información entre *espacio de usuario* y *espacio del kernel* de forma controlada, el siguiente paso consistió en implementar un mecanismo que permitiese al módulo operar sobre un fichero real del sistema de ficheros, materializado en `fd_handoff.c`. Para ello, se planteó el traspaso de la referencia de un fichero (*File Descriptor*) desde *espacio de usuario* hacia el *LKM*, utilizando como interfaz de entrada el propio fichero virtual `/proc/fdev` desarrollado en el prototipo anterior.

El objetivo general de este prototipo fue comprobar que el módulo podía recibir desde *espacio de usuario* el *FD* de un fichero codificado como cadena de *bytes* y convertirlo a un valor de tipo *int* (entero), resolver internamente dicha referencia para localizar en el *kernel* el fichero correspondiente y utilizarlo para transferir información desde dicha región privilegiada hacia un fichero del sistema. Con ello, el administrador puede decidir en qué fichero se deposita la información solicitada de forma controlada.

Fue necesario ampliar nuevamente las cabeceras del módulo para su desarrollo:

```
#include <linux/file.h>
#include <linux/fs.h>
#include <linux/kstrtox.h>
#include <linux/security.h>
```

Código A.10: Cabeceras adicionales para el mecanismo de *File Handoff*.

La cabecera `<linux/file.h>` permite trabajar con las referencias internas a ficheros del *kernel*, como la estructura de datos `struct file`, así como resolver un *FD* mediante `fget`. Por otro lado, `<linux/fs.h>` aporta definiciones generales del subsistema de ficheros, incluyendo modos y *flags* de apertura. También se añade la cabecera `<linux/kstrtox.h>`, que proporciona funciones de conversión seguras, como `kstrtoint`, para transformar una cadena de *bytes* en un entero. Finalmente, `<linux/security.h>` permite validar permisos sobre ficheros referenciados mediante mecanismos de control de acceso del *kernel*, como `file_permission`.

En cuanto a la función manejadora de lectura, se simplificó respecto a la del anterior prototipo, limitándose a devolver una cadena de información determinista que indicase al administrador que el módulo se encuentra preparado para recibir un *Descriptor de Fichero* (*FD*) y realizar el *File Handoff*:

```
static ssize_t myread(struct file *file, char __user *ubuf,
                    size_t count, loff_t *ppos)
{
    char buf[] = "Waiting for a FD to perform file handoff\n";
    int len = strlen(buf);

    if (*ppos > 0 || count < len)
        return 0;
    printk(KERN_DEBUG "/proc/fdev: read handler\n");

    if (copy_to_user(ubuf, buf, len))
        return -EFAULT;

    *ppos = len;
    printk(KERN_DEBUG "/proc/fdev: %d bytes sent to userspace\n", len);
    return len;
}
```

Código A.11: Función de lectura para el mecanismo de *File Handoff*.

Sin embargo, la funcionalidad principal de este prototipo se concentra en la función manejadora de escritura, la cual necesita funciones auxiliares para organizar la lógica de escritura de forma más clara y modular:

```
static int write_full(struct file *f, const char *buf, int len)
{
    int w = 0;
    loff_t *ppos = &f->f_pos;

    w = kernel_write(f, buf, len, ppos);
}
```

```
if (w < 0)
    return w;

if (w != len)
    return -EIO;

return w;
}
```

Código A.12: Función auxiliar de escritura sobre un fichero desde el *LKM*.

La función auxiliar `write_full` encapsula la escritura sobre un fichero ya abierto desde *espacio del kernel* mediante una única llamada a `kernel_write`. Para ello, utiliza como posición de escritura el puntero asociado al fichero (`f_pos`) para escribir al final del mismo, comprueba si la llamada devuelve un error y verifica que el número de *bytes* escritos (`w`) coincida con la longitud solicitada (`len`). En caso contrario, se retorna un código de error (`-EIO`), manteniendo centralizada la comprobación de la escritura dentro del *LKM*.

El uso de `kernel_write` se apoyó en la verificación de su implementación dentro del código fuente del *VFS* del *kernel*, concretamente en el fichero `fs/read_write.c`.

```
static int val_metadata(struct file *f)
{
    int rv = 0;

    if (!(f->f_mode & FMODE_WRITE) || !(f->f_flags & O_APPEND))
        return -EBADF;

    rv = file_permission(f, MAY_WRITE | MAY_APPEND);
    if (rv < 0)
        return rv;

    return 0;
}
```

Código A.13: Función auxiliar para la validación de permisos de un fichero desde el *LKM*.

En este caso, la función auxiliar `val_metadata` encapsula la lógica de validación previa del fichero referenciado desde *espacio de usuario*, comprobando que se puede usar de forma segura en modo *append* desde *espacio del kernel*. En primer lugar, se

verifica que haya sido abierto con capacidad de escritura (`FMODE_WRITE`) y con el *flag* `O_APPEND`, evitando accesos incompatibles con la política definida para este prototipo. Seguidamente, se comprueban sus permisos efectivos mediante `file_permission`, asegurando que el *kernel* puede realizar sobre él operaciones de escritura y anexo, mediante los permisos `MAY_WRITE` y `MAY_APPEND`. En caso de que alguna de estas restricciones no se cumpla, la función retorna el código de error correspondiente, devolviendo `-EBADF` (*bad file error*) cuando los metadatos de apertura no son válidos.

En esta parte se tuvo que consultar también el código fuente del *kernel*. El uso de `file_permission` se apoyó en su definición en el fichero `include/linux/fs.h`, mientras que la relación entre `O_APPEND` y `MAY_APPEND` se contrastó en el fichero `fs/open.c`. Además, en `fs/namei.c`, dentro de la función `inode_permission`, se expone explícitamente que toda comprobación con `MAY_APPEND` debe incluir también `MAY_WRITE`.

```
static ssize_t mywrite(struct file *file, const char __user *ubuf,
                      size_t count, loff_t *ppos)
{
    int fd = 0;
    int rv = 0;
    char buf[BUFSIZE];
    static const char msg[] = "Kernel-Space: Writing test for file
                               handoff\n";
    struct file *f = NULL;

    if (*ppos > 0 || count > BUFSIZE)
        return -EFAULT;

    printk(KERN_DEBUG "/proc/fdev: write handler\n");

    if (copy_from_user(buf, ubuf, count))
        return -EFAULT;

    rv = strlen(buf);
    *ppos = rv;

    if (kstrtoint(buf, 10, &fd))
        return -EINVAL;

    if (fd < 0)
        return -EBADF;
```

```

    f = fget(fd);
    if (!f)
        return -EBADF;

    rv = val_metadata(f);
    if (rv < 0)
        goto out_put;

    rv = write_full(f, msg, strlen(msg));
    if (rv < 0)
        goto out_put;

    printk(KERN_DEBUG "/proc/fdev: %d bytes written to handoff file\n", rv);

out_put:
    fput(f);
    return rv;
}

```

Código A.14: Función de escritura para el mecanismo de *File Handoff*.

Finalmente, la función manejadora `mywrite` concentra la lógica principal del mecanismo de *File Handoff*, recibiendo desde *espacio de usuario* el descriptor de fichero codificado como cadena de *bytes* a través de `/proc/fdev`. En primer lugar, al igual que en el anterior prototipo, se comprueba que se cumpla la semántica de acceso de tipo *single-shot* y que no se exceda el tamaño máximo fijado por `BUFSIZE`, realizando posteriormente el traspaso de información a memoria del *kernel* mediante `copy_from_user`.

A continuación, la cadena de *bytes* recibida se convierte a un valor entero decimal con `kstrtoint`, obteniendo así el *FD* correspondiente. De esta manera, mediante `fget` y el *FD* obtenido, se recupera la referencia al fichero real dentro del *kernel*, tras lo cual se verifican sus permisos con `val_metadata` para comprobar que se ajustan a la política fijada, y se escribe en él una cadena estática de prueba mediante `write_full`.

Finalmente, la referencia al fichero se libera con `fput`, devolviendo en cada caso el código de error correspondiente si alguna de las comprobaciones falla, con ayuda del uso de la etiqueta `goto` (`out_put`).

El empleo de `kstrtoint`, `fget` y `fput` se contrastó con el código fuente del *kernel*, localizando `kstrtoint` en el fichero `lib/kstrtox.c`, mientras que `fget` y `fput` se

encuentran dentro de `fs/file.c` y `fs/file_table.c`, variando las líneas en las que se localizan según la versión del núcleo considerada.

Para completar este prototipo fue necesario implementar también una utilidad extra en *espacio de usuario*, `mk_fhandoff.c`:

```
#include <stdlib.h>
#include <stdio.h>
#include <string.h>
#include <err.h>
#include <errno.h>
#include <fcntl.h>
#include <unistd.h>

static int write_full(int fd, const char *buf, int len)
{
    int w = 0;

    w = write(fd, buf, len);
    if (w < 0)
        return -1;

    if (w != len) {
        errno = EIO;
        return -errno;
    }

    return w;
}

int main(int argc, char *argv[])
{
    if (argc != 3) {
        errx(EXIT_FAILURE, "Usage: %s <common_path> <proc_path>\n"
            "e.g.: %s ./file_handoff /proc/fdev", argv[0], argv[0]);
    }

    const char *f_handoff = argv[1];
    const char *f_proc = argv[2];

    int fd_handoff = open(f_handoff, O_CREAT | O_RDWR | O_APPEND, 0666);

    if (fd_handoff < 0) {
        err(EXIT_FAILURE, "Error opening/creating the file %s",
            f_handoff);
    }
}
```

```

int fd_proc = open(f_proc, O_RDWR);
if (fd_proc < 0) {
    err(EXIT_FAILURE, "Error opening %s", f_proc);
}

char fd_str[12];
snprintf(fd_str, sizeof(fd_str), "%d", fd_handoff);

if (write_full(fd_proc, fd_str, strlen(fd_str)) <= 0) {
    close(fd_proc);
    close(fd_handoff);

    err(EXIT_FAILURE, "Error writing in %s", f_proc);
}

printf("File descriptor written in %s:\n%d\n", f_proc, fd_handoff);

close(fd_proc);
close(fd_handoff);

exit(EXIT_SUCCESS);
}

```

Código A.15: Programa `mk_fhandoff.c` de *espacio de usuario* para activar el *File Handoff*.

Este programa actúa como intermediario desde *espacio de usuario*, creando o abriendo el fichero real que actuará como destino a partir del nombre indicado como primer argumento, obteniendo su *FD* y enviándolo al módulo a través de `/proc/fdev`, cuya ruta se indica como segundo argumento al ejecutar el programa, completando así el mecanismo de *File Handoff*. Cabe destacar que en esta utilidad de usuario se implementó también una función auxiliar de escritura (`write_full`), encargada de encapsular la llamada a `write` y comprobar que la operación no falle y que el número de *bytes* escritos coincida con el tamaño solicitado.

Una vez terminado el mecanismo completo de *File Handoff* básico, se procedió a validar su funcionamiento, comprobando que el módulo recibía correctamente el *FD* de un fichero real desde *espacio de usuario*, lo resolvía dentro del *kernel* y escribía sobre dicho fichero la cadena definida en `mywrite`:

```

$ uname -r
6.12.41-current-bcm2711
$ cd LKM
$ make
make -C /lib/modules/6.12.41-current-bcm2711/build M=/home/LKM modules

```

```

make[1]: Entering directory '/usr/src/linux-headers-6.12.41-current-bcm2711'
  CC [M] /home/LKM/fd_handoff.o
  MODPOST /home/LKM/Module.symvers
  CC [M] /home/LKM/fd_handoff.mod.o
  LD [M] /home/LKM/fd_handoff.ko
make[1]: Leaving directory '/usr/src/linux-headers-6.12.41-current-bcm2711'
$ insmod fd_handoff.ko
$ dmesg -T | tail
...
[Wed Oct 29 16:25:15 2025] /proc/fdev created
$ cd ..
$ gcc -c -Wshadow -Wvla -Wall mk_fhandoff.c
$ gcc -o mk_fhandoff mk_fhandoff.o
$ cat /proc/fdev
Waiting for a FD to perform file handoff
$ dmesg -T | tail
...
[Wed Oct 29 16:28:27 2025] /proc/fdev: read handler
[Wed Oct 29 16:28:27 2025] /proc/fdev: 41 bytes sent to userspace
$ ./mk_fhandoff ./file_handoff /proc/fdev
File descriptor written in /proc/fdev:
3
$ dmesg -T | tail
...
[Wed Oct 29 16:30:11 2025] /proc/fdev: write handler
[Wed Oct 29 16:30:11 2025] /proc/fdev: 1 bytes written to handoff file
$ cat ./file_handoff
Kernel-Space: Writing test for file handoff

$ cd LKM
$ rmmod fd_handoff
$ dmesg -T | tail
...
[Wed Oct 29 16:32:20 2025] /proc/fdev removed

```

Código A.16: Validación del mecanismo de *File Handoff*.

A través de esta validación, se comprobó que el módulo podía recibir correctamente desde *espacio de usuario* el *FD* de un fichero real, resolver su referencia en el *kernel* y utilizarla como destino efectivo de escritura mediante el mecanismo de *File Handoff*.

Como consecuencia, quedó implementado un canal funcional para transferir información generada en *espacio del kernel* de forma controlada hacia el fichero indicado por *espacio de usuario*, sirviendo como base para la incorporación posterior del contenido crítico autenticado del reporte seguro.

A.1.4. Mecanismo criptográfico de autenticación *HMAC-SHA256*

Una vez validado el proceso de *File Handoff*, el siguiente paso consistió en incorporar en el módulo `crypto_fd.c` un mecanismo de autenticación criptográfica del contenido generado en *espacio del kernel* (*HMAC-SHA256*) e incluir dicha etiqueta *HMAC* en el fichero final de transferencia, con el fin de garantizar su integridad y autenticidad apoyándose en la infraestructura validada anteriormente. De este modo, dicho contenido puede verificarse posteriormente desde *espacio de usuario* con la misma clave secreta compartida.

Para ello, fue necesario ampliar las cabeceras necesarias para la compilación del módulo e incorporar la clave simétrica embebida con la que se calcula el *HMAC*, la cual debe estar también disponible para el administrador durante su posterior verificación.

```
#include <crypto/hash.h>
#include <linux/base64.h>
#include <linux/crypto.h>

#include "k_embedded.h"
```

Código A.17: Cabeceras adicionales y clave embebida para el mecanismo de autenticación *HMAC-SHA256*.

En este punto, `<crypto/hash.h>` y `<linux/crypto.h>` permiten utilizar las funcionalidades de la *Crypto API* del *kernel* para solicitar una instancia síncrona de tipo `shash`, mientras que `<linux/base64.h>` permite codificar el resultado binario del *HMAC* en una representación textual portable en formato *Base64*. Por otro lado, la inclusión de `"k_embedded.h"` introduce una clave secreta simétrica binaria generada externamente y embebida en el módulo como un vector constante de *bytes*, de forma que puede emplearse directamente durante el cálculo del *HMAC*.

En consecuencia, el proceso de compilación del módulo deja de depender únicamente del fichero principal y pasa a requerir previamente la cabecera que contiene la clave embebida. Para ello, se diseñó un script auxiliar `gen_k_embedded.sh`, encargado de generar una clave aleatoria de 64 *bytes*, almacenarla en un fichero binario `key.bin`

y transformarla después en la cabecera `k_embedded.h` como un vector constante de *bytes*, pudiendo así importarla en el *LKM*. A su vez, el *Makefile* de compilación del *LKM* pasa a declarar `k_embedded.h` como dependencia previa de la regla `all`.

```
#!/bin/sh
set -e

KEY_LEN=64
openssl rand -out key.bin ${KEY_LEN}

xxd -i -n K key.bin | \
    sed -e 's/^unsigned char/const unsigned char/' \
        -e 's/^unsigned int/const unsigned int/' > k_embedded.h

echo "OK: generated k_embedded.h with key K (${KEY_LEN} bytes)."
```

Código A.18: Script `gen_k_embedded.sh` para generar la cabecera `k_embedded.h` con la clave embebida.

Cabe señalar que, para poder utilizar este script, una vez creado, fue necesario asignarle permisos de ejecución externamente mediante `chmod +x gen_k_embedded.sh`, pudiendo ejecutarse posteriormente con `./gen_k_embedded.sh` en el mismo directorio de trabajo.

En cuanto a la programación de la lógica de codificación del contenido para obtener el *HMAC* en el *LKM*, se implementó esta funcionalidad para calcular el *HMAC-SHA256* a partir de un bloque de datos tratado como *array* de *bytes*:

```
static int get_hmac_sha256(const u8 *buf, int buf_len, u8 **hmac, int
    *hmac_len)
{
    int rv = 0;
    struct crypto_shash *tfm;

    tfm = crypto_alloc_shash("hmac(sha256)", 0, 0);

    if (IS_ERR(tfm))
        return PTR_ERR(tfm);

    rv = crypto_shash_setkey(tfm, K, K_len);
    if (rv)
        goto out_free_tfm;

    *hmac_len = crypto_shash_digestsize(tfm);
```

```

    *hmac = kmalloc(*hmac_len, GFP_KERNEL);
    if (!*hmac) {
        rv = -ENOMEM;
        goto out_free_tfm;
    }

    SHASH_DESC_ON_STACK(desc, tfm);
    desc->tfm = tfm;

    rv = crypto_shash_digest(desc, buf, buf_len, *hmac);

out_free_tfm:
    crypto_free_shash(tfm);
    return rv;
}

```

Código A.19: Cálculo del *HMAC-SHA256* para una cadena de *bytes* de datos.

Esta función encapsula la lógica principal de cálculo del autenticador criptográfico *HMAC-SHA256* sobre un bloque arbitrario de datos representado como un *array* de *bytes* en memoria del *kernel*.

Para ello, solicita a la *Crypto API* una instancia *hash* síncrona (*shash*) asociada al algoritmo *hmac*(*sha256*) mediante `crypto_alloc_shash`. Después, se le asocia la clave secreta embebida con `crypto_shash_setkey` y se obtiene el tamaño del *digest* con `crypto_shash_digestsize`. Posteriormente, se utiliza reserva en memoria dinámica para almacenar el *HMAC* a calcular con la longitud correspondiente (`hmac_len`) mediante `kmalloc`, y adicionalmente, la macro `SHASH_DESC_ON_STACK` construye en la pila de ejecución el descriptor temporal requerido por el subsistema criptográfico (`desc`). Tras ello, `crypto_shash_digest` calcula el *HMAC* completo en una única operación sobre el contenido recibido.

Una vez calculado el *HMAC* puro del contenido, se implementó una función auxiliar para convertirlo a formato *Base64* y poder así incluir en el fichero de transferencia de forma legible y portable:

```

static int get_hmac_b64(const u8 *hmac, int hmac_len, u8 **hmac_b64,
                       int *hmac_b64len)
{
    int hmac_b64cap = BASE64_CHARS(hmac_len);

    *hmac_b64 = kmalloc(hmac_b64cap, GFP_KERNEL);

```

```

    if (!*hmac_b64)
        return -ENOMEM;

    *hmac_b64len = base64_encode(hmac, hmac_len, *hmac_b64);
    if (*hmac_b64len < 0) {
        kfree(*hmac_b64);
        return *hmac_b64len;
    }

    return 0;
}

```

Código A.20: Conversión del *HMAC* binario a *Base64*.

Esta función auxiliar facilita la serialización del autenticador criptográfico en el fichero de transferencia, transformando el *HMAC* binario previamente calculado en una representación textual en *Base64*.

En primer lugar, determina la capacidad máxima necesaria para la cadena codificada mediante la macro `BASE64_CHARS` y reserva memoria dinámica para almacenarla. A continuación, la función `base64_encode` realiza la conversión efectiva de dicho *HMAC* puro, devolviendo en `hmac_b64` la cadena resultante y en `hmac_b64len` su longitud final. De este modo, el autenticador puede incorporarse al fichero de salida en un formato legible, portable y fácilmente verificable posteriormente desde *espacio de usuario*.

El empleo de `crypto_alloc_shash`, `crypto_shash_setkey`, `crypto_shash_digestsize`, `crypto_shash_digest` y la macro `SHASH_DESC_ON_STACK` se contrastó su funcionamiento e implementación en la infraestructura de *hash* síncrono de la *Crypto API* del *kernel*, expuesta en `include/crypto/hash.h` y desarrollada en el subsistema `crypto` dentro del código fuente del propio núcleo. Por otra parte, la codificación del autenticador en *Base64* se apoyó en `include/linux/base64.h` y en la implementación correspondiente dentro de `lib/base64.c`.

A nivel de formato de salida, el contenido ya no se escribe de forma aislada en el fichero de transferencia, sino acompañado por una separación explícita entre los datos generados por el *kernel* y su etiqueta *HMAC* de autenticación en formato *Base64*:

```

static const char sep[] = "\n--KERNEL-PS--\n";
static const int sep_len = sizeof(sep) - 1;
static const char sep_hmac[] = "\n--HMAC--\n";
static const int sep_hmac_len = sizeof(sep_hmac) - 1;

```

```
static int write_cont_hmac(struct file *f, const char *cont, int cont_len,
                          const char *hmac_b64, int hmac_b64len)
{
    int w = 0;
    int total = 0;

    w = write_full(f, sep, sep_len);
    if (w < 0)
        return w;
    total += w;

    w = write_full(f, cont, cont_len);
    if (w < 0)
        return w;
    total += w;

    w = write_full(f, sep_hmac, sep_hmac_len);
    if (w < 0)
        return w;
    total += w;

    w = write_full(f, hmac_b64, hmac_b64len);
    if (w < 0)
        return w;
    total += w;

    return total;
}
```

Código A.21: Escritura conjunta del contenido y su *HMAC* en *Base64*.

Esta función organiza la serialización final del contenido autenticado dentro del fichero de transferencia. Para ello, escribe secuencialmente separadores textuales distinguibles, empezando con un separador inicial previo al bloque generado por el *kernel* (*sep*), el contenido a proteger, el separador *sep_hmac* y, finalmente, el *HMAC* ya codificado en *Base64*, reutilizando la función *write_full* validada en el prototipo anterior.

De este modo, el fichero resultante queda estructurado de forma determinista, facilitando tanto la separación entre el contenido principal a transferir y el *HMAC* necesario para su posterior validación desde *espacio de usuario*.

Ya explicado el desarrollo del mecanismo de autenticación, se procedió a modificar la función manejadora de escritura para integrar todos los componentes unificados:

```

static int printh(struct file *f)
{
    int rv = 0;

    static const char cont[] =
        "Kernel-Space: Authentic content protected by HMAC-SHA256\n";
    const int cont_len = sizeof(cont) - 1;

    u8 *hmac = NULL;
    int hmac_len = 0;

    u8 *hmac_b64 = NULL;
    int hmac_b64len = 0;

    rv = get_hmac_sha256(cont, cont_len, &hmac, &hmac_len);
    if (rv < 0)
        goto out;

    rv = get_hmac_b64(hmac, hmac_len, &hmac_b64, &hmac_b64len);
    if (rv < 0)
        goto out_free_hmac;

    rv = write_cont_hmac(f, cont, cont_len, hmac_b64, hmac_b64len);
    if (rv < 0)
        goto out_free_hmac;

    printk(KERN_INFO "printh: file has been written with HMAC\n");

out_free_hmacs:
    kfree(hmac_b64);

out_free_hmac:
    kfree(hmac);

out:
    return rv;
}

```

Código A.22: Integración del mecanismo de autenticación.

Ahora, la función `printh` pasa a concentrar la lógica principal de integración del mecanismo diseñado en este prototipo. En este caso de prueba, se utiliza una cadena de contenido constante `cont`, a la cual se le calcula su *HMAC-SHA256*, se codifica en *Base64* y se escribe todo bien estructurado por separado en el fichero de transferencia.

```

static ssize_t mywrite(struct file *file, const char __user *ubuf,
                      size_t count, loff_t *ppos)
{
    int fd = 0;
    int rv = 0;
    char buf[BUFSIZE];

    if (*ppos > 0 || count > BUFSIZE)
        return -EFAULT;
    printk(KERN_DEBUG "/proc/fdev: write handler\n");

    if (copy_from_user(buf, ubuf, count))
        return -EFAULT;

    rv = strlen(buf);
    *ppos = rv;

    if (kstrtoint(buf, 10, &fd))
        return -EINVAL;
    if (fd < 0)
        return -EBADF;

    struct file *f = fget(fd);
    if (!f)
        return -EBADF;

    rv = val_metadata(f);
    if (rv < 0)
        goto out_put;

    rv = printh(f);
    if (rv < 0)
        goto out_put;
    printk(KERN_DEBUG "mywrite: printh OK for fd %d (%d bytes written)\n",
           fd, rv);

out_put:
    fput(f);

    return rv;
}

```

Código A.23: Manejador de escritura adaptado al mecanismo de autenticación de contenido.

Por su parte, `mywrite` mantiene la lógica general del prototipo anterior para recibir y resolver el *FD*, pero sustituye la escritura estática simple por la llamada a `printh`.

Una vez incorporado el mecanismo de autenticación al flujo del mecanismo de *File Handoff* en el *LKM*, fue necesario desarrollar una utilidad adicional en *espacio de usuario* que permitiese verificar la validez del contenido autenticado generado, destinada a ser utilizada con permisos de administrador. Para ello, se implementó el programa `check_hmac.c`, encargado de deserializar el contenido del fichero de transferencia y así separar el contenido principal de la etiqueta *HMAC* en formato *Base64*, pudiendo recalcularse de esta forma el autenticador con la misma clave secreta compartida y comparar ambos resultados para asegurar la integridad del contenido.

Antes de nada, se añadieron las cabeceras y constantes necesarias para la compilación de este programa, incluyendo la clave embebida compartida entre el *kernel* y el administrador para el cálculo del *HMAC*:

```
#include <stdlib.h>
#include <stdio.h>
#include <string.h>
#include <err.h>
#include <errno.h>
#include <fcntl.h>
#include <unistd.h>
#include <openssl/hmac.h>
#include <openssl/evp.h>
#include <openssl/crypto.h>

#include "./LKM/k_embedded.h"

enum { LEN = 1024 * 40 };

const char sep[] = "--KERNEL-PS--\n";
const char sep_hmac[] = "--HMAC--\n";
```

Código A.24: Cabeceras, constantes y clave simétrica necesarias para la verificación.

En primer lugar, las funciones `read_until_separator`, `read_hmac_line` y `extract_data` se encargan de deserializar el contenido del fichero de transferencia generado mediante el mecanismo de *File Handoff*, separando el contenido principal de la etiqueta *HMAC* almacenada en formato *Base64* mediante la detección de los separadores definidos:

```
int read_until_separator(FILE *f, char **content, int *content_len)
{
    char *line = NULL;
    size_t cap = 0;
    int len = 0;
```

```
int sep_found = 0;
int sep_hmac_found = 0;

*content = (char *)malloc(LEN);
if (!(*content)) {
    warnx("malloc failed for the content\n");

    return 1;
}

while ((len = getline(&line, &cap, f)) != -1) {

    if (!sep_found) {
        if (strcmp(line, sep) == 0) {
            sep_found = 1;
        }

        continue;
    }

    if (strcmp(line, sep_hmac) == 0) {
        sep_hmac_found = 1;

        break;
    }

    if (*content_len + len > LEN) {
        free(line);
        warnx("Content read exceeds 4KB buffer before separator\n");

        return 1;
    }

    memcpy(*content + *content_len, line, len);
    *content_len += len;
}

free(line);

if (len == -1) {
    warnx("Lines reading with buffering failed\n");

    return 1;
}

if (*content_len > 0 && (*content)[*content_len - 1] == '\n') {
    *content_len -= 1;
}
```

```
    if (sep_hmac_found == 0) {
        warnx("Separator not found\n");

        return 1;
    }

    return 0;
}

int read_hmac_line(FILE *f, char **hmac_b64, int *hmac_b64_len)
{
    size_t cap = 0;

    *hmac_b64_len = getline(hmac_b64, &cap, f);
    if (*hmac_b64_len == -1) {
        warnx("Error reading HMAC\n");

        return 1;
    }

    return 0;
}

int extract_data(FILE *fhandoff, char **content, int *content_len,
                 char **hmac_b64, int *hmac_b64_len)
{
    if (read_until_separator(fhandoff, content, content_len)
        || read_hmac_line(fhandoff, hmac_b64, hmac_b64_len)) {
        return 1;
    }

    return 0;
}
```

Código A.25: Deserialización del contenido del fichero autenticado en *espacio de usuario*.

A continuación, `calculate_hmac`, `encode_base64` y `calc_and_encode_hmac` recalculan localmente el autenticador a partir del contenido extraído y de la misma clave secreta utilizada en el módulo, la cual es compartida con el administrador que ejecuta este programa, utilizando funcionalidades de la librería `openssl` para simplificar los cálculos criptográficos.

Concretamente, la función `HMAC` recibe el algoritmo criptográfico correspondiente, la clave y el contenido con las longitudes de ambas variables, pudiendo calcular así el *HMAC* correspondiente al contenido directamente.

Por otro lado, `EVP_ENCODE_LENGTH` y `EVP_EncodeBlock` se emplean principalmente para codificar el *HMAC* previamente recalculado a formato *Base64*.

```

int calculate_hmac(const unsigned char *content, int content_len,
                  unsigned char *hmac_calc, unsigned int *hmac_calc_len)
{
    if (!HMAC(EVP_sha256(), K, K_len, content, content_len, hmac_calc,
              hmac_calc_len)) {
        warnx("HMAC calculation failed\n");

        return 1;
    }

    return 0;
}

int encode_base64(unsigned char *hmac_calc, unsigned int hmac_calc_len,
                  unsigned char **hmac_b64_calc, int *hmac_b64_calc_len)
{
    *hmac_b64_calc_len = EVP_ENCODE_LENGTH(hmac_calc_len);
    *hmac_b64_calc = (unsigned char *)malloc(*hmac_b64_calc_len + 1);

    if (!*hmac_b64_calc) {
        warnx("malloc failed for HMAC encoded in base 64\n");

        return 1;
    }

    *hmac_b64_calc_len = EVP_EncodeBlock(*hmac_b64_calc, hmac_calc,
                                          hmac_calc_len);
    if (*hmac_b64_calc_len <= 0) {
        warnx("EVP_EncodeBlock failed or returned zero length");

        return 1;
    }

    return 0;
}

int calc_and_encode_hmac(const char *content, int content_len,
                         unsigned char **hmac_b64_calc, int *hmac_b64_calc_len)
{
    unsigned char hmac_calc[EVP_MAX_MD_SIZE];
    unsigned int hmac_calc_len = 0;

    if (calculate_hmac((const unsigned char *)content, content_len,
                      hmac_calc, &hmac_calc_len)) {
        return 1;
    }
}

```

```

    if (encode_base64(hmac_calc, hmac_calc_len, hmac_b64_calc,
                      hmac_b64_calc_len)) {
        return 1;
    }

    return 0;
}

```

Código A.26: Cálculo local del *HMAC* y su codificación en *Base64*.

Finalmente, `verify_hmac` compara el *HMAC* extraído del fichero con el recalculado directamente en *Base64* mediante `CRYPTO_memcmp`. Todo este conjunto de funcionalidades es integrado en un flujo de trabajo coordinado en la propia función `main`, desde la apertura del fichero hasta la validación final del contenido autenticado:

```

int verify_hmac(const char *hmac_b64, int hmac_b64_len,
               unsigned char *hmac_b64_calc, int hmac_b64_calc_len)
{
    if (CRYPTO_memcmp(hmac_b64_calc, hmac_b64, hmac_b64_calc_len)) {
        warnx("\nHMAC verification failed: Invalid HMAC\n");

        return 1;
    }

    printf("\nHMAC verification successful: Valid HMAC\n");
    return 0;
}

int main(int argc, char *argv[])
{
    if (argc != 2) {
        errx(EXIT_FAILURE, "Usage: %s <path_file_handoff>\n"
            "e.g.: %s ./file_handoff", argv[0], argv[0]);
    }

    const char *path_fhandoff = argv[1];

    FILE *fhandoff = fopen(path_fhandoff, "r");
    if (!fhandoff) {
        warnx("fopen failed\n");

        goto err_fhandoff;
    }

    char *content = NULL;
    int content_len = 0;
    char *hmac_b64 = NULL;
    int hmac_b64_len = 0;

```

```

    if (extract_data(fhandoﬀ, &content, &content_len,
        &hmac_b64, &hmac_b64_len)) {
        goto free_cont;
    }

    fclose(fhandoﬀ);
    fhandoﬀ = NULL;

    printf("Extracted kernel content:\n%s\n", content);
    printf("Extracted HMAC(Base64):\n%s\n", hmac_b64);

    unsigned char *hmac_b64_calc = NULL;
    int hmac_b64_calc_len = 0;

    if (calc_and_encode_hmac(content, content_len,
        &hmac_b64_calc, &hmac_b64_calc_len)) {
        goto free_all;
    }
    free(content);
    content = NULL;

    printf("Calculated HMAC(Base64): \n%s\n", hmac_b64_calc);
    int ext = verify_hmac(hmac_b64, hmac_b64_len, hmac_b64_calc,
        hmac_b64_calc_len);

    free(hmac_b64);
    free(hmac_b64_calc);
    exit(ext);

free_all:
    if (hmac_b64_calc) {
        free(hmac_b64_calc);
    }
    if (hmac_b64) {
        free(hmac_b64);
    }

free_cont:
    if (content) {
        free(content);
    }

err_fhandoﬀ:
    if (fhandoﬀ) {
        fclose(fhandoﬀ);
    }

    exit(EXIT_FAILURE);
}

```

Código A.27: Comparación del autenticador y flujo principal del verificador.

Una vez completado el prototipo, se procedió a validar su correcto funcionamiento comprobando que el módulo podía generar correctamente contenido autenticado en *espacio del kernel*, escribirlo sobre el fichero recibido mediante el mecanismo de *File Handoff* y verificar posteriormente su integridad desde *espacio de usuario* con la misma clave secreta compartida entre el administrador y el *kernel*:

```
$ uname -r
6.12.41-current-bcm2711
$ cd LKM
$ chmod +x gen_k_embedded.sh
$ ./gen_k_embedded.sh
OK: generated k_embedded.h with key K (64 bytes).
$ cat k_embedded.h
const unsigned char K[] = {
    0x4a, 0xd3, 0x21, 0xda, 0xc3, 0x6c, 0x38, 0x13, 0xb1, 0x70, 0x48,
    0xac,
    0xb2, 0x6b, 0xf7, 0xe4, 0x1b, 0x7d, 0x56, 0xf2, 0xfe, 0x69, 0xb4,
    0x59,
    0x0e, 0x31, 0xd6, 0xa7, 0xf1, 0xee, 0x40, 0x8a, 0x17, 0x97, 0x4e,
    0xb0,
    0x7a, 0x40, 0x39, 0x40, 0x3f, 0x29, 0x27, 0xd8, 0xa9, 0x84, 0xda,
    0x01,
    0x12, 0x70, 0x23, 0xdd, 0x87, 0x02, 0x1b, 0xb9, 0xaf, 0x8c, 0xee,
    0xe2,
    0x90, 0x7c, 0x00, 0x10
};

const unsigned int K_len = 64;
$ make
make -C /lib/modules/6.12.41-current-bcm2711/build M=/home/LKM modules
make[1]: Entering directory '/usr/src/linux-headers-6.12.41-current-bcm2711'
  CC [M] /home/LKM/crypto_fd.o
  MODPOST /home/LKM/Module.symvers
  CC [M] /home/LKM/crypto_fd.mod.o
  LD [M] /home/LKM/crypto_fd.ko
make[1]: Leaving directory '/usr/src/linux-headers-6.12.41-current-bcm2711'
$ insmod crypto_fd.ko
$ cat /proc/fdev
Waiting for a FD to perform file handoff
$ dmesg -T | tail
...
[Fri Nov 14 18:25:38 2025] /proc/fdev created
...
[Fri Nov 14 18:28:52 2025] /proc/fdev: read handler
[Fri Nov 14 18:28:52 2025] /proc/fdev: 41 bytes sent to userspace
$ cd ..
$ gcc -c -Wshadow -Wvla -Wall mk_fhandoff.c
$ gcc -o mk_fhandoff mk_fhandoff.o
$ gcc -c -Wshadow -Wvla -Wall check_hmac.c
$ gcc -o check_hmac check_hmac.o -lcrypto
```

```

$ ./mk_fhandoff ./file_handoff /proc/fdev
File descriptor written in /proc/fdev:
3
$ dmesg -T | tail
...
[Fri Nov 14 18:31:22 2025] /proc/fdev: write handler
[Fri Nov 14 18:31:22 2025] printH: file has been written with HMAC
[Fri Nov 14 18:31:22 2025] mywrite: printH OK for fd 3 (119 bytes written)
$ cat ./file_handoff

--KERNEL-PS--
Kernel-Space: Authentic content protected by HMAC-SHA256

--HMAC--
xUPsVlgNMcm1Uxj2mac0uTD3gdkDTyeNvbXuBZjq6Bw=

$ ./check_hmac ./file_handoff
Extracted kernel content:
Kernel-Space: Authentic content protected by HMAC-SHA256

Extracted HMAC(Base64):
xUPsVlgNMcm1Uxj2mac0uTD3gdkDTyeNvbXuBZjq6Bw=
Calculated HMAC(Base64):
xUPsVlgNMcm1Uxj2mac0uTD3gdkDTyeNvbXuBZjq6Bw=

HMAC verification successful: Valid HMAC
$ cd LKM
$ rmmod crypto_fd
$ dmesg -T | tail
...
[Fri Nov 14 18:31:47 2025] /proc/fdev removed

```

Código A.28: Validación del mecanismo de autenticación mediante *HMAC-SHA256*.

Mediante esta validación se comprobó que el módulo podía generar correctamente contenido autenticado en *espacio del kernel*, adjuntarle su etiqueta *HMAC-SHA256* en formato *Base64* y transferir el conjunto completo al fichero real recibido mediante el mecanismo de *File Handoff*. Asimismo, el programa `check_hmac.c` permitió verificar desde *espacio de usuario* que la etiqueta de autenticación recibida en el fichero coincidía con la recalculada localmente a partir del contenido extraído y de la misma clave secreta compartida, quedando así correctamente validado el funcionamiento del mecanismo de autenticidad del contenido diseñado en este prototipo.

A.1.5. Sistema de reporte seguro de los procesos en ejecución del sistema

Por último, ya validado el mecanismo de autenticación del contenido a transferir mediante *HMAC-SHA256*, el siguiente paso consistió en sustituir la cadena fija de prueba por un contenido real y verdaderamente crítico para el sistema, generado dinámicamente en *espacio del kernel*. Para ello, se implementó un listado global de todos los procesos en ejecución del sistema, con un formato inspirado en la salida de *ps*, de forma que dicho contenido pudiese transferirse al fichero de transferencia de manera autenticada y verificarse posteriormente desde *espacio de usuario* del mismo modo que el prototipo anterior. Este módulo se diseñó en el fichero `crypto_ps.c`.

En este punto, toda la infraestructura desarrollada en los prototipos anteriores se reutiliza sin cambios sustanciales, incluyendo el mecanismo de *File Handoff*, la escritura conjunta del contenido y su *HMAC* en *Base64*, así como las utilidades de escritura en el fichero de `/proc` y verificación en *espacio de usuario*.

Inicialmente, se tuvo que incorporar nuevas constantes y cabeceras al módulo:

```
#include <linux/cred.h>
#include <linux/pid.h>
#include <linux/rcupdate.h>
#include <linux/security.h>
#include <linux/sched.h>
#include <linux/nsproxy.h>
#include <linux/pid_namespace.h>
enum {
    MAX_SIZE = 1024 * 40,
    TIPIC_HMACB64_SIZE = 44,
    UID_SIZE = 12,
    UIDNS_SIZE = 14,
    PID_SIZE = 12,
    PIDNS_SIZE = 14,
    GID_SIZE = 11,
    CMD_SIZE = 50,
    PS_LINE_SIZE = UID_SIZE + UID_SIZE + UIDNS_SIZE +
        PID_SIZE + PID_SIZE + PIDNS_SIZE + GID_SIZE + CMD_SIZE,
};
static const char header[] =
    "UID_KERNEL UID_LOCAL UID_NS      PID_KERNEL PID_LOCAL PID_NS
    GID      COMMAND\n";
static const int header_len = sizeof(header) - 1;
```

Código A.29: Cabeceras y constantes para la generación del listado de procesos.

Para poder construir un listado de todos los procesos que se encuentran en ejecución en el sistema y en sus distintos espacios de nombres, fue necesario ampliar nuevamente las cabeceras del módulo. Concretamente, `<linux/cred.h>` y `<linux/security.h>` permiten al módulo acceder a las credenciales de cada proceso, `<linux/pid.h>`, `<linux/sched.h>` y `<linux/pid_namespace.h>` aportan las definiciones necesarias para recorrer la lista global de tareas y obtener sus identificadores en los distintos espacios de nombres, mientras que `<linux/rcupdate.h>` proporciona el mecanismo de sincronización *RCU* necesario para realizar este recorrido de forma segura. Por otra parte, las constantes auxiliares definidas acotan el tamaño máximo del contenido generado, el tamaño reservado para cada campo serializado y la estructura básica de la tabla de procesos generada con sus metadatos correspondientes.

Una vez establecidas las cabeceras y constantes necesarias para estructurar el contenido a transferir, se procedió a incorporar dos funciones auxiliares que permitiesen formatear y construir dinámicamente y de forma segura la tabla completa de procesos dentro de un único *buffer* de salida:

```
static int pad_str_right(char *buf, int curr_len, int buf_len, char
    pad_char)
{
    int padding = 0;

    if (curr_len >= buf_len) {
        padding = 0;

        curr_len = buf_len;
    } else {
        padding = buf_len - curr_len;
    }

    if (padding > 0) {
        memset(buf + curr_len, pad_char, padding);

        curr_len += padding;
    }

    return buf_len;
}

static int safe_chunk(u8 *dst, int *current_len, char *src, int src_len)
{
    int max_len = MAX_SIZE - *current_len;
```

```

    if (max_len < src_len)
        return -ENOSPC;

    memcpy(dst + *current_len, src, src_len);

    *current_len += src_len;

    return src_len;
}

```

Código A.30: Funciones auxiliares para el formateo y la construcción segura del contenido serializado.

La función `pad_str_right` permite completar cada campo textual con caracteres de relleno hasta alcanzar una anchura fija, garantizando así una representación tabulada homogénea de los metadatos de cada proceso en la tabla.

Por su parte, `safe_chunk` permite incorporar secuencialmente cada fragmento de información al *buffer* global que contendrá la tabla final, comprobando previamente que exista espacio suficiente antes de realizar la copia.

A continuación, se implementó la función principal encargada de recorrer la lista global de tareas del sistema y construir la tabla completa en memoria:

```

static int get_ps(u8 **cont, int *cont_len)
{
    struct task_struct *task;

    *cont = (u8 *) kmalloc(MAX_SIZE, GFP_KERNEL);
    if (!*cont) {
        printk(KERN_ERR "get_ps: kmalloc failed\n");

        return -ENOMEM;
    }

    if (safe_chunk(*cont, cont_len, (char *)header, header_len) < 0)
        goto out_fail;

    rcu_read_lock();

    for_each_process(task) {
        if (*cont_len >= MAX_SIZE - PS_LINE_SIZE -
            sep_len - sep_hmac_len - TIPIC_HMACB64_SIZE) {
            printk(KERN_WARNING "get_ps: Buffer full -> Truncating\n");
            goto out_fail;
        }
    }
}

```

```

    if (ps_data(task, *cont, cont_len)) {
        printk(KERN_ERR "get_ps: Extracting process data failed\n");

        goto out_fail;
    }

}

rcu_read_unlock();

printk(KERN_INFO "get_ps: Generated ps-like output of %d bytes.\n",
        *cont_len);

return 0;

out_fail:
    rcu_read_unlock();

    if (*cont)
        kfree(*cont);

    *cont = NULL;
    *cont_len = 0;

    return -ENOSPC;
}

```

Código A.31: Construcción del listado global de procesos en memoria del *kernel*.

Este código actúa como el núcleo principal de generación del contenido a transferir, reservando inicialmente un *buffer* de tamaño máximo fijo donde se almacenará todo el listado completo, incorporando en primer lugar la cabecera definida anteriormente en las constantes.

Posteriormente, recorre la lista global de procesos activos del sistema utilizando la macro `for_each_process` bajo protección de lectura *RCU*, garantizando así un acceso seguro frente a múltiples accesos concurrentes a las estructuras críticas de `task_struct` mientras se extraen sus metadatos. Durante este recorrido, se comprueba que siga existiendo espacio disponible suficiente antes de incorporar cada nueva entrada, delegando en `ps_data` la consulta y serialización de los campos correspondientes a cada proceso.

Finalmente, si el recorrido concluye correctamente, devuelve en `cont` el contenido

completo generado, el cual será el utilizado en `printh`, y en `cont_len` su longitud total. Por otra parte, en caso de fallo, se libera la memoria reservada y se devuelve el código de error correspondiente.

```
static int ps_data(struct task_struct *task, u8 *cont, int *cont_len)
{
    char kuid_str[UID_SIZE];
    char luid_str[UID_SIZE];
    char uidns_str[UIDNS_SIZE];
    char kpid_str[PID_SIZE];
    char lpid_str[PID_SIZE];
    char pidns_str[PIDNS_SIZE];
    char gid_str[GID_SIZE];
    char comm_str[CMD_SIZE];

    int kuid_len = get_kuid_str(task_uid(task), kuid_str);
    if (kuid_len <= 0)
        goto out_fail;

    if (safe_chunk(cont, cont_len, kuid_str, UID_SIZE) < 0)
        goto out_fail;

    int luid_len = get_luid_str(task, luid_str);
    if (luid_len <= 0)
        goto out_fail;

    if (safe_chunk(cont, cont_len, luid_str, UID_SIZE) < 0)
        goto out_fail;

    int uidns_len = get_uidns_str(task, uidns_str);
    if (uidns_len <= 0)
        goto out_fail;

    if (safe_chunk(cont, cont_len, uidns_str, UIDNS_SIZE) < 0)
        goto out_fail;

    int kpid_len = get_kpid_str(task_pid_nr(task), kpid_str);
    if (kpid_len <= 0)
        goto out_fail;

    if (safe_chunk(cont, cont_len, kpid_str, PID_SIZE) < 0)
        goto out_fail;

    int lpid_len = get_lpid_str(task, lpid_str);
    if (lpid_len <= 0)
        goto out_fail;

    if (safe_chunk(cont, cont_len, lpid_str, PID_SIZE) < 0)
        goto out_fail;
```

```

int pidns_len = get_pidns_str(task, pidns_str);
if (pidns_len <= 0)
    goto out_fail;

if (safe_chunk(cont, cont_len, pidns_str, PIDNS_SIZE) < 0)
    goto out_fail;

int gid_len = get_gid_str(task, gid_str);
if (gid_len <= 0)
    goto out_fail;

if (safe_chunk(cont, cont_len, gid_str, GID_SIZE) < 0)
    goto out_fail;

int comm_len = get_command_str(task, comm_str);
if (comm_len <= 0)
    goto out_fail;

if (safe_chunk(cont, cont_len, comm_str, comm_len) < 0)
    goto out_fail;

return 0;

out_fail:
    return 1;
}

```

Código A.32: Serialización de los metadatos de un proceso.

La función `ps_data` encapsula la construcción de una única fila de la tabla final correspondiente a la tarea actualmente recorrida en `for_each_process`, extrayendo y serializando secuencialmente los principales metadatos de cada proceso concreto.

Para ello, se optó por encapsular en funciones independientes la extracción de cada metadato por separado:

A continuación, se implementaron varias funciones auxiliares encargadas de extraer y formatear los distintos identificadores de usuario asociados a cada proceso, distinguiendo entre el identificador global visible desde el *kernel*, su valor relativo dentro del espacio de nombres del proceso y el identificador del propio espacio de nombres:

```

static int get_kuid_str(kuid_t uid_struct, char *uid_str)
{
    int uid_len = 0;

    int uid = from_kuid(&init_user_ns, uid_struct);

```



```
uid_len = snprintf(uid_str, UID_SIZE, "%d", uid);

pad_str_right(uid_str, uid_len, UID_SIZE, ' ');

return uid_len;
}

static int get_luid_str(struct task_struct *task, char *uid_str)
{
    int uid_len = 0;

    const struct cred *cred = get_task_cred(task);
    if (!cred) {
        printk(KERN_ERR "get_luid_str: No credentials found for task\n");

        return -ENOSPC;
    }

    int uid = from_kuid(cred->user_ns, cred->uid);

    put_cred(cred);

    uid_len = snprintf(uid_str, UID_SIZE, "%d", uid);

    pad_str_right(uid_str, uid_len, UID_SIZE, ' ');

    return uid_len;
}

static int get_uidns_str(struct task_struct *task, char *uidns_str)
{
    const struct cred *cred = get_task_cred(task);
    if (!cred) {
        printk(KERN_ERR "get_uidns_str: No credentials found for task\n");

        return -ENOSPC;
    }

    unsigned int uid_ns = cred->user_ns->ns.inum;

    put_cred(cred);

    int uidns_len = snprintf(uidns_str, UIDNS_SIZE, "%u", uid_ns);

    pad_str_right(uidns_str, uidns_len, UIDNS_SIZE, ' ');

    return uidns_len;
}
```

Estas funciones permiten obtener y formatear los identificadores de usuario (*UIDs*) asociados a cada proceso. En concreto, `get_kuid_str` obtiene el UID global visible desde el *kernel*, `get_luid_str` el UID relativo al espacio de nombres del proceso y `get_uidns_str` el espacio de nombres de usuario al que pertenecen dichas credenciales.

```
static int get_kpid_str(int pid, char *pid_str)
{
    int pid_len = snprintf(pid_str, PID_SIZE, "%d", pid);

    pad_str_right(pid_str, pid_len, PID_SIZE, ' ');

    return pid_len;
}

static int get_lpid_str(struct task_struct *task, char *pid_str)
{
    struct pid_namespace *ns = task_active_pid_ns(task);
    if (!ns) {
        printk(KERN_ERR "get_lpid_str: No PID namespace found for task\n");
        return -ENOSPC;
    }

    int pid = task_pid_nr_ns(task, ns);

    int pid_len = snprintf(pid_str, PID_SIZE, "%d", pid);

    pad_str_right(pid_str, pid_len, PID_SIZE, ' ');

    return pid_len;
}

static int get_pidns_str(struct task_struct *task, char *pidns_str)
{
    struct pid_namespace *pid_ns = task_active_pid_ns(task);
    if (!pid_ns) {
        printk(KERN_ERR "get_pidns_str: No PID namespace found for task\n");
        return -ENOSPC;
    }

    int pidns_len = snprintf(pidns_str, PIDNS_SIZE, "%u", pid_ns->ns.inum);

    pad_str_right(pidns_str, pidns_len, PIDNS_SIZE, ' ');

    return pidns_len;
}
```

Código A.34: Identificadores de proceso de cada tarea.

En este caso, estas funciones permiten obtener y formatear los identificadores de proceso (*PIDs*) de cada tarea. En particular, `get_kpid_str` obtiene el PID global visible desde el *kernel*, `get_lpid_str` recupera el PID relativo al espacio de nombres de proceso activo y `get_pidns_str` identifica el espacio de nombres de proceso al que pertenece la tarea analizada.

```
static int get_gid_str(struct task_struct *task, char *gid_str)
{
    int gid_len = 0;

    const struct cred *cred = get_task_cred(task);
    if (!cred)
        return -ENOSPC;

    int gid = from_kgid(&init_user_ns, cred->gid);
    put_cred(cred);

    gid_len = snprintf(gid_str, GID_SIZE, "%d", gid);

    pad_str_right(gid_str, gid_len, GID_SIZE, ' ');

    return gid_len;
}

static int get_command_str(struct task_struct *task, char *comm_str)
{
    int comm_len = 0;

    if (task->mm == NULL) {
        comm_len = snprintf(comm_str, CMD_SIZE, "[%s]\n", task->comm);

        goto commd_finished;
    }

    struct file *exe_file = task->mm->exe_file;

    if (exe_file) {
        char *path = d_path(&exe_file->f_path, comm_str, CMD_SIZE);

        if (!IS_ERR(path)) {
            comm_len = snprintf(comm_str, CMD_SIZE, "%s\n", path);

            goto commd_finished;
        }
    }
}
```

```

    comm_len = snprintf(comm_str, CMD_SIZE, "%s\n", task->comm);

commd_finished:
    if (comm_len >= CMD_SIZE)
        comm_len = CMD_SIZE - 1;

    return comm_len;
}

```

Código A.35: Identificadores de grupo y nombre del comando asociado a cada proceso.

Finalmente, estas funciones completan la información del proceso con el identificador de grupo y el comando asociado a su ejecución. Concretamente, `get_gid_str` obtiene y formatea el identificador de grupo global asociado al proceso y `get_command_str` recupera el comando correspondiente a su ejecución, distinguiendo entre procesos de usuario con ejecutable accesible, hilos del *kernel* y casos en los que solo puede utilizarse el nombre almacenado en `task->comm`.

Una vez integrada la generación dinámica de los metadatos de los procesos activos del sistema en el flujo autenticado de escritura en el fichero de transferencia, se procedió a validar el comportamiento completo del prototipo final incorporando la funcionalidad de los contenedores para simular distintos espacios de nombres en el sistema:

```

$ uname -r
6.12.41-current-bcm2711
$ cd LKM
$ chmod +x gen_k_embedded.sh
$ ./gen_k_embedded.sh
OK: generated k_embedded.h with key K (64 bytes).
$ make
make -C /lib/modules/6.12.41-current-bcm2711/build M=/home/LKM modules
make[1]: Entering directory '/usr/src/linux-headers-6.12.41-current-bcm2711'
  CC [M] /home/LKM/crypto_ps.o
  MODPOST /home/LKM/Module.symvers
  CC [M] /home/LKM/crypto_ps.mod.o
  LD [M] /home/LKM/crypto_ps.ko
make[1]: Leaving directory '/usr/src/linux-headers-6.12.41-current-bcm2711'
$ insmod crypto_ps.ko
$ cat /proc/fdev
LKM ready to receive file descriptors from user.
$ dmesg -T | tail
...
[Thu Nov 27 18:42:11 2025] New proc file created: /proc/fdev
..
[Thu Nov 27 18:42:11 2025] /proc/fdev: read handler
[Thu Nov 27 18:42:11 2025] /proc/fdev myread: read 48 bytes by userspace
$ cd ..
$ gcc -c -Wshadow -Wvla -Wall mk_fhandooff.c
$ gcc -o mk_fhandooff mk_fhandooff.o
$ gcc -g -c -Wshadow -Wvla -Wall check_hmac.c
$ gcc -g -o check_hmac check_hmac.o -lcrypto
$ cat /etc/docker/daemon.json
{
    "userns-remap": "default"
}
$ systemctl restart docker
$ docker run -d --name docker1 debian:stable sleep 10000
7fd1c45a07c0...
$ docker run -d --name docker2 debian:stable sleep 10000
77276c1a7026...
$ docker run -d --name docker3 debian:stable sleep 10000
4a4421829e80...
$ docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        NAMES
4a4421829e80   debian:stable "sleep 10000"           29 seconds ago Up 28 seconds docker3
77276c1a7026   debian:stable "sleep 10000"           35 seconds ago Up 34 seconds docker2
7fd1c45a07c0   debian:stable "sleep 10000"           About a minute ago Up 58 seconds docker1
$ ./mk_fhandooff ./file_handooff /proc/fdev
File descriptor written in /proc/fdev:
3
$ dmesg -T | tail
...
[Thu Nov 27 18:43:49 2025] /proc/fdev: write handler

```

```

[Thu Nov 27 18:43:49 2025] /proc/fdev write: written 1 bytes from the user
[Thu Nov 27 18:43:49 2025] get_ps: Generated ps-like output of 21251 bytes.
[Thu Nov 27 18:43:49 2025] printH: file has been written with HMAC
[Thu Nov 27 18:43:49 2025] mywrite: printH OK for fd 3 (21320 bytes written)
$ cat ./file_handoff

--KERNEL-PS--
UID_KERNEL UID_LOCAL UID_NS PID_KERNEL PID_LOCAL PID_NS GID COMMAND
0 0 4026531837 1 1 4026531836 0 /usr/lib/systemd/systemd
0 0 4026531837 2 2 4026531836 0 [kthreadd]
0 0 4026531837 3 3 4026531836 0 [pool_workqueue_]
...
113 113 4026531837 1440 1440 4026531836 119 /usr/sbin/chronyd
113 113 4026531837 1448 1448 4026531836 119 /usr/sbin/chronyd
0 0 4026531837 1484 1484 4026531836 0 /usr/sbin/gdm3
0 0 4026531837 1489 1489 4026531836 1000 /usr/libexec/gdm-session-worker
1000 1000 4026531837 1496 1496 4026531836 1000 /usr/lib/systemd/systemd
1000 1000 4026531837 1497 1497 4026531836 1000 /usr/lib/systemd/systemd-executor
...
0 0 4026531837 8211 8211 4026531836 0 /usr/bin/dockerd
0 0 4026531837 8212 8212 4026531836 0 [kworker/1:1]
0 0 4026531837 8239 8239 4026531836 0 [kworker/1:2H]
0 0 4026531837 8500 8500 4026531836 0 /usr/bin/containerd-shim-runc-v2
100000 0 4026532674 8520 1 4026532678 100000 /usr/bin/sleep
0 0 4026531837 8532 8532 4026531836 0 [kworker/2:0]
0 0 4026531837 8596 8596 4026531836 0 /usr/bin/containerd-shim-runc-v2
100000 0 4026532754 8617 1 4026532758 100000 /usr/bin/sleep
0 0 4026531837 8667 8667 4026531836 0 [kworker/2:2H]
0 0 4026531837 8678 8678 4026531836 0 /usr/bin/containerd-shim-runc-v2
100000 0 4026532834 8698 1 4026532838 100000 /usr/bin/sleep
0 0 4026531837 8741 8741 4026531836 0 [kworker/u20:2]
0 0 4026531837 8935 8935 4026531836 0 [kworker/0:0]
0 0 4026531837 8947 8947 4026531836 0 [kworker/u16:3]
0 0 4026531837 8989 8989 4026531836 0 [kworker/3:2]
0 0 4026531837 9033 9033 4026531836 0 /home/noel/Desktop/TFG/Project/mk_fhandoff

--HMAC--
ckvLUckcgmt6RFfdGzuvgNAEgt9rLPJ5q9FYJoFKAW0=

$ ./check_hmac ./file_handoff
Extracted kernel content:
UID_KERNEL UID_LOCAL UID_NS PID_KERNEL PID_LOCAL PID_NS GID COMMAND
0 0 4026531837 1 1 4026531836 0 /usr/lib/systemd/systemd
0 0 4026531837 2 2 4026531836 0 [kthreadd]
0 0 4026531837 3 3 4026531836 0 [pool_workqueue_]
...
113 113 4026531837 1440 1440 4026531836 119 /usr/sbin/chronyd
113 113 4026531837 1448 1448 4026531836 119 /usr/sbin/chronyd
0 0 4026531837 1484 1484 4026531836 0 /usr/sbin/gdm3
0 0 4026531837 1489 1489 4026531836 1000 /usr/libexec/gdm-session-worker
1000 1000 4026531837 1496 1496 4026531836 1000 /usr/lib/systemd/systemd
1000 1000 4026531837 1497 1497 4026531836 1000 /usr/lib/systemd/systemd-executor
...
0 0 4026531837 8211 8211 4026531836 0 /usr/bin/dockerd
0 0 4026531837 8212 8212 4026531836 0 [kworker/1:1]
0 0 4026531837 8239 8239 4026531836 0 [kworker/1:2H]
0 0 4026531837 8500 8500 4026531836 0 /usr/bin/containerd-shim-runc-v2
100000 0 4026532674 8520 1 4026532678 100000 /usr/bin/sleep
0 0 4026531837 8532 8532 4026531836 0 [kworker/2:0]
0 0 4026531837 8596 8596 4026531836 0 /usr/bin/containerd-shim-runc-v2
100000 0 4026532754 8617 1 4026532758 100000 /usr/bin/sleep
0 0 4026531837 8667 8667 4026531836 0 [kworker/2:2H]
0 0 4026531837 8678 8678 4026531836 0 /usr/bin/containerd-shim-runc-v2
100000 0 4026532834 8698 1 4026532838 100000 /usr/bin/sleep
0 0 4026531837 8741 8741 4026531836 0 [kworker/u20:2]
0 0 4026531837 8935 8935 4026531836 0 [kworker/0:0]
0 0 4026531837 8947 8947 4026531836 0 [kworker/u16:3]
0 0 4026531837 8989 8989 4026531836 0 [kworker/3:2]
0 0 4026531837 9033 9033 4026531836 0 /home/noel/Desktop/TFG/Project/mk_fhandoff

Extracted HMAC(Base64):
ckvLUckcgmt6RFfdGzuvgNAEgt9rLPJ5q9FYJoFKAW0=
Calculated HMAC(Base64):
ckvLUckcgmt6RFfdGzuvgNAEgt9rLPJ5q9FYJoFKAW0=

HMAC verification successful: Valid HMAC
$ cd LKM
$ rmmod crypto_ps
$ dmesg -T | tail
...
[Thu Nov 27 18:46:06 2025] Proc file deleted: /proc/fdev

```

Código A.36: Validación del sistema de reporte seguro de metadatos de los procesos activos de manera autenticada mediante *HMAC-SHA256*.

Cabe destacar que no se muestran los resultados completos de `cat ./file_handoff` y `./check_hmac ./file_handoff` debido a la gran cantidad de procesos activos en el sistema, lo que genera una tabla de gran extensión. Por este motivo, en la validación se presenta únicamente un extracto representativo del contenido transferido y verificado, correspondiéndose el *HMAC* y los tamaños reportados con la salida completa generada durante la ejecución real.

Mediante esta validación se comprobó que el módulo podía generar correctamente un listado global de los procesos activos del sistema desde *espacio del kernel*. Asimismo, se comprobó que en dicho listado también se incluían aquellos procesos que se ejecutan en otros *Namespaces*, como se observó mediante la ejecución de varios contenedores con *Docker*, observándose diferencias respecto a los metadatos del resto de procesos, principalmente en los valores relativos a `UID_LOCAL`, `PID_LOCAL`, `UID_NS` y `PID_NS`.

Por otro lado, al igual que en el anterior prototipo, se valida correctamente desde *espacio de usuario* mediante el programa `check_hmac.c` que la etiqueta *HMAC-SHA256* en *Base64* coincide con la recalculada respecto a la tabla completa de procesos, garantizando así un reporte seguro y autenticado de datos críticos del sistema entre las dos regiones del mismo.

Bibliografía

- [1] Schneier, Bruce, “A Plea for Simplicity,” Schneier on Security, online essay, November 1999. [Online]. Available: https://www.schneier.com/essays/archives/1999/11/a_plea_for_simplicit.html
- [2] Souppaya, Murugiah and Scarfone, Karen, “Guide to Malware Incident Prevention and Handling for Desktops and Laptops,” National Institute of Standards and Technology (NIST), Special Publication (SP), July 2013. [Online]. Available: <https://csrc.nist.gov/pubs/sp/800/83/r1/final>
- [3] Akamai Technologies, “¿Qué es el malware?” Artículo web, visto el 10 de marzo de 2026. [Online]. Available: <https://www.akamai.com/es/glossary/what-is-malware>
- [4] Fonticons, Inc., “Font Awesome Free,” Sitio web, visto el 13 de marzo de 2026. [Online]. Available: <https://fontawesome.com/>
- [5] MITRE ATT&CK, “MITRE ATT&CK,” Fuente oficial de información sobre tácticas y técnicas de adversarios, The MITRE Corporation, Tech. Rep., 2026, visto el 26 de marzo de 2026. [Online]. Available: <https://attack.mitre.org/>
- [6] European Union Agency for Cybersecurity (ENISA), “ENISA Threat Landscape 2025,” ENISA, Tech. Rep., October 2025, visto el 10 de marzo de 2026. [Online]. Available: <https://www.enisa.europa.eu/publications/enisa-threat-landscape-2025>
- [7] Joint Task Force, “Security and Privacy Controls for Information Systems and Organizations,” National Institute of Standards and Technology (NIST), Special Publication (SP), September 2020. [Online]. Available: <https://csrc.nist.gov/publications/detail/sp/800-53/rev-5/final>
- [8] Scarfone, Karen and Mell, Peter, “Guide to Intrusion Detection and Prevention Systems (IDPS),” National Institute of Standards and Technology (NIST), Special Publication (SP), February 2007. [Online]. Available: <https://csrc.nist.gov/publications/detail/sp/800-94/final>

- [9] Soriano-Salvador, Enrique and Guardiola Múzquiz, Gorka and González Gómez, Juan, “User-space library rootkits revisited: Are user-space detection mechanisms futile?” Universidad Rey Juan Carlos, Tech. Rep., June 2025. [Online]. Available: <https://arxiv.org/abs/2506.07827>
- [10] National Institute of Standards and Technology (NIST), “Rootkit,” Glosario de ciberseguridad, visto el 14 de marzo de 2026. [Online]. Available: <https://csrc.nist.gov/glossary/term/rootkit>
- [11] Guardiola Múzquiz, Gorka and Soriano Salvador, Enrique, *Fundamentos de Sistemas Operativos: Una aproximación práctica usando Linux*, 3rd ed. GSyC, Universidad Rey Juan Carlos, November 2025. [Online]. Available: <https://honecomp.github.io/librossoo.html>
- [12] VARGUX, “File:Unix history-simple es.svg,” Vectorial file SVG, June 2023, publicado el 26 de junio de 2023. Licencia CC BY-SA 4.0. Visto el 13 de marzo de 2026. [Online]. Available: https://commons.wikimedia.org/wiki/File:Unix_history-simple_es.svg
- [13] Colaboradores de Wikipedia, “Anillo (seguridad informática),” Wikipedia, online article, January 2026. [Online]. Available: [https://es.wikipedia.org/wiki/Anillo_\(seguridad_inform%C3%A1tica\)](https://es.wikipedia.org/wiki/Anillo_(seguridad_inform%C3%A1tica))
- [14] Love, Robert, *Linux Kernel Development*, 3rd ed. Developer’s Library, Addison-Wesley, 2010, ch. 1, 13. [Online]. Available: <https://github.com/swadhinsekhar/books/blob/master/Linux%20Kernel%20Development%203rd%20Edition%20-%20Love%20-%202010.pdf>
- [15] Silberschatz, Abraham and Galvin, Peter Baer and Gagne, Greg, *Fundamentos de sistemas operativos*, 9th ed. Madrid, España: McGraw-Hill Interamericana, 2013, ch. 9. [Online]. Available: <https://books.google.es/books?id=b2qynQEACAAJ>
- [16] Albritton, William McDaniel, “Module 2, Session 7 - Linux File System Structure,” University of Hawaii, Tech. Rep., 2016, visto el 24 de febrero de 2026. [Online]. Available: https://tldp.org/LDP/intro-linux/html/chap_03.html
- [17] Gmkziz, “Understanding Linux Namespaces,” Hackerstack, online article, 2024, visto el 25 de febrero de 2026. [Online]. Available: <https://www.hackerstack.org/understanding-linux-namespaces/>

- [18] *namespaces(7)* - *Linux Programmer's Manual*, Linux Kernel Organization, January 2024, disponible ejecutando `man 7 namespaces` en un terminal Linux o en la documentación oficial del Kernel. Vista la última actualización. [Online]. Available: <https://man7.org/linux/man-pages/man7/namespaces.7.html>
- [19] Docker, Inc., “What is Docker?” Docker Docs, Tech. Rep., 2026, vista el 13 de marzo de 2026. [Online]. Available: <https://docs.docker.com/get-started/docker-overview/>
- [20] —, “Docker Brand Guidelines and Logo Kit,” Página oficial de recursos de marca, 2026, kit oficial de logotipos. Visto el 13 de marzo de 2026. [Online]. Available: <https://www.docker.com/company/newsroom/media-resources/>
- [21] GeeksforGeeks, *Linux/Unix — Linux Kernel Module Programming (Hello World Program)*, Online article, January 2019, visto el 7 de marzo de 2026. [Online]. Available: <https://www.geeksforgeeks.org/linux-unix/linux-kernel-module-programming-hello-world-program/>
- [22] Deeppadmani, “Understanding the /proc file system in Linux,” Artículo online, February 2025, visto el 8 de marzo de 2026. [Online]. Available: <https://medium.com/@deeppadmani98.2021/understanding-the-proc-file-system-in-linux-90746e3ba76a>
- [23] Raspberry Pi Ltd, “Raspberry Pi 4 Model B specifications,” Raspberry Pi, Tech. Rep., 2019, visto el 23 de marzo de 2026. [Online]. Available: <https://www.raspberrypi.com/products/raspberry-pi-4-model-b/specifications/>
- [24] —, “Raspberry Pi 4,” Raspberry Pi, Tech. Rep., 2019, visto el 23 de marzo de 2026. [Online]. Available: <https://www.raspberrypi.com/products/raspberry-pi-4-model-b/>
- [25] Armbian Community, “Armbian Documentation,” Armbian Documentation, Tech. Rep., 2026, visto el 23 de marzo de 2026. [Online]. Available: <https://docs.armbian.com/>
- [26] Prima produkcija, “Raspberry Pi - Armbian,” Armbian Documentation, Tech. Rep., 2026, visto el 23 de marzo de 2026. [Online]. Available: <https://www.armbian.com/rpi4b/>

- [27] Armbian Community, “Armbian Linux branding material,” Repositorio oficial en GitHub, 2026, visto el 23 de marzo de 2026. [Online]. Available: <https://github.com/armbian/stationery/blob/master/logo/PDF.PDF>
- [28] NIST CSRC, “Rootkit - Glossary,” Computer Security Resource Center based on NIST SP 800-83 Rev. 1 under Rootkit resource, visto el 30 de marzo de 2026. [Online]. Available: <https://csrc.nist.gov/glossary/term/rootkit>
- [29] Marier, Francois and Gouin, Patrick, *unhide-linux(8) — unhide — forensic tool to find hidden processes*, Debian Manpages, January 2017, visto el 26 de marzo de 2026. [Online]. Available: <https://manpages.debian.org/testing/unhide/unhide-linux.8.en.html>
- [30] Debian Manpages, “chkrootkit(8) — Scan the system for signs of rootkits,” 2026, visto el 27 de marzo de 2026. [Online]. Available: <https://manpages.debian.org/testing/chkrootkit/chkrootkit.8.en.html>
- [31] K. Thompson, “Reflections on Trusting Trust,” *Communications of the ACM*, vol. 27, no. 8, pp. 761–763, August 1984. [Online]. Available: <https://doi.org/10.1145/358198.358210>
- [32] Parmar, Kelvin, “Deep Dive into Linux Namespaces: Building Your Own Containers,” Hashnode, online article, August 2024, visto el 2 de marzo de 2026. [Online]. Available: <https://kelvin-parmar.hashnode.dev/deep-dive-into-linux-namespaces-building-your-own-containers>